



Universidad de Extremadura

Escuela Politécnica

Grado en Ingeniería Informática en Ingeniería de Software

Trabajo Fin de Grado

Agente basado en LLMs para asistir al investigador en la revisión y redacción de la sección de trabajos relacionados de artículos científicos

Sergio Durán Pérez

Convocatoria Septiembre, 2026





Universidad de Extremadura

Escuela Politécnica

Grado en Ingeniería Informática en
Ingeniería de Software

Trabajo Fin de Grado

Agente basado en LLMs para asistir al investigador en la revisión y redacción de la sección de trabajos relacionados de artículos científicos

Autor: Sergio Durán Pérez

Tutor: José Ángel Barriga Corchero

Co-Tutor: Pedro José Clemente Martín

Resumen

La redacción de la sección Related Works de un artículo científico es una de las tareas más exigentes del proceso investigador: requiere localizar, analizar y sintetizar la literatura previa relevante, identificar el vacío (gap) que el propio trabajo viene a cubrir, y posicionar la contribución propia frente al estado del arte existente, todo ello con un elevado nivel de rigor académico. Este Trabajo de Fin de Grado presenta el diseño, desarrollo y validación de un agente basado en Modelos de Lenguaje de Gran Escala (LLM) que asiste al investigador en la revisión y redacción de esta sección, automatizando gran parte del proceso sin renunciar a la supervisión humana en los puntos de decisión críticos. El sistema se ha implementado como un grafo de estados con LangGraph, que orquesta una arquitectura de dos niveles de modelo (uno ligero para tareas sencillas y otro más potente para tareas más complejas y pesadas). Este agente guía al investigador a través de un flujo conversacional desde la ingesta y análisis de los documentos PDF de los trabajos relacionados, pasando por su clasificación en categorías temáticas y el diseño de una tabla comparativa de capacidades, hasta la redacción final de la introducción, el cuerpo y la conclusión de la sección, ensamblados automáticamente en un documento LaTeX listo para su integración en el artículo del investigador. A lo largo de todo el proceso, el sistema incorpora mecanismos de confirmación y modificación que permiten al investigador validar o corregir cada propuesta del agente antes de avanzar. La herramienta se ha validado mediante un caso de uso real end-to-end y una batería de pruebas específicas (tolerancia a fallos en la ingesta de documentos, comparación de configuraciones de modelo y análisis de escalabilidad), cuyos resultados confirman que el sistema cumple los objetivos planteados y aporta un valor real al proceso de escritura científica, si bien también permiten identificar

limitaciones concretas que se documentan como líneas de trabajo futuro.
Palabras clave: Inteligencia Artificial, Agentes basados en LLM, Modelos de Lenguaje de Gran Escala, Agentic AI, Related Works, LangGraph.

Abstract

Writing the Related Works section of a scientific paper is one of the most demanding tasks in the research process: it requires locating, analyzing, and synthesizing the relevant prior literature, identifying the gap that the work itself aims to fill, and positioning the author's own contribution against the existing state of the art, all while maintaining a high standard of academic rigor. This Bachelor's Thesis presents the design, development, and validation of a Large Language Model (LLM) based agent that assists researchers in reviewing and drafting this section, automating most of the process while preserving human oversight at critical decision points. The system is implemented as a state graph using LangGraph, orchestrating a two-tier model architecture (a lightweight model for simple tasks and a more capable one for more complex and heavier tasks). This agent guides the researcher through a conversational workflow from the ingestion and analysis of the related works' PDF documents, through their classification into thematic categories and the design of a comparative capability table, to the final drafting of the section's introduction, body, and conclusion, automatically assembled into a LaTeX document ready to be integrated into the researcher's paper. Throughout this process, the system incorporates confirmation and modification mechanisms that allow the researcher to validate or correct each proposal made by the agent before proceeding. The tool was validated through a real end-to-end use case and a dedicated set of tests (fault tolerance during document ingestion, comparison of model configurations, and scalability analysis), whose results confirm that the system meets its stated objectives and provides real value to the scientific writing process, while also revealing specific limitations that are documented as future work.

Keywords: Artificial Intelligence, LLM-based Agents, Large Language Models, Agentic AI, Related Works, LangGraph.

Índice general

1. Introducción **1**

1.1. Contextualización 1

1.2. Motivación 3

1.3. Objetivos y Contribuciones 6

1.4. Planificación 9

1.5. Estructura del TFG 10

2. Estado del Arte **12**

2.1. Fundamentos: LLMs y sus Dos Limitaciones Críticas 13

2.1.1. Origen y Definición Fundamental 13

2.1.2. Clasificación y Tipos de Modelos 14

2.1.3. Beneficios y Limitaciones en Entornos Complejos 16

2.2. Agentes basados en LLM y Orquestación 17

2.2.1. Paradigma y Componentes de un Agente basado en LLM 17

2.2.2. Adquisición de Capacidades 19

2.2.3. Sistemas Multiagente: ¿Por qué no? 19

2.2.4. Ecosistema de Desarrollo: Inferencia Local y Orquestación 21

2.3. Generación Asistida de Revisiones de Literatura 23

2.3.1. Generación Autónoma de Surveys Científicos 24

2.3.2. Sistemas Paper-Centric 24

2.3.3. Enfoques Human-in-the-Loop e Interactivos 25

2.3.4. Síntesis Comparativa y Research Gap 28

3. Diseño **29**

3.1. Elicitación de Requisitos 30

3.1.1. Definición de Requisitos 30

3.1.2. Diagramas de Casos de Uso y de Secuencia 36

3.2. Arquitectura de Software 39

3.2.1. Vista de Alto Nivel 40

3.2.2. El Estado Compartido del Agente 42

3.2.3. Vista del Grafo de Estados 43

3.2.4. Justificación de las Decisiones Arquitectónicas 45

4. Implementación y desarrollo **49**

4.1. Metodología de Trabajo 50

4.2. Visión General del Sistema 52

4.3. Capa de Configuración e Ingesta de Datos 54

4.3.1. Seleccionar Idioma y Definición de Temática 54

4.3.2. Búsqueda y Lectura de Documentos PDF 57

4.4. Capa de Análisis 58

4.5. Capa de Clasificación 61

4.5.1. Recomendación y Decisión 62

4.5.2. Propuesta, Confirmación y Modificación 62

4.6. Capa de Redacción 64

4.7. Capa de Tabla Comparativa 66

4.7.1. El Mecanismo de Anti-Repetición 67

4.7.2. Generación de la Tabla y Descripción 69

4.8. Capa de Cierre 70

4.8.1. Redacción de Conclusión 70

4.8.2. Creación del Documento LaTeX 70

4.8.3. Red de Seguridad Post-Generación 72

4.9. Capa de Orquestación 74

4.9.1. Definición del Grafo 74

4.9.2. Persistencia del Estado 74
4.9.3. Bucle Principal de Ejecución 75
4.9.4. Motor de Configuración de Modelos 78
4.10. Interfaz de Usuario 82
4.10.1. Página de Chat 82
4.10.2. Página de Ajustes 84
4.11. Funciones Auxiliares 87

5. Caso de Uso **89**
5.1. Definición del Caso 90
5.2. Ejecución de la Herramienta 92
5.3. Resultados 104
5.3.1. Introducción de Related Works 105
5.3.2. Cuerpo de Related Works 106
5.3.3. Descripción y Tabla Comparativa 107
5.3.4. Conclusión 109
5.4. Detalles y Hallazgos Adicionales del Caso de Uso 110
5.4.1. Comparación de estrategias de categorización 110
5.4.2. Confirmación de una decisión de diseño: redacción intermedia en español y traducción final 111
5.4.3. El LLM complejo resolviendo fallos del LLM simple 112
5.4.4. Otros detalles destacados 113
5.4.5. Síntesis 114

6. Pruebas Adicionales **116**
6.1. Bucle de Modificación (Categorías y Tabla) 117
6.1.1. Objetivo y Metodología 117
6.1.2. Resultados: Categorización 118
6.1.3. Resultados: Tabla Comparativa 120
6.1.4. Discusión 122

6.2. Tolerancia a Fallos en la Ingesta . . . 123
6.2.1. Directorio Vacío . . . 124
6.2.2. Directorio Inexistente . . . 125
6.2.3. Documento PDF Corrupto . . . 126
6.2.4. Archivos No-PDF . . . 128
6.2.5. Discusión . . . 130
6.3. Comparación de Configuraciones de Modelos . . . 131
6.3.1. Objetivo y Metodología . . . 131
6.3.2. Resultados: Fidelidad Estructural y de Formato . . . 132
6.3.3. Resultados: Completitud del Contenido . . . 133
6.3.4. Resultados: Tiempos de Ejecución . . . 134
6.3.5. Discusión . . . 135
6.4. Escalabilidad . . . 135
6.4.1. Objetivo y Metodología . . . 135
6.4.2. Resultados: Tiempos de Ejecución . . . 137
6.4.3. Resultados: Categorización y Tabla Comparativa . . . 138
6.4.4. Resultados: Errores de Extracción . . . 138
6.4.5. Discusión . . . 139

7. Conclusiones y trabajos futuros **140**
7.1. Repaso de Objetivos . . . 140
7.2. Conclusiones . . . 144
7.3. Trabajos Futuros . . . 145

8. Reflexión Personal **150**

Anexos **153**

A. Prompts Completos de los Nodos Principales del Agente **154**
A.1. Análisis de Documentos (`analizar_trabajos_node`) . . . 155
A.2. Propuesta de Categorías (`proponer_categorias_node`) . . . 155

A.3. Propuesta de Estructura de la Tabla Comparativa (`proponer_estructura_node`) . . . 157
A.4. Redacción de la Introducción (`redactar_introduccion_node`) . . . 161
A.5. Redacción del Cuerpo de Trabajos Relacionados (`redactar_trabajos_relacionados_node`) . . . 162
A.6. Redacción de la Conclusión (`redactar_conclusion_node`) . . . 164
A.7. Ensamblado y Revisión Final (`revision_final_node`) . . . 167

B. Documento `related_works.tex` Generado en el Caso de Uso 174
B.1. Introducción de Related Works . . . 175
B.2. Cuerpo de Related Works . . . 175
B.3. Descripción de Tabla Comparativa . . . 180
B.4. Conclusión . . . 181
B.5. Bibliografía . . . 182
B.6. Tabla Comparativa . . . 183

Bibliografía 185

Índice de tablas

1.1. Planificación temporal estimada del proyecto. 9

2.1. Comparativa de frameworks de orquestación para agentes basados en LLM. 23

2.2. Comparativa de sistemas de generación/asistencia a la revisión de literatura. 28

3.1. Bloques funcionales del estado compartido `AgentState`. 42

B.1. Comparative analysis of related works and SimulateIoT. 184

Índice de figuras

3.1. Diagrama de casos de uso del sistema. 37
3.2. Diagrama de secuencia del flujo completo sin incidencias. 38
3.3. Diagrama de secuencia del bucle de propuesta, rechazo y modificación. . . 39
3.4. Vista de alto nivel de la arquitectura del sistema. 41
3.5. Vista interna del grafo de estados del agente. 44

5.1. Pantalla de Inicio de la interfaz web. 92
5.2. Gestión de documentos PDF desde la página de Ajustes. 92
5.3. Configuración de los dos *slots* de modelo LLM (simple y complejo). 93
5.4. Verificación de la clave de API de Google en la página de Ajustes. 93
5.5. Mensaje de bienvenida y selección del idioma de salida. 94
5.6. Confirmación del idioma seleccionado y solicitud de la descripción del artículo propio. 94
5.7. Descripción en lenguaje natural del artículo propio introducida por el investigador. 95
5.8. Ficha técnica estructurada generada para el artículo propio (SimulateIoT). 95
5.9. Localización de los 14 documentos PDF de trabajos relacionados. 96
5.10. Extracción de texto e inicio del análisis de los 14 trabajos relacionados. . . 96
5.11. Confirmación de que la extracción se completó correctamente. 97
5.12. Fichas técnicas estructuradas generadas para los 14 trabajos relacionados. . 97
5.13. Recomendación emitida por `evaluar_categorizacion_node` sobre la conveniencia de categorizar. 98
5.14. Aceptación de la recomendación de categorización por parte del investigador. 98

5.15. Propuesta de dos categorías temáticas y menú de acciones sobre la propuesta. 99
5.16. Avance automático del grafo hacia la redacción de la introducción. 99
5.17. Redacción de la introducción de la sección Related Works. 100
5.18. Trabajos relacionados redactados y avance hacia Tabla Comparativa. . . . 100
5.19. Recomendación emitida por `recomendar_tabla_node` sobre la inclusión de una tabla comparativa. 100
5.20. Inicio del subproceso de diseño de la tabla comparativa. 101
5.21. Propuesta de estructura de la tabla comparativa (8 columnas justificadas). 101
5.22. Aceptación de la estructura e inicio de la generación del contenido de la tabla. 102
5.23. Tabla comparativa completa generada por el sistema. 102
5.24. Descripción de la tabla generada. 103
5.25. Párrafo de conclusión generado. 103
5.26. Confirmación de finalización del proceso y guardado de los ficheros de sesión. 104

6.1. Categoría 1 de la propuesta inicial (*first try*). 118
6.2. Categoría 2 de la propuesta inicial (*first try*), con 10 trabajos asociados. . . 118
6.3. Categoría 1 tras solicitar una nueva propuesta (*retry*). 119
6.4. Categoría 2 tras la regeneración: “Simulación De Redes Y Sistemas Distribuidos”. 119
6.5. Categoría 3 tras la regeneración: “Simulación Específica De WSN”. 119
6.6. Instrucción de modificación introducida por el investigador y confirmación del sistema. 120
6.7. Categoría 1 tras aplicar la instrucción de modificación manual. 120
6.8. Estructura de tabla propuesta inicialmente (*first try*), 8 columnas. 121
6.9. Estructura de tabla tras solicitar una nueva propuesta a partir de la versión ya modificada. 121
6.10. Estructura de tabla tras la modificación manual, 9 columnas. 122
6.11. Página de Ajustes confirmando que el directorio de trabajos relacionados está vacío. 124

6.12. Mensaje del sistema ante un directorio de trabajos relacionados vacío. . . . 124
6.13. Cascada de mensajes de error tras el directorio vacío, hasta llegar a la evaluación de categorización. 125
6.14. Mensaje del sistema al detectar que el directorio de trabajos relacionados no existía, tras crearlo automáticamente. 125
6.15. Cascada de mensajes de error tras el directorio inexistente, idéntica a la observada con el directorio vacío. 126
6.16. Página de Ajustes mostrando el PDF corrupto junto a cuatro documentos válidos. 127
6.17. Extracción de texto completada para los documentos válidos, con reporte aislado del fallo en el documento corrupto. 127
6.18. Fichas técnicas generadas correctamente para los cuatro trabajos válidos, sin rastro del documento corrupto. 128
6.19. Contenido del directorio de trabajos relacionados, incluyendo dos archivos no-PDF. 129
6.20. El sistema indexa únicamente los cuatro documentos PDF, ignorando los archivos no-PDF sin generar advertencia. 129
6.21. Los cinco documentos PDF empleados como conjunto de entrada en ambas ejecuciones. 131
6.22. Configuración de modelos empleada en la ejecución local (`llama3.2:3b` en ambos *slots*). 132
6.23. Configuración de modelos empleada en la ejecución híbrida (`llama3.2:3b` + Gemini `2.5-flash`). 132
6.24. Conjunto de entrada de 3 documentos. 136
6.25. Conjunto de entrada de 8 documentos (superconjunto de los 3 anteriores). 136
6.26. Conjunto de entrada de 14 documentos (el empleado en el caso de uso principal). 137

Capítulo 1

Introducción

Contenidos

1.1. Contextualización 1

1.2. Motivación 3

1.3. Objetivos y Contribuciones 6

1.4. Planificación 9

1.5. Estructura del TFG 10

Este primer capítulo sitúa al lector en el punto de partida del proyecto. Se comienza contextualizando la evolución reciente de la Inteligencia Artificial y de los modelos de lenguaje que sustentan este trabajo, para después explicar la motivación concreta que dio origen al proyecto. A continuación, se detallan los objetivos y las contribuciones específicas de la herramienta desarrollada, la planificación temporal seguida durante el desarrollo y, finalmente, se resume la estructura del resto de la memoria.

1.1. Contextualización

La Inteligencia Artificial (IA) no es, ni mucho menos, un campo nuevo. Su origen formal se sitúa en 1956, y desde entonces ha atravesado varias décadas de avances y de "inviernos" en los que las limitaciones de los enfoques estadísticos de cada momento frenaron en seco

las expectativas depositadas en ella [1]. Lo que sí ha cambiado radicalmente es la escala, ya que durante la última década, disponer de muchísima más capacidad de cómputo y de volúmenes de datos ha permitido que ideas que llevaban años sobre la mesa empezaran por fin a funcionar en la práctica, a un ritmo que hace que cualquier afirmación sobre lo que la IA "no puede hacer todavía" tenga fecha de caducidad corta.

Dentro de esa evolución, el subcampo que más ha cambiado en poco tiempo es el del Procesamiento del Lenguaje Natural (PLN), de la mano de los Modelos de Lenguaje de Gran Escala o LLMs (*Large Language Models*). El punto de inflexión se sitúa entre 2017 y 2018, con la publicación de la arquitectura *Transformer* [2, 3]. Su mecanismo de atención sustituyó a las redes recurrentes que se usaban hasta entonces y permitió, por primera vez, procesar el contexto de un texto completo de forma simultánea en lugar de palabra por palabra. A partir de ahí, los LLMs pasaron de resolver tareas puramente sintácticas a abordar problemas de comprensión semántica mucho más exigentes como clasificaciones, extracciones de información, resúmenes de textos, etc. [4].

El salto que hizo que todo esto llegara al público general se produjo a finales de 2022, con el lanzamiento de ChatGPT (OpenAI). De la noche a la mañana, los LLMs dejaron de ser una herramienta casi exclusiva de laboratorios de investigación para convertirse en algo que cualquier persona podía usar desde el navegador. Dos propiedades explican en buena medida ese éxito: la capacidad de resolver tareas nuevas a partir de muy pocos ejemplos (*few-shot learning*), sin necesidad de reentrenar el modelo para cada caso concreto [5], y el hecho de que toda la interacción se reduzca a escribir instrucciones en lenguaje natural, que el modelo traduce internamente en representaciones vectoriales (*embeddings*) para generar una respuesta.

Sin embargo, a medida que la adopción de los LLMs se ha vuelto más ambiciosa en entornos de alta exigencia intelectual, las limitaciones del paradigma convencional basado en instrucciones aisladas (*simple prompting*) han quedado al descubierto. Cuando un sistema opera bajo un único prompt, la ejecución es estrictamente lineal y reactiva, lo que restringe drásticamente la profundidad de su razonamiento. En este contexto de interacción estática emergen problemas críticos como la pérdida de consistencia en

la generación de textos largos, la incapacidad nativa para validar la veracidad de la información y la ausencia de mecanismos de planificación autónoma.

Para solventar las carencias del *simple prompting*, la frontera de la ingeniería de software se está desplazando hacia el paradigma de los agentes autónomos (*Agentic AI*). Mediante este enfoque, el núcleo de razonamiento de los LLMs se encapsula dentro de arquitecturas complejas dotadas de memoria a corto y largo plazo, acceso a herramientas externas y bucles de interacción cíclicos [6]. Bajo este prisma, el modelo se transforma en una entidad autónoma capaz de descomponer un objetivo abstracto en diversas subtareas y evaluar de forma crítica sus propios resultados mediante procesos de autorreflexión. Dentro de este paradigma agéntico, la literatura reciente distingue explícitamente entre arquitecturas *workflow*, en las que un grafo de topología orientada define de forma explícita los nodos de procesamiento (llamadas al LLM, herramientas o subagentes) y las aristas que dictan el enrutamiento y arquitecturas multiagente propiamente dichas, en las que varias entidades autónomas independientes negocian y se comunican entre sí para alcanzar un objetivo compartido [7].

Es precisamente en el primero de estos dos enfoques donde se contextualiza el presente Trabajo de Fin de Grado. El proyecto consiste en el diseño y desarrollo de una herramienta de software cuyo núcleo es un agente basado en LLM orquestado mediante un grafo de estados cíclico (*workflow* agéntico), concebida específicamente para asistir de forma interactiva en una de las fases más complejas y rigurosas de la actividad científica como es el análisis y estructuración de la literatura académica previa.

1.2. Motivación

La motivación de este proyecto surge de la oportunidad de aplicar el paradigma agéntico para resolver un problema práctico y recurrente en el ámbito académico como es la elaboración de la sección de Trabajos Relacionados (*Related Works*). Para cualquier investigador, la búsqueda, el análisis y la posterior redacción de este apartado constituyen una de las fases que demandan mayor tiempo y esfuerzo. Esta sección es fundamental en un artículo científico, ya que define el marco teórico, establece las taxonomías de partida

y fija los criterios de comparación necesarios para justificar el valor y la originalidad de la investigación frente a la literatura ya existente.

Además, este esfuerzo es aún más notable en el escenario contemporáneo, ya que los investigadores se enfrentan a una crisis de sobreinformación derivada del crecimiento exponencial de publicaciones científicas. El proceso manual de mapear la información conceptual, indexar variables metodológicas y consolidar una narrativa fluida desemboca en un consumo ingente de tiempo y esfuerzo. Desafortunadamente, los sistemas basados en Modelos de Lenguaje de Gran Escala tradicionales fallan sistemáticamente al abordar esta tarea de forma aislada, tendiendo a descontextualizar las metodologías originales o a colapsar ante volúmenes masivos de datos. Por consiguiente, una solución de software basada en un agente orquestado mediante un grafo de estados que interactúe estrechamente con el usuario no solo representa un avance significativo en la aplicación práctica de la IA agéntica, sino que se erige como una herramienta de alto valor estratégico para potenciar la productividad y la calidad editorial científica.

Para comprender en detalle el problema que resuelve este Trabajo de Fin de Grado y la verdadera necesidad de la herramienta desarrollada, es imperativo desgranar las complejas barreras técnicas y vulnerabilidades que manifiestan los LLMs genéricos al enfrentarse a la literatura científica, las cuales se detallan a continuación:

- **El fenómeno de la alucinación semántica:** Por su propia naturaleza arquitectónica, los LLMs son modelos probabilísticos autorregresivos optimizados para predecir el token sintácticamente más adecuado a partir de una secuencia previa, careciendo de un conocimiento determinista integrado o de un mecanismo de verificación fáctica nativo [8]. Al interactuar con un LLM comercial genérico, el sistema prioriza la fluidez textual, el tono persuasivo y la coherencia gramatical por encima de la fidelidad de los hechos. Como consecuencia, surge una tendencia crítica a la invención de metadatos académicos, tales como nombres de autores, referencias bibliográficas inexistentes o métricas estadísticas distorsionadas, con el fin de satisfacer la narrativa demandada. En la ciencia, la integridad y la trazabilidad de los datos son principios innegociables, ya que la inclusión de una sola cita

apócrifa o un dato falsificado compromete la credibilidad del investigador y devalúa la investigación en cualquier proceso de arbitraje académico.

- **La pérdida de coherencia global y el sesgo de posición:** Otra de las vulnerabilidades más acusadas en la generación de textos de grandes dimensiones es la degradación de la consistencia semántica. A pesar de que la ventana de contexto de los modelos actuales ha aumentado significativamente, dicha capacidad de admisión de tokens no se traduce de forma directa en una comprensión relacional profunda de la información. Al delegar la redacción del *Related Works* en un flujo lineal, el LLM es víctima del fenómeno conocido como *Lost in the Middle* (Perdidos en el medio) [9]. Este sesgo cognitivo provoca que el modelo retenga y priorice la información situada en los extremos de la secuencia de entrada, perdiendo, distorsionando o ignorando los datos críticos localizados en la zona central del texto.

Más allá de estas limitaciones técnicas generales de los LLMs, cabe destacar que la generación automática de revisiones de literatura constituye en sí misma un área de investigación emergente y todavía activa. Trabajos recientes como AutoSurvey han demostrado que es viable emplear agentes basados en LLM para redactar estados del arte extensos combinando recuperación de información y generación por secciones [10]. No obstante, este tipo de sistemas se centra fundamentalmente en la generación de *surveys* generalistas sobre un área de conocimiento amplia, ejecutados de forma mayoritariamente autónoma y con escasa intervención del usuario durante el proceso. La tarea de redactar específicamente la sección *Related Works* de un artículo propio es, por el contrario, una tarea centrada en el propio manuscrito del investigador (*paper-centric*), cuyo objetivo no es resumir un campo completo, sino posicionar de forma argumentada una contribución concreta frente a un conjunto acotado de trabajos previos [11]. Esta distinción evidencia que las soluciones existentes de generación automática de literatura no cubren de forma satisfactoria el caso de uso que aborda este TFG, en el que el investigador debe mantener el control editorial sobre cada decisión (selección de fuentes, categorización, estructura comparativa) y no simplemente recibir un texto generado de forma opaca y desatendida.

La herramienta desarrollada en este trabajo resuelve de manera directa este escenario.

Al fragmentar el problema y distribuir las cargas de trabajo a través de un grafo de estados interactivo, el sistema combate el efecto Lost in the Middle al procesar las fuentes de forma modular, y mitiga el impacto de las alucinaciones precisamente gracias a su diseño human-in-the-loop, que mantiene al investigador en el centro de cada decisión editorial y le permite validar o corregir cualquier contenido alucinado antes de que este avance en el flujo de trabajo. De este modo, la motivación del proyecto se consolida en la creación de un asistente agéntico que proporciona un soporte robusto, seguro e interactivo a la comunidad académica en la gestación de sus marcos teóricos.

1.3. Objetivos y Contribuciones

El objetivo general de este Trabajo de Fin de Grado es asistir al investigador en la automatización, estructuración y redacción de la sección Related Works de un artículo científico, mediante el diseño y desarrollo de una herramienta software basada en un agente de IA con arquitectura en grafo que interactúe con él a lo largo de todo el proceso.

Para cumplir este objetivo general, se definen los siguientes requisitos principales:

1. **Configuración e Ingesta de Datos:** El sistema debe ofrecer un módulo de entrada que permita al investigador indicar de manera sencilla el tema de su artículo de investigación y gestionar de manera autónoma los documentos y credenciales necesarios para el funcionamiento del sistema.

2. **Lectura y Análisis de Documentos:** El sistema debe ser capaz de leer y analizar los documentos que se aportan, extrayendo toda la información relevante para su posterior uso.

3. **Clasificación Dinámica:** El sistema debe determinar si resulta conveniente agrupar los trabajos analizados en categorías y, en caso afirmativo, proponer dicha categorización al investigador.

4. **Generación de Tablas:** El sistema debe ser capaz de proponer una tabla comparativa de los trabajos analizados, garantizando que la información presentada

no resulte densa ni redundante.

5. **Redacción Automática del Texto:** El sistema debe redactar de forma automática los distintos bloques de texto que componen la sección, conforme a una serie de instrucciones predefinidas.

6. **Ensamblado y Exportación del Documento Final:** El sistema debe consolidar todos los bloques de texto generados en un único documento coherente en formato LaTeX, incorporando de forma automática las citas correspondientes a cada trabajo analizado.

7. **Control Interactivo:** El sistema debe estructurarse mediante una arquitectura en grafo que permita establecer pausas estratégicas para el control detallado del investigador.

8. **Configuración y Extensibilidad del Motor de Modelos:** El sistema debe permitir seleccionar y combinar de forma independiente los modelos de lenguaje empleados en las distintas tareas, admitiendo tanto proveedores locales como servicios en la nube, sin necesidad de modificar el código fuente.

Las contribuciones de este trabajo se traducen en un software modular donde cada uno de los componentes definidos resuelve directamente uno de los requisitos principales y cubre una parte del flujo de trabajo:

1. **Módulo de Interfaz y Configuración:** Se ha programado una interfaz web que recibe las directrices del investigador acerca del artículo de investigación que se está desarrollando. Este componente arranca el flujo de la aplicación y asegura que el agente entienda perfectamente la temática del paper desde el primer momento. Incorpora además una página de Ajustes que permite al investigador gestionar de forma autónoma los documentos analizados, las credenciales de acceso a servicios externos y la configuración de los modelos de lenguaje, sin necesidad de intervenir directamente sobre el sistema de archivos ni el código fuente.

2. **Agente de Análisis y Extracción:** Es un componente especializado en literatura científica. Su función es leer los textos a fondo para extraer solo los enfoques metodológicos más relacionados con el trabajo del usuario, evitando errores comunes de los LLMs tradicionales como la pérdida de contexto o la invención de datos.

3. **Generador de Clasificación:** Este nodo del software analiza las fuentes y decide si es conveniente agruparlas. Si lo ve viable, propone un máximo de tres categorías con sus respectivas descripciones y detiene la aplicación para que el investigador las apruebe o las edite a su gusto.

4. **Creador de Tablas:** Se ha diseñado un subgrafo de nodos que evalúa si una tabla comparativa aporta valor al texto. Si decide incluirla, define las columnas de manera dinámica, construye la tabla con datos reales y precisos extraídos de las fuentes, y escribe un párrafo descriptivo para explicarla.

5. **Motor de Redacción:** Es el componente encargado de la producción del texto. Se divide en varias subtareas. Algunas de ellas son: un nodo que escribe la introducción de la sección, otro que resume cada artículo seleccionado en un párrafo breve y enfocado, u otro nodo que redacta una conclusión robusta comparando el estado del arte con la propuesta del autor.

6. **Ensamblador y Referenciador Final:** Es el componente responsable de fusionar todos los bloques de texto generados en un único documento LaTeX coherente, insertando de forma automática los comandos de cita cruzada (`\cite`) junto a cada trabajo mencionado y generando el entorno bibliográfico (`\bibitem`) correspondiente, dejando el resultado listo para su compilación.

7. **Orquestador de Grafo:** Es la pieza central de la arquitectura del software. En lugar de usar un flujo lineal donde el usuario no tiene control, este componente gestiona los estados de la aplicación mediante un grafo cíclico, permitiendo que la máquina y el investigador colaboren activamente en decisiones clave, como la selección final de los artículos o la estructura de la matriz comparativa.

8. **Motor de Configuración de Modelos:** Es el componente responsable de resolver, en tiempo de ejecución y a partir de una configuración editable por el investigador, qué modelo concreto debe emplearse en cada una de las dos categorías de tareas del sistema (ligeras y pesadas), admitiendo de forma indistinta modelos locales y servicios externos, y permitiendo incorporar nuevos proveedores sin necesidad de modificar la lógica de negocio del resto del sistema.

1.4. Planificación

La siguiente tabla recoge la planificación temporal seguida a lo largo del desarrollo del proyecto, desglosada en las distintas fases de trabajo, con una breve descripción de cada una y una estimación de las horas dedicadas.

Tabla 1.1: Planificación temporal estimada del proyecto.

Fase	**Descripción**	**Horas**
1. Investigación inicial	Estudio de conceptos fundamentales (LLMs, agentes, sistemas multiagente), lectura de artículos proporcionados por los tutores, primeras pruebas con Ollama, LangGraph, CrewAI y LlamaIndex	35h
2. Elicitación de requisitos	Definición de objetivos, requisitos funcionales/no funcionales y restricciones del sistema	10h
3. Diseño de la arquitectura	Diseño del grafo de estados, definición del `AgentState`, diagramas de arquitectura y de casos de uso	20h
4. Implementación - Core del agente	Desarrollo de los nodos de ingesta, análisis, clasificación, tabla comparativa, redacción y ensamblado LaTeX con modelos fijos	≈90h

Continúa en la página siguiente

(Continuación de la Tabla 1.1)

Fase	**Descripción**	**Horas**
5. Implementación - Interfaz	Desarrollo de la interfaz Gradio de una sola página, integrada con el grafo mediante `interrupt()`/`Command`	20h
6. Refactorización - Motores de modelos configurables	Diseño e implementación de `LLMPerezoso`, el registro de proveedores, la resolución dinámica de modelos y la persistencia de configuración	6h
7. Ampliación - Interfaz de Ajustes	Página de gestión de PDFs, variables de entorno y modelos LLM desde la GUI; resolución de problemas de navegación/colas de eventos	10h
8. Pruebas y caso de uso	Ejecución de casos de uso con *papers* reales, ajuste de *prompts*, corrección de errores	30h
9. Redacción de la memoria	Redacción y revisión de todos los capítulos del TFG	55h
	Total	**≈276h**

1.5. Estructura del TFG

El resto de este documento se organiza en los siguientes capítulos:

- **Capítulo 2:** presenta el estado del arte sobre el que se sustenta este trabajo, abordando los fundamentos de los Modelos de Lenguaje de Gran Escala y sus limitaciones críticas, el paradigma de los agentes basados en LLM y la discusión sobre la conveniencia de los sistemas multiagente frente a arquitecturas más simples, el ecosistema de *frameworks* de inferencia local y orquestación, y la literatura específica sobre generación asistida de revisiones de literatura científica, con especial

atención a los sistemas *paper-centric* y a los enfoques *human-in-the-loop*.

- **Capítulo 3:** describe el proceso de diseño de la herramienta, incluyendo la elicitación y definición de los requisitos del sistema, los diagramas de casos de uso y de secuencia elaborados, y la arquitectura de software adoptada, detallando las capas que componen el sistema y las decisiones técnicas que sustentan su diseño.

- **Capítulo 4:** detalla la implementación de cada uno de los componentes del sistema, organizados por capas funcionales, explicando las decisiones de ingeniería más relevantes tomadas durante el desarrollo, tanto en el núcleo del agente como en la interfaz de usuario.

- **Capítulo 5:** presenta un caso de uso real de la herramienta, aplicándola sobre un artículo de investigación y un conjunto de trabajos relacionados, describiendo la ejecución paso a paso y analizando los resultados obtenidos en comparación con literatura de referencia.

- **Capítulo 6:** recoge un conjunto de pruebas adicionales que van más allá del caso de uso principal, dirigidas a validar aspectos concretos del sistema que dicho caso de uso no llegó a ejercitar como son el bucle completo de regeneración y modificación manual de categorías y tabla, la tolerancia a fallos ante distintos escenarios de ingesta de documentos, el efecto de la configuración de modelos sobre la calidad y validez del documento generado, y el comportamiento del sistema ante un volumen creciente de trabajos relacionados.

- **Capítulo 7:** expone las conclusiones extraídas del proyecto en relación con los objetivos planteados, así como las posibles líneas de trabajo futuro identificadas durante el desarrollo.

- **Capítulo 8:** ofrece una reflexión personal sobre el proceso de realización del Trabajo de Fin de Grado.

Capítulo 2

Estado del Arte

Contenidos

2.1. Fundamentos: LLMs y sus Dos Limitaciones Críticas 13

2.1.1. Origen y Definición Fundamental 13

2.1.2. Clasificación y Tipos de Modelos 14

2.1.3. Beneficios y Limitaciones en Entornos Complejos 16

2.2. Agentes basados en LLM y Orquestación 17

2.2.1. Paradigma y Componentes de un Agente basado en LLM . . . 17

2.2.2. Adquisición de Capacidades 19

2.2.3. Sistemas Multiagente: ¿Por qué no? 19

2.2.4. Ecosistema de Desarrollo: Inferencia Local y Orquestación . . . 21

2.3. Generación Asistida de Revisiones de Literatura 23

2.3.1. Generación Autónoma de Surveys Científicos 24

2.3.2. Sistemas Paper-Centric 24

2.3.3. Enfoques Human-in-the-Loop e Interactivos 25

2.3.4. Síntesis Comparativa y Research Gap 28

El desarrollo de un sistema software interactivo que sea capaz de asistir en la redacción de un artículo científico requiere de un marco teórico sólido sobre las tecnologías que lo sustentan. Este capítulo describe primero los fundamentos de los LLMs y de los agentes basados en LLM estrictamente necesarios para justificar las decisiones de diseño del sistema (Secciones 2.1 y 2.2), para situar después este TFG dentro de la literatura específica que aborda la generación asistida de revisiones de literatura científica (Sección 2.3), que es precisamente el problema concreto que este trabajo pretende resolver.

2.1. Fundamentos: LLMs y sus Dos Limitaciones Críticas

Antes de justificar cualquier decisión de diseño relativa a los agentes o al flujo de orquestación del sistema, resulta necesario fijar una base conceptual mínima sobre los LLMs que lo sustentan. Esta sección repasa primero su origen y funcionamiento básico, para después analizar los distintos tipos de modelos disponibles y el criterio de selección adoptado en este trabajo (Subsección 2.1.2), y cerrar con las dos limitaciones críticas que motivan la necesidad de arquitecturas agénticas más sofisticadas (Subsección 2.1.3).

2.1.1. Origen y Definición Fundamental

Los Modelos de Lenguaje de Gran Escala constituyen el núcleo principal de este proyecto. Hasta la introducción de la arquitectura Transformer [2], el procesamiento del lenguaje natural se apoyaba en redes recurrentes (RNN/LSTM) que procesaban el texto palabra por palabra, lo que impedía paralelizar el entrenamiento y degradaba la memoria del modelo ante secuencias largas [1]. El mecanismo de auto-atención (*self-attention*) del Transformer eliminó ambas limitaciones al permitir evaluar de forma simultánea y bidireccional la relación entre todas las palabras de un texto, con independencia de la distancia que las separa [2, 3, 4].

Desde una perspectiva computacional, un LLM moderno se define como un modelo

predictivo probabilístico y autorregresivo de millones de parámetros [3]. Es fundamental establecer, tal y como se observó en las fases iniciales de investigación de este trabajo, que estos sistemas no operan bajo un esquema conceptual humano ni poseen una comprensión consciente de los datos que manejan. Su funcionamiento se basa en el cálculo continuo de probabilidades textuales. Cuando un usuario introduce una secuencia de comandos de entrada, denominada prompt, la red analiza la estructura sintáctica previa y genera una distribución de probabilidad sobre todo el vocabulario para determinar qué fragmento de texto es el más adecuado para continuar la secuencia. Este ciclo se repite de manera recursiva, palabra por palabra, logrando construir respuestas que emulan el razonamiento humano a partir de un aprendizaje puramente estadístico.

2.1.2. Clasificación y Tipos de Modelos

Elegir qué modelo actuará como núcleo de procesamiento es una de las primeras decisiones que exige cualquier aplicación basada en LLMs. En la práctica, esa elección se reduce a dos grandes vertientes, que se diferencian por su infraestructura de despliegue, su modelo de licenciamiento y la gestión de la privacidad [3].

Por un lado, se encuentran los modelos de código cerrado (closed-source) operados en la nube por las grandes corporaciones del sector tecnológico, entre los que destacan los modelos GPT de OpenAI, Claude de Anthropic o Gemini de Google. Estos sistemas se caracterizan por contar con escalas de parámetros masivas y son ejecutados en centros de datos de muy alto rendimiento. Los desarrolladores de software interactúan con dichos modelos a través de interfaces de programación de aplicaciones (APIs). La ventaja de este enfoque se centra en la enorme capacidad de razonamiento semántico y en que liberan al ingeniero de cualquier tarea relacionada con el hardware o servidores. No obstante, esto introduce problemas de gran calado para proyectos de investigación. El principal inconveniente es la existencia de costes asociados a la cantidad de tokens consumidos, lo que dificulta la sostenibilidad financiera a largo plazo. Además, plantea un riesgo mucho más severo para la privacidad y la seguridad de los datos ofrecidos, puesto que obligan a transferir información confidencial o manuscritos hacia servidores externos controlados

por terceros [12].

Como contraposición directa a estos sistemas comerciales, han emergido con fuerza los modelos de código abierto (*open-source*) orientados a la ejecución local, tales como Llama de Meta, Qwen de Alibaba o Mistral. La proliferación de herramientas de abstracción como Ollama ha democratizado el uso de estos modelos, permitiendo cargarlos en el hardware privado del usuario. La mayor virtud que tienen estos modelos es la absoluta garantía de la confidencialidad de los datos, ya que toda inferencia se realiza dentro del entorno local y los datos nunca pasan a través de la red informática. A esto se suma que el costo operativo por token es nulo, permitiendo realizar infinitas pruebas de desarrollo sin restricciones de tipo económico. A pesar de estos beneficios, la ejecución local traslada toda la presión al hardware del usuario, puesto que estos modelos requieren de una gran cantidad de memoria RAM y VRAM. Si el software está mal optimizado o intenta procesar flujos de datos muy densos de manera descuidada, la capacidad del hardware local se satura con facilidad, provocando colapsos en el sistema informático.

En lugar de resolver esta disyuntiva mediante una elección estática de modelos fijada de antemano, el sistema desarrollado en este trabajo adopta un enfoque más flexible, inspirado en la práctica de dirigir cada tarea al modelo más adecuado según su complejidad, en lugar de emplear un único modelo indiscriminadamente para toda la carga de trabajo. Esta práctica, conocida en la literatura como LLM routing, parte de la observación de que buena parte de las tareas que resuelve un sistema basado en LLM son de dificultad heterogénea. Las consultas sencillas pueden resolverse con modelos ligeros y económicos sin pérdida apreciable de calidad, mientras que las tareas que exigen mayor capacidad de razonamiento se benefician de modelos más potentes, habitualmente también más costosos [13]. Aplicando este mismo principio, aunque de forma simplificada (mediante una asignación configurada por el investigador en lugar de un clasificador automático de complejidad como los propuestos en la literatura de routing), el sistema organiza sus tareas en dos categorías (ligeras y pesadas) y permite asignar de forma independiente a cada una de ellas cualquier modelo disponible, ya sea un modelo local servido mediante Ollama o un servicio en la nube (Google Gemini, OpenAI o Anthropic),

pudiendo así adaptar la solución final a las restricciones de coste, privacidad y hardware de cada investigador concreto, en lugar de imponer una única combinación de modelos válida para todos los casos de uso.

2.1.3. Beneficios y Limitaciones en Entornos Complejos

La incorporación de los modelos de lenguaje en las metodologías de desarrollo de software actuales aporta una serie de beneficios indiscutibles que justifican plenamente su elección. Estos sistemas destacan por su flexibilidad operativa, su habilidad para comprender estructuras de datos complejas y su capacidad para resolver problemas mediante el aprendizaje contextual con pocos ejemplos (*few-shot learning*), una propiedad extensamente documentada a partir del desarrollo de los modelos fundacionales masivos [5], lo que evita la necesidad de realizar costosos reentrenamientos del modelo. Asimismo, demuestran una competencia sobresaliente en la codificación sintáctica y el formateo automatizado de la información.

Pero, cuando estas herramientas se aplican sin ningún tipo de control, salen a la luz limitaciones técnicas intrínsecas que pueden llegar a comprometer la integridad de las investigaciones. La más destructiva de estas limitaciones es el fenómeno de la alucinación semántica en la generación de lenguaje natural [8]. Dado que los LLMs son optimizados de manera estadística para maximizar la fluidez y verosimilitud, priorizan la coherencia del tono por encima de la fidelidad a los hechos reales. Si el modelo no posee una base informativa dentro de su ventana de contexto o si el comando de entrada es ambiguo, el sistema tenderá a inventar de manera sumamente convincente citas, nombres, años o incluso resultados que parecen auténticos.

A esta problemática se suma la restricción de ventana de contexto limitada y la aparición del sesgo de posición conocido como el efecto *Lost in the Middle* [9]. Aunque los modelos de lenguaje actuales han expandido su capacidad para albergar decenas de miles de palabras en una sola consulta, la distribución de la atención computacional a lo largo del texto no es la misma. Los experimentos empíricos demuestran que los LLMs tienen una marcada tendencia para recordar y procesar la información ubicada en los

extremos (inicio y fin del texto suministrado), mientras que la zona central sufre una grave degradación, tendiendo a omitir o ignorar información crítica [9]. Esto resulta de suma gravedad ya que, a la hora de analizar grandes cantidades de textos o artículos extensos, es muy posible que información clave sea ignorada debido a este fenómeno, lo que justifica la necesidad de saltar hacia arquitecturas agénticas más sofisticadas [6].

2.2. Agentes basados en LLM y Orquestación

Superada la base conceptual de los LLMs, esta sección se centra en cómo dotar a un modelo de lenguaje de capacidad de acción autónoma y persistente. Para ello se introduce primero el paradigma del agente basado en LLM y los cuatro módulos que habitualmente estructuran su arquitectura, junto con las estrategias de adquisición de capacidades disponibles (Subsección 2.2.2). A continuación se analiza por qué, a pesar de su popularidad creciente, los sistemas multiagente no resultan la opción más adecuada para este trabajo, para finalmente revisar el ecosistema de *frameworks* de inferencia local y orquestación que justifica la elección tecnológica adoptada (Subsección 2.2.4).

2.2.1. Paradigma y Componentes de un Agente basado en LLM

Un agente inteligente es una entidad capaz de percibir su entorno, razonar sobre él y ejecutar acciones orientadas a un objetivo, sin necesidad de que un humano prescriba de antemano cada uno de los caminos lógicos posibles [14]. A diferencia del software tradicional, que exige anticipar todos los casos en una estructura condicional rígida, un agente evalúa de forma continua el estado del problema y decide de manera autónoma qué acción tomar, replanificando su estrategia si algo falla [14, 15].

De las tres dimensiones que la literatura exige a un agente (autonomía, equilibrio entre proactividad y reactividad, y continuidad en el tiempo [15]), esta última es la que condiciona directamente el diseño de este TFG. Frente a las ejecuciones aisladas y sin estado (stateless) propias de una llamada convencional a un LLM, un agente requiere una ejecución persistente que mantenga un hilo de estado a lo largo de todo el proceso,

sobreviviendo a la finalización de cada subtarea concreta [6]. Esta propiedad es la que sostiene, como se verá en la Subsección 3.2.2, el diseño del estado compartido AgentState como memoria única de toda la ejecución del agente.

Una vez que se han comprendido los aspectos fundamentales de los LLMs y el concepto de agente inteligente, es momento de analizar el punto en el que se basa este trabajo, que es una fusión de ambos conceptos. Históricamente, dotar a un agente de un componente de razonamiento flexible requería de arquitecturas simbólicas o de costosos entrenamientos. La maduración de los LLMs ha cambiado eso, permitiendo que estos asuman directamente el rol del núcleo dentro del sistema (lo que se denomina agente basado en LLM [6]).

Bajo este nuevo enfoque técnico, el LLM deja de comportarse como una interfaz de chat pasiva y estática para convertirse en el centro de un bucle cerrado de ejecución. Al integrarlo en una arquitectura agéntica, el sistema pasa a orquestar por sí solo el ciclo recurrente de percepción, razonamiento y acción [6]. La percepción se nutre de los datos del entorno que son preprocesados y formateados como texto plano. Luego, el LLM asume la fase de razonamiento analizando el contexto, evaluando las metas y decidiendo el siguiente paso. Finalmente, esta resolución se traduce en una acción concreta a través de herramientas, cuyas salidas vuelven a alimentar al entorno. Esta transición desde una herramienta hacia una entidad con capacidad de ejecución persistente redefine por completo la ingeniería de software orientada a la automatización de procesos complejos.

La arquitectura de un agente basado en LLM se organiza habitualmente en cuatro módulos que compensan las carencias nativas del modelo [16]. El Módulo Perfil (*Profiling*) moldea la identidad y las pautas de comportamiento del agente mediante directrices de sistema (*System Prompts*), restringiendo el espacio de respuesta del LLM a la tarea encomendada. El Módulo Memoria separa la información en memoria a corto plazo, que gestiona la ventana de contexto inmediata, y memoria a largo plazo, que introduce persistencia externa (ficheros, bases de datos) para superar la naturaleza efímera del modelo [17]. El Módulo Planificación descompone problemas complejos en subtareas mediante técnicas como *Chain-of-Thought* [18] y procesos de autorreflexión que evalúan críticamente la propia salida del agente [19]. Por último, el Módulo Herramientas (Tools)

conecta el razonamiento del agente con el entorno real a través de invocaciones a APIs, scripts u otras acciones ejecutables.

2.2.2. Adquisición de Capacidades

La construcción y el despliegue de un agente plantea una decisión crítica en el ámbito del desarrollo software: determinar cómo debe el sistema adquirir el conocimiento y las capacidades requeridas para el entorno de trabajo. Históricamente, las dos grandes vertientes tecnológicas para resolver este problema han sido el *Fine-Tuning* y la *Prompt Engineering* coordinada por arquitectura [20]. El *Fine-Tuning* implica someter a un modelo a un entrenamiento supervisado utilizando una gran cantidad de datos específicos, modificando directamente los pesos de una red neuronal para adaptarla a la tarea encomendada.

A pesar de que el *Fine-Tuning* ofrece una especialización profunda en el dominio del lenguaje, su uso para el control de flujos de trabajo en proyectos software presenta algunos inconvenientes. Principalmente requiere de infraestructuras de computación de altísimo coste, una gran cantidad de datos y penaliza la flexibilidad del sistema, lo que lo hace rígido ante cambios. Por estas razones, para el diseño arquitectónico de este proyecto se justifica la adopción del *Prompt Engineering*. Al delegar la lógica a una arquitectura modular se logra que modelos genéricos de código abierto ejecuten flujos completos de una manera bastante eficaz. Este enfoque resulta más accesible y sostenible, agilizando las pruebas en el desarrollo y permitiendo actualizar los LLMs sin alterar la lógica del sistema.

2.2.3. Sistemas Multiagente: ¿Por qué no?

Antes de justificar la arquitectura de un único agente adoptada en este TFG, conviene revisar brevemente el paradigma frente al que se contrasta, que son los sistemas en los que cooperan varios agentes a la vez. Tradicionalmente, un sistema multiagente se concebía como una red de nodos computacionales descentralizados, cuya principal barrera técnica residía en protocolos de comunicación rígidos como las especificaciones FIPA-ACL. La

llegada de las arquitecturas basadas en LLM sustituyó estas cadenas rígidas de código por el lenguaje natural como interfaz de comunicación entre agentes [21, 22], dando lugar a los sistemas multiagente basados en LLMs (LLM-MA), que se organizan habitualmente en topologías jerárquicas (jefe-subordinado) u horizontales (peer-to-peer) [22].

A pesar de todas las ventajas que ofrecen los sistemas multiagente, su implementación práctica en proyectos de ingeniería software destapa algunos desafíos técnicos.

El primer gran obstáculo es el problema de la propagación en cascada de errores. Dado que los agentes operan de manera encadenada y la salida de una entidad se convierte en el contexto de entrada de la siguiente, cualquier fallo o alucinación será arrastrado e incrementado por los nodos subsiguientes.

El segundo desafío crítico de carácter computacional es la sobrecarga de tokens (*Token Overhead*) derivada de interacciones cruzadas. Cuando varios agentes conversan entre sí en lenguaje natural para coordinar un plan, el volumen de texto generado crece de manera exponencial. Dado que cada mensaje debe introducirse de manera constante en la ventana de contexto de todos los miembros del grupo para así poder mantener el hilo de la conversación, las ventanas de memoria de los LLMs se saturan con gran rapidez. Esto provoca un incremento en el tiempo de procesamiento y satura el hardware en entornos de ejecución local, o dispara de manera considerable los costes económicos si se consumen APIs.

Finalmente, el tercer reto reside en la gestión de la consistencia del estado global del sistema. En un entorno donde múltiples agentes modifican, leen y actualizan de manera concurrente los mismos datos, el riesgo de que se produzcan colisiones de escritura es bastante alto. Garantizar que el estado de la aplicación permanezca intacto y que todos los agentes compartan la misma versión en cada instante requiere del uso de frameworks de orquestación capaces de controlar el ciclo de vida y los flujos de memoria del sistema.

Estos tres desafíos son, precisamente, costes que solo tiene sentido asumir cuando el problema realmente exige varias entidades autónomas negociando entre sí. Como se argumentará en el Capítulo 3, no es el caso de este TFG, de ahí que la arquitectura elegida sea un único agente orquestado mediante un grafo de estados, y no un sistema

multiagente propiamente dicho.

2.2.4. Ecosistema de Desarrollo: Inferencia Local y Orquestación

El despliegue práctico de sistemas basados en LLM ha impulsado el desarrollo de herramientas dedicadas a la abstracción de bajo nivel y al servicio eficiente de los modelos en entornos locales. Tradicionalmente, la inferencia de redes neuronales con miles de millones de parámetros requería de configuraciones muy rígidas y dependencias de hardware. En este escenario, herramientas como Ollama han emergido como estándares de facto en la industria para democratizar el uso de modelos de código abierto (*Open-Source*) [23]. Desde una perspectiva de ingeniería, estas plataformas encapsulan la complejidad de la cuantización de pesos y la optimización de memoria, exponiendo una interfaz de programación de aplicaciones (API) unificada.

Estudios destacan dos factores críticos al analizar la viabilidad de la inferencia local frente a los servicios en la nube. El primer factor es la soberanía y la confidencialidad de la información [12]. En entornos de investigación, el envío de documentos o datos sensibles a servidores de terceros plantea dilemas éticos y de privacidad sustanciales. La inferencia local elimina esta vulnerabilidad al garantizar que los flujos de datos permanezcan dentro de los límites físicos de la infraestructura local. El segundo factor es de naturaleza económica y operativa, dado que el coste por token en arquitecturas agénticas recurrentes puede volverse prohibitivo en modelos de pago por uso, convirtiendo a la ejecución local en una alternativa sostenible para pruebas de desarrollo intensivas.

La necesidad de coordinar la ejecución de un agente, gestionar su estado y permitir la invocación de herramientas externas ha dado lugar a diferentes paradigmas de diseño dentro de los frameworks de orquestación actuales. La literatura especializada clasifica estas herramientas en función de su flexibilidad operativa y del nivel de abstracción que ofrecen al desarrollador, destacando tres de las corrientes principales, representadas por las plataformas CrewAI, LlamaIndex y LangGraph [24, 25, 26].

Por un lado, frameworks como CrewAI proponen un enfoque de alto nivel centrado

en la emulación de estructuras organizacionales humanas, donde el desarrollo se articula en torno a la definición de roles y tareas secuenciales, orientado de forma natural a la construcción de sistemas multiagente [24]. Esta aproximación es altamente eficiente para automatizaciones lineales y prototipos rápidos; sin embargo, la literatura técnica señala que presenta limitaciones severas cuando los sistemas requieren un control determinista estricto. Al carecer de mecanismos nativos para gestionar transiciones de estado complejas de forma granular, resulta complejo implementar en estas plataformas flujos no lineales donde el proceso deba retroceder, pivotar o bifurcarse ante contingencias dinámicas del entorno, algo que en este proyecto ocurre de forma constante (por ejemplo, en los bucles de confirmación/modificación de categorías o de la tabla comparativa).

Por otro lado, plataformas como LlamaIndex abordan el problema desde un enfoque estrictamente centrado en los datos y en las arquitecturas de Generación Aumentada por Recuperación (RAG) [25]. Estos entornos destacan por su capacidad para indexar grandes volúmenes de información no estructurada y optimizar las consultas semánticas. No obstante, las evaluaciones empíricas demuestran que, al priorizar las estructuras de datos orientadas a índices, estos sistemas pueden manifestar una sobrecarga notable en el consumo de memoria volátil (RAM) bajo flujos conversacionales de larga duración, limitando su flexibilidad para actuar como orquestadores de lógicas conversacionales e interactivas puras.

Como alternativa a las limitaciones de linealidad y rigidez de datos, ha ganado tracción el paradigma de los Grafos de Estado Cíclicos, cuyo máximo exponente en la ingeniería de software es LangGraph [26]. Este enfoque modela los flujos de trabajo de la inteligencia artificial como máquinas de estado explícitas y deterministas, donde cada proceso es un nodo y las decisiones lógicas son aristas, sin necesidad de instanciar múltiples entidades autónomas independientes [7]. La ventaja teórica fundamental de este modelo es su capacidad para gestionar ciclos (bucles de retroalimentación) de manera segura y centralizar la memoria compartida mediante un estado único de la aplicación. Asimismo, este diseño basado en grafos facilita la implementación nativa del paradigma *Human-in-the-Loop* (el ser humano en el bucle), proveyendo puntos de interrupción lógicos

donde la máquina detiene su autonomía para requerir la supervisión o validación de un operador humano antes de proceder. Es precisamente esta combinación de control determinista, gestión nativa de ciclos y soporte de primer nivel para la interrupción humana la que motiva la elección de LangGraph como framework de orquestación para este proyecto, frente a las alternativas de CrewAI y LlamaIndex.

La Tabla 2.1 resume esta comparación entre los tres paradigmas revisados.

Framework	**Paradigma**	**Fortaleza principal**	**Limitación clave respecto a este TFG**
CrewAI	Roles y tareas secuenciales (alto nivel), orientado a multiagente	Prototipado rápido; automatizaciones lineales eficientes	Sin control determinista granular; difícil implementar bucles de retroceso/pivote ante contingencias
LlamaIndex	Centrado en datos e indexación, arquitecturas RAG	Indexación de grandes volúmenes; consultas semánticas optimizadas	Sobrecarga de memoria en flujos conversacionales largos; no orientado a orquestar lógica interactiva pura
LangGraph	Grafo de estados cíclico, máquina de estados explícita y determinista	Control determinista; gestión nativa de ciclos; soporte de primer nivel para HITL	Requiere modelar explícitamente cada nodo y arista del flujo (mayor carga de diseño inicial)

Tabla 2.1: Comparativa de frameworks de orquestación para agentes basados en LLM.

En cuanto a los estándares de interoperabilidad, destaca el Model Context Protocol (MCP) [27] como estándar abierto para homogeneizar la comunicación entre un LLM y recursos externos (archivos, APIs, repositorios). Sin embargo, el sistema no lo integra, al acceder directamente al sistema de ficheros local. Su adopción se plantea como línea de trabajo futuro (Capítulo 7).

2.3. Generación Asistida de Revisiones de Literatura

Más allá de los fundamentos generales sobre LLMs, agentes y frameworks de orquestación, es necesario situar este TFG dentro de la literatura específica que aborda la asistencia mediante IA a la redacción de revisiones de literatura científica, dado que es precisamente el problema concreto que este trabajo pretende resolver.

2.3.1. Generación Autónoma de Surveys Científicos

En los últimos años ha emergido un cuerpo de investigación centrado en la generación automática de surveys científicos completos mediante LLMs. El sistema AutoSurvey, por ejemplo, propone un pipeline estructurado en cuatro fases (recuperación inicial y generación de esquema, redacción por subsecciones, integración y refinamiento, y evaluación) para producir revisiones de literatura extensas sobre un área de conocimiento, combinando técnicas de recuperación de información con generación por secciones especializadas [10]. En la misma línea, trabajos posteriores como SurveyGen han introducido conjuntos de datos y marcos de evaluación específicos para medir la calidad de los surveys generados por LLMs frente a los redactados por humanos, concluyendo que, si bien los pipelines semiasistidos logran resultados parcialmente competitivos, la generación completamente automática todavía presenta problemas de baja calidad de citación y de análisis crítico limitado [28].

2.3.2. Sistemas Paper-Centric

Sin embargo, estos sistemas se orientan de forma mayoritaria a la producción de surveys generalistas sobre un campo de conocimiento amplio, ejecutados con un grado de autonomía elevado y una intervención del usuario reducida durante el propio proceso de generación. La tarea que aborda este TFG es de naturaleza distinta. La redacción de la sección Related Works de un artículo científico propio constituye una tarea paper-centric, cuyo objetivo no es resumir exhaustivamente un área de conocimiento, sino posicionar de forma argumentada una contribución concreta frente a un conjunto acotado de trabajos previos, estableciendo similitudes, diferencias y el vacío de investigación (*research gap*) que la nueva propuesta viene a cubrir [11].

Dentro de este subconjunto de trabajos paper-centric, el sistema LitLLM [29] es la referencia más directa a este TFG. LitLLM recibe el *abstract* del investigador, recupera papers potencialmente relevantes mediante una combinación de extracción de palabras clave y búsqueda externa, los reordena mediante un módulo de re-ranking, y genera la sección final ya sea en modo zero-shot o mediante un plan de oraciones explícito que

fija qué cita debe aparecer en cada frase. Un trabajo de seguimiento de los mismos autores, *LitLLMs: ¿Ya estamos ahí?* [30], descompone deliberadamente la tarea en sus dos componentes (recuperación y redacción) para evaluar por separado la capacidad de los LLMs actuales en cada una, concluyendo que, si bien el rendimiento ha mejorado sustancialmente, ninguno de los dos componentes alcanza aún un nivel que permita prescindir de la revisión humana, en particular en la fidelidad de las citas generadas. Esta conclusión es relevante para este TFG por dos motivos. Primero, confirma que la tarea concreta que aborda este trabajo no está resuelta por los sistemas existentes, ni siquiera en su versión más cercana. Segundo, refuerza la necesidad de un diseño human-in-the-loop en lugar de una generación completamente autónoma, ya que ambos trabajos de LitLLM operan mayoritariamente en modo zero-shot, sin puntos de confirmación intermedios equivalentes a los que plantea este sistema para la categorización o la tabla comparativa.

Un segundo sistema paper-centric relevante es ChatCite [31], que aborda específicamente el paso de resumen comparativo dentro del proceso de revisión de literatura. Su arquitectura se organiza en dos módulos: un Extractor de Elementos Clave, que interroga cada paper mediante un conjunto fijo de preguntas estructuradas (objetivo, metodología, resultados, entre otras) para obtener una representación homogénea de cada trabajo relacionado, y un Generador Incremental Reflexivo, que redacta el resumen comparativo de forma iterativa, incorporando en cada paso el resumen generado hasta el momento y aplicando un mecanismo de autoevaluación antes de aceptar el resultado. El paralelismo con el diseño de este TFG es directo. El Extractor de Elementos Clave de ChatCite desempeña un papel equivalente al de la ficha técnica estructurada que este sistema genera para cada trabajo relacionado. Sin embargo, ChatCite ejecuta todo el proceso de forma completamente autónoma y sin puntos de interrupción para el investigador, a diferencia del enfoque adoptado en este trabajo.

2.3.3. Enfoques Human-in-the-Loop e Interactivos

Esta distinción no es meramente académica, sino que tiene una consecuencia directa sobre el nivel de control que el investigador debe conservar durante el proceso. Estudios

recientes centrados específicamente en la experiencia de investigadores que emplean LLMs para tareas de revisión de literatura reportan una desconfianza generalizada hacia la capacidad de estos modelos para citar fuentes o resumir resultados científicos de forma fiable, documentando casos recurrentes de invención de referencias o atribución incorrecta de hallazgos. Como consecuencia, los investigadores rara vez permiten que un LLM redacte de forma autónoma la sección de trabajos relacionados de su propio artículo, empleándolo en su lugar como apoyo al descubrimiento, la lluvia de ideas o la organización previa, siempre sujeto a una verificación manual posterior de las fuentes [32]. Esta observación empírica constituye una motivación adicional, más allá de las limitaciones técnicas ya descritas en la Sección 1.2, para adoptar un diseño human-in-the-loop en el que el investigador confirme o modifique cada decisión relevante del proceso (selección de categorías, estructura de la tabla comparativa, aceptación del texto generado), en lugar de delegar la redacción de forma completamente desatendida.

Más allá de los sistemas centrados en la generación automática de texto, ha emergido recientemente una línea de investigación en el ámbito de la Interacción Persona-Ordenador (cs.HC) que cuestiona directamente el paradigma autónomo. InsightAgent [33] parte de la constatación explícita de que los agentes LLM existentes para revisión de literatura operan de forma mayoritariamente autónoma, y de que la falta de interacción y transparencia en su proceso de toma de decisiones constituye una limitación significativa. Para solucionarlo, InsightAgent combina un diseño multiagente con una interfaz gráfica que permite al investigador supervisar en tiempo real la partición del corpus documental y guiar la síntesis incremental de evidencia, visualizando la procedencia y las rutas de razonamiento seguidas por los agentes. Aunque InsightAgent se aplica a un dominio distinto, su motivación es esencialmente la misma que la de este TFG: la falta de interacción y transparencia de los sistemas autónomos existentes constituye un problema en sí mismo, no solo un matiz de diseño, lo que refuerza directamente la pertinencia del enfoque human-in-the-loop adoptado en este trabajo.

Un antecedente más temprano de este mismo principio, situado ya en el terreno de la interacción persona-ordenador antes de la consolidación de los LLMs actuales, es Relatedly

[34]. A diferencia de los sistemas anteriores, Relatedly no genera texto nuevo, sino que ayuda al investigador a leer y explorar múltiples secciones de Related Works ya publicadas sobre un tema, mediante reordenación dinámica, resaltado de información no explorada y generación automática de encabezados descriptivos por párrafo. Su estudio con usuarios mostró que este tipo de scaffolding produce esquemas de tema más coherentes y completos que la lectura no asistida. Aunque su tarea es distinta de la abordada en este TFG, Relatedly es relevante como precedente que demuestra empíricamente, ya en el ámbito HCI, el valor de mantener al investigador en control activo del proceso de revisión, en lugar de delegarlo por completo en un sistema autónomo.

Este enfoque colaborativo se alinea, además, con propuestas recientes de frameworks de revisión de literatura que combinan LLMs con supervisión humana sistemática para garantizar la fiabilidad de los resultados antes de su uso final, en dominios como la clasificación temática de grandes corpus documentales apoyada en RAG y validada por expertos humanos [35]. Si bien estos trabajos operan a una escala considerablemente mayor (corpus de miles de artículos) y con objetivos distintos (clasificación temática y análisis bibliométrico frente a la redacción directa de una sección de un artículo), comparten con este TFG el mismo principio de diseño subyacente, según el cual la intervención humana no debe entenderse como una limitación del sistema agéntico, sino como un mecanismo de control de calidad integrado en la propia arquitectura.

Por último, conviene situar este TFG frente a las herramientas de uso real que un investigador tiene hoy a su disposición, como Elicit, SciSpace, Consensus o ResearchRabbit. Estas plataformas resuelven de forma eficaz tareas de descubrimiento, cribado sistemático o exploración visual de redes de citación sobre un corpus amplio de literatura, pero ninguna de ellas aborda la tarea paper-centric concreta que plantea este trabajo. Este trabajo trata de partir de la contribución propia del investigador y redactar, de forma asistida y con confirmación humana en cada decisión relevante, el texto final de la sección Related Works que posiciona esa contribución frente a un conjunto acotado de trabajos previos, ensamblado y listo para su integración directa en el artículo.

2.3.4. Síntesis Comparativa y Research Gap

La Tabla 2.2 sintetiza esta revisión y permite formular de manera explícita el hueco de investigación que este TFG contribuye a cubrir. Dentro de los sistemas paper-centric, ninguno combina simultáneamente una intervención humana en cada decisión estructural del proceso (no solo al final, como una revisión post-hoc) con el uso directo de los PDF completos aportados por el propio investigador como fuente documental, sin depender de una búsqueda externa que pueda introducir trabajos ajenos a la selección del investigador. Cabe señalar, además, que la propia Tabla 2.2 constituye un ejemplo del tipo de artefacto que el sistema desarrollado en este TFG genera de forma asistida.

Sistema	**Ámbito**	**Autonomía**	**Punto de intervención humana**	**Fuente documental**	**Modelo**	**Evaluación**
AutoSurvey	Survey generalista	Alta	Ninguno (post-hoc)	Búsqueda web/BD	Nube	Automática (métricas)
SurveyGen	Survey generalista	Alta	Ninguno	Corpus indexado	Nube	Dataset + humana
LitLLM	Paper-centric	Alta (zero-shot / plan)	Plan de oraciones opcional	Búsqueda web (abstracts)	Nube	Automática + humana
ChatCite	Paper-centric	Alta	Ninguno	PDFs proporcionados	Nube	Automática + humana
InsightAgent	Systematic review (salud)	Media	Continua (tiempo real)	Corpus local (RIS)	Nube	Humana (expertos)
Relatedly	Exploración (no genera)	Baja (herramienta lectura)	Continua	Papers ya publicados	N/A	Estudio de usuarios
Este TFG	Paper-centric	Media (HITL por diseño)	En cada decisión estructural	PDFs locales del investigador	Local/nube (routing)	Caso de uso cualitativo

Tabla 2.2: Comparativa de sistemas de generación/asistencia a la revisión de literatura.

Capítulo 3

Diseño

Contenidos

3.1. Elicitación de Requisitos 30

3.1.1. Definición de Requisitos 30

3.1.2. Diagramas de Casos de Uso y de Secuencia 36

3.2. Arquitectura de Software 39

3.2.1. Vista de Alto Nivel 40

3.2.2. El Estado Compartido del Agente 42

3.2.3. Vista del Grafo de Estados 43

3.2.4. Justificación de las Decisiones Arquitectónicas 45

El presente capítulo describe el diseño seguido para la construcción de la herramienta, partiendo de la elicitación de los requisitos que debía satisfacer el sistema hasta la definición de la arquitectura de software que finalmente se ha implementado. La solución diseñada adopta la forma de un agente basado en LLM orquestado mediante un grafo de estados, por lo que a lo largo de este capítulo se hará especial hincapié en cómo dicha decisión arquitectónica condiciona tanto la forma en que se han definido los requisitos como la manera en que se estructuran los componentes del sistema.

3.1. Elicitación de Requisitos

La fase de elicitación de requisitos tuvo como punto de partida los requisitos principales planteados en la Sección 1.3, los cuales surgieron de un proceso iterativo de reflexión conjunta con los tutores del Trabajo de Fin de Grado sobre el problema a resolver, la ya comentada asistencia de forma interactiva y controlada a un investigador en la redacción de la sección *Related Works* de un artículo científico. A partir de estos objetivos, y siguiendo el espíritu de buenas prácticas ampliamente aceptadas en la ingeniería de requisitos, como las recogidas en la norma IEEE 830 para la especificación de requisitos software [36], se ha llevado a cabo un proceso de refinamiento progresivo en el que cada objetivo general se ha ido descomponiendo en requisitos de sistema más concretos y verificables.

Es importante remarcar que, siguiendo la indicación de mantener este análisis a nivel de sistema, los requisitos que se presentan a continuación describen el comportamiento observable del sistema desde el punto de vista de un investigador que lo utiliza, sin descender todavía al detalle de qué nodo concreto del grafo de LangGraph implementa cada funcionalidad ni de cómo interactúan entre sí los distintos componentes internos. Esa descomposición a nivel de componente se abordará más adelante, en el Capítulo 4, una vez que se haya presentado la arquitectura de software en la Sección 3.2.

Asimismo, dado que el sistema se concibe como una herramienta de uso individual por parte de un investigador (y no como un servicio multiusuario), se ha identificado un único actor relevante a efectos de la elicitación de requisitos: el Investigador, entendido como la persona que desea redactar la sección *Related Works* de su propio artículo científico y que interactúa con el sistema para ello.

3.1.1. Definición de Requisitos

Siguiendo la práctica habitual en la especificación de requisitos software, se ha optado por clasificar los requisitos identificados en tres categorías: requisitos funcionales (RF), que describen una capacidad concreta que el sistema debe ofrecer; requisitos no funcionales (RNF), que describen atributos de calidad que el sistema debe satisfacer al proporcionar

dichas capacidades; y restricciones (R), que imponen límites externos al diseño no negociables por el equipo de desarrollo. A continuación, se presenta esta clasificación organizada de forma jerárquica en torno a los ocho objetivos específicos definidos en la Sección 1.3.

Objetivo 1 – Configuración e Ingesta de Datos.

- **RF-01:** El sistema deberá permitir al investigador seleccionar el idioma en el que se redactará el documento final, entre español académico e inglés académico.
- **RF-02:** El sistema deberá permitir al investigador describir en lenguaje natural la temática, metodología y aportaciones de su propio artículo de investigación.
- **RF-03:** El sistema deberá transformar la descripción en lenguaje natural proporcionada por el investigador en una ficha técnica estructurada (título, autores, año, problema abordado, metodología, aportaciones y limitaciones).
- **RF-04:** El sistema deberá localizar de forma automática los documentos en formato PDF disponibles en el directorio de trabajos relacionados destinado a tal efecto.
- **RNF-01 (robustez):** El sistema deberá informar de forma clara al investigador si el directorio de trabajos relacionados no existe o se encuentra vacío, sin finalizar de forma abrupta la ejecución.
- **R-01:** El sistema únicamente admite como entrada documentos en formato PDF.

Objetivo 2 – Lectura y Análisis de Documentos.

- **RF-05:** El sistema deberá extraer el contenido textual de cada uno de los documentos PDF proporcionados por el investigador.
- **RF-06:** El sistema deberá generar, para cada trabajo relacionado, una ficha técnica estructurada equivalente a la definida para el artículo propio del investigador (RF-03).

- **RF-07:** El sistema deberá permitir al investigador configurar el número de caracteres de cada documento que se envían al modelo en la fase de análisis, incluyendo la posibilidad de enviar el texto completo del documento sin truncar.
- **RNF-02 (tolerancia a fallos):** El sistema deberá continuar el procesamiento del resto de documentos si la lectura o el análisis de alguno de ellos falla, informando de qué documentos no pudieron procesarse correctamente.
- **R-02:** Por defecto, el análisis de cada documento se realiza sobre un fragmento inicial acotado a 12.000 caracteres de su contenido textual, en lugar de sobre la totalidad del documento, como consecuencia de las limitaciones de ventana de contexto y de rendimiento de los modelos ejecutados de forma local descritas en la Sección 2.1.3. Este límite es configurable por el investigador, pudiendo desactivarse por completo para enviar el texto íntegro de cada documento cuando el modelo empleado disponga de una ventana de contexto suficientemente amplia.

Objetivo 3 – Clasificación Dinámica.

- **RF-08:** El sistema deberá evaluar, en función de los trabajos analizados, si resulta recomendable organizar la sección *Related Works* en categorías temáticas, comunicando dicha recomendación al investigador.
- **RF-09:** El sistema deberá permitir al investigador decidir si desea o no aplicar una categorización, con independencia de la recomendación emitida.
- **RF-10:** En caso de que se opte por categorizar, el sistema deberá proponer un conjunto de categorías temáticas junto con una descripción de cada una y la asignación de los trabajos correspondientes.
- **RF-11:** El sistema deberá permitir al investigador aceptar, solicitar una nueva propuesta de, o modificar manualmente la categorización propuesta, tantas veces como sea necesario hasta su conformidad. Además, debe existir una acción para poder rechazar la propuesta, dando por finalizado el bucle de categorización y rechazando la inclusión de categorías.

- **RNF-03 (usabilidad/legibilidad):** El número de categorías propuestas no deberá superar las tres, con el fin de mantener la legibilidad de la sección resultante.

Objetivo 4 – Generación de Tablas.

- **RF-12:** El sistema deberá evaluar si la inclusión de una tabla comparativa aporta valor a la sección *Related Works*, comunicando dicha recomendación al investigador.
- **RF-13:** El sistema deberá permitir al investigador decidir si desea o no incluir una tabla comparativa, con independencia de la recomendación emitida.
- **RF-14:** En caso de que se opte por incluir una tabla, el sistema deberá proponer una estructura de columnas relevante para la comparación de los trabajos analizados frente a la propuesta del investigador.
- **RF-15:** El sistema deberá permitir al investigador aceptar, solicitar una nueva propuesta de, o modificar manualmente la estructura de columnas, tantas veces como sea necesario hasta su conformidad. Además, debe existir una acción para poder rechazar la propuesta, dando por finalizado el bucle de la tabla comparativa y rechazando la inclusión de una tabla comparativa.
- **RF-16:** El sistema deberá generar la tabla comparativa completa, asegurando que ninguna celda quede sin rellenar.
- **RF-17:** El sistema deberá redactar un texto descriptivo que explique los criterios de comparación utilizados en la tabla.
- **RNF-04 (calidad del contenido):** El sistema deberá evitar proponer una estructura de columnas idéntica o muy similar a una estructura previamente descartada por el investigador dentro de la misma sesión.

Objetivo 5 – Redacción Automática del Texto.

- **RF-18:** El sistema deberá redactar un texto de introducción para la sección *Related Works*, coherente con la temática del artículo propio y con la existencia o no de categorías.
- **RF-19:** El sistema deberá redactar un párrafo descriptivo por cada trabajo relacionado analizado, incluyendo su objetivo, metodología, aportaciones y limitaciones.
- **RF-20:** El sistema deberá redactar una conclusión para la sección que posicione la propuesta del investigador frente a los trabajos analizados.
- **RNF-05 (calidad lingüística):** Todo el texto generado deberá respetar el idioma seleccionado por el investigador en el RF-01 y mantener un registro académico formal.

Objetivo 6 – Ensamblado y Exportación del Documento Final.

- **RF-21:** El sistema deberá consolidar en un único documento la introducción, el cuerpo de trabajos relacionados, la descripción de la tabla (si existe), la propia tabla y la conclusión.
- **RF-22:** El sistema deberá generar el documento final en formato LaTeX.
- **RF-23:** El sistema deberá insertar de forma automática comandos de cita (`\cite`) junto a cada mención de un trabajo relacionado en el texto generado.
- **RF-24:** El sistema deberá generar automáticamente el entorno bibliográfico correspondiente a los trabajos analizados.
- **RNF-06 (robustez):** En caso de error durante el ensamblado final, el sistema deberá generar un documento de contingencia que preserve, al menos, el contenido textual generado hasta ese momento.

Objetivo 7 – Control Interactivo.

- **RF-25:** El sistema deberá detener su ejecución en cada uno de los puntos de decisión relevantes del proceso, a la espera de una respuesta del investigador.
- **RF-26:** El sistema deberá reanudar su ejecución exactamente desde el punto en el que fue detenido, una vez recibida la respuesta del investigador.
- **RF-27:** El sistema deberá ser accesible tanto mediante una interfaz de consola como mediante una interfaz web de tipo conversacional.
- **RNF-07 (persistencia):** El estado completo de la interacción deberá conservarse durante toda la duración de una sesión, de modo que ninguna decisión tomada previamente por el investigador se pierda al reanudar la ejecución.
- **R-03:** El sistema requiere de una conexión a internet activa para las tareas cuyo modelo configurado corresponda a un proveedor en la nube, así como de un equipo con capacidad suficiente de cómputo (CPU/GPU y memoria RAM/VRAM) para la ejecución de aquellas tareas cuyo modelo configurado sea un proveedor local mediante Ollama.

Objetivo 8 – Configuración y Extensibilidad del Motor de Modelos.

- **RF-28:** El sistema deberá permitir al investigador seleccionar, de forma independiente para cada una de las dos categorías de tarea del sistema (ligera y pesada), el proveedor de modelo de lenguaje a utilizar, de entre un conjunto admitido de proveedores locales y en la nube.
- **RF-29:** El sistema deberá permitir al investigador especificar el nombre concreto del modelo y ajustar su temperatura, de forma independiente para cada una de las dos categorías de tarea.
- **RF-30:** El sistema deberá conservar la configuración de modelos seleccionada entre distintas ejecuciones, sin requerir que el investigador la vuelva a introducir en cada sesión.

- **RF-31:** El sistema deberá permitir al investigador añadir y eliminar documentos PDF del directorio de trabajos relacionados directamente desde la interfaz web, sin necesidad de manipular el sistema de archivos.
- **RF-32:** El sistema deberá permitir al investigador introducir y almacenar las credenciales de acceso (claves de API) que requiera cada proveedor de modelos en la nube que decida utilizar.
- **RNF-08 (extensibilidad):** La incorporación de un nuevo proveedor de modelos de lenguaje no deberá requerir modificaciones en la lógica de negocio del núcleo del agente.
- **RNF-09 (seguridad):** Las credenciales introducidas por el investigador no deberán mostrarse en texto claro en la interfaz ni quedar expuestas en el histórico conversacional del sistema.
- **R-04:** Un cambio en la configuración de modelos aplicado a un *slot* que ya haya sido utilizado durante la sesión en curso no surtirá efecto hasta que la aplicación se reinicie.

3.1.2. Diagramas de Casos de Uso y de Secuencia

Con el fin de complementar la especificación textual de requisitos con una representación gráfica de la interacción entre el investigador y el sistema, se han elaborado un diagrama de casos de uso y dos diagramas de secuencia, siguiendo la notación estándar del Lenguaje Unificado de Modelado (UML).

Diagrama de casos de uso. Dado que el sistema cuenta con un único actor (el Investigador), el diagrama de casos de uso se ha centrado en representar de forma clara los puntos de decisión del proceso, que se corresponden en buena medida con los requisitos funcionales que exigen una respuesta explícita del investigador (RF-01, RF-02, RF-08, RF-10, RF-12, RF-14). A continuación, se muestra la especificación del diagrama:

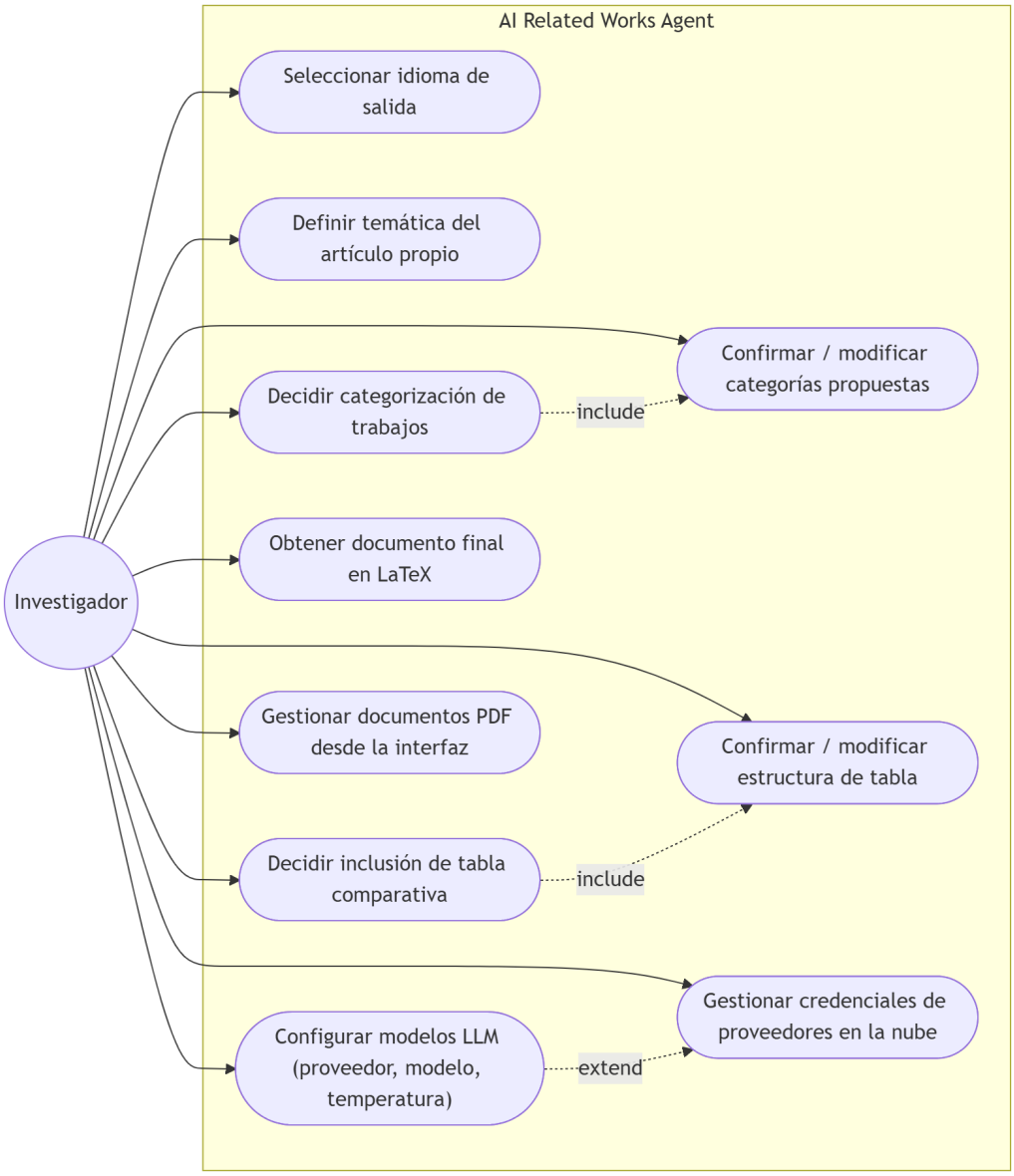


Figura 3.1: Diagrama de casos de uso del sistema.

Diagrama de secuencia — flujo completo sin incidencias. El primer diagrama de secuencia representa el recorrido íntegro del sistema en el caso en que el investigador acepta a la primera todas las propuestas del sistema (categorías y tabla incluidas), sin solicitar ninguna modificación:

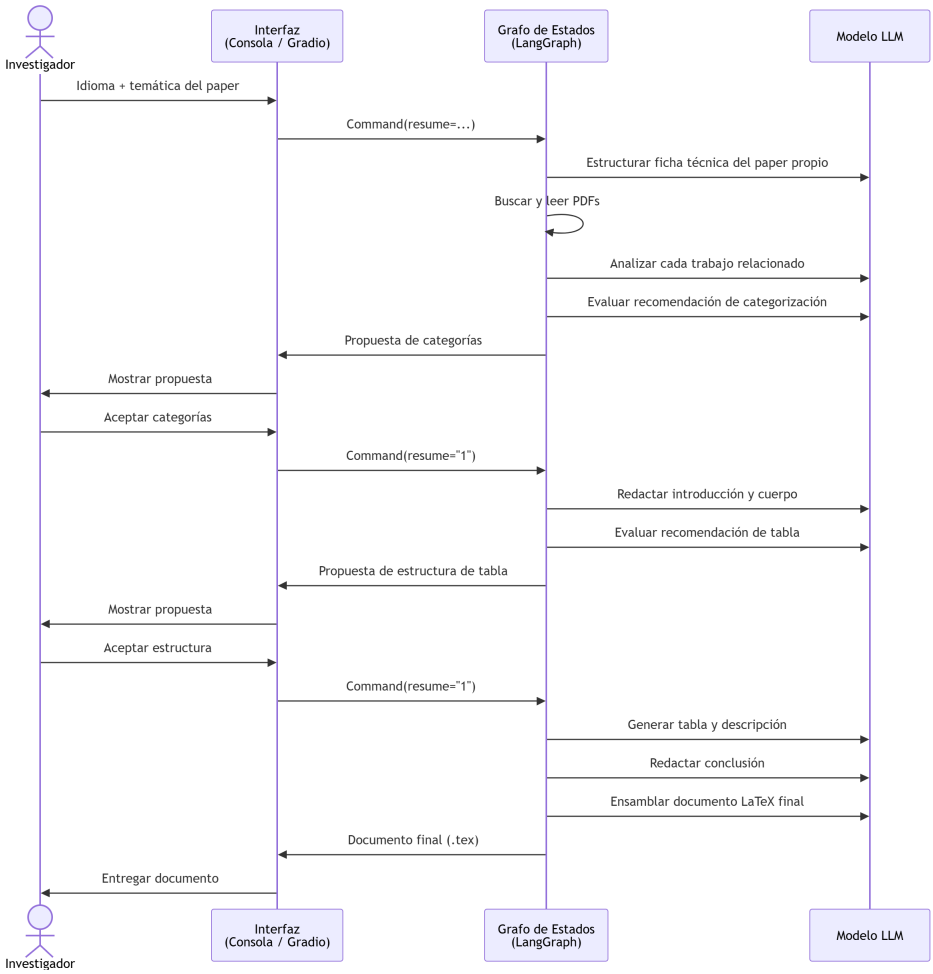


Figura 3.2: Diagrama de secuencia del flujo completo sin incidencias.

Diagrama de secuencia — bucle de modificación. El segundo diagrama de secuencia ilustra un patrón que se repite de forma idéntica tanto en la confirmación de categorías como en la confirmación de la estructura de la tabla comparativa: el ciclo de propuesta, rechazo parcial e instrucciones de modificación, que se repite hasta que el investigador queda conforme con el resultado:

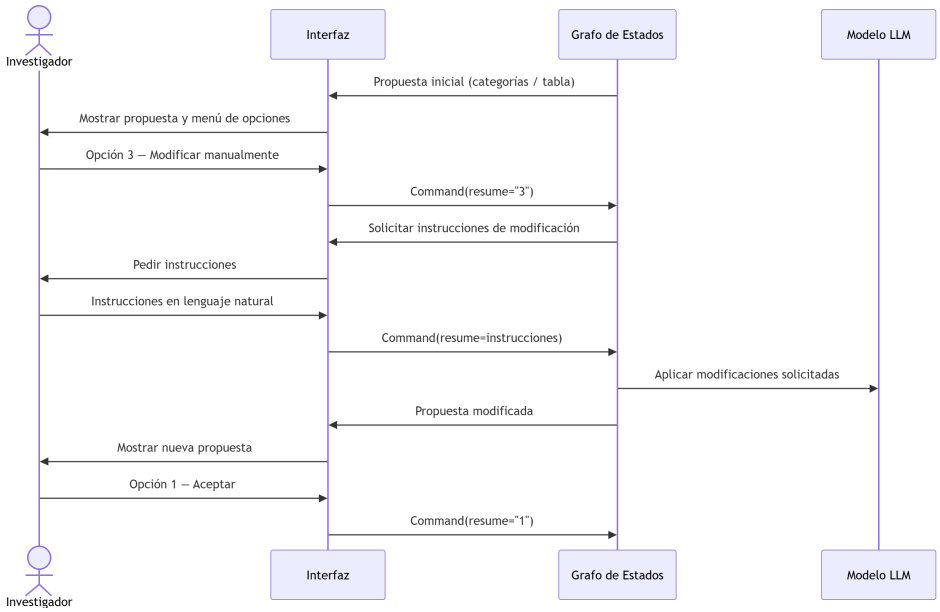


Figura 3.3: Diagrama de secuencia del bucle de propuesta, rechazo y modificación.

Este segundo diagrama es especialmente representativo del diseño del sistema, ya que evidencia cómo el uso de un grafo de estados cíclico permite modelar de forma natural un patrón de interacción iterativo que, en una arquitectura de flujo estrictamente lineal, resultaría mucho más costoso de implementar.

3.2. Arquitectura de Software

Una vez definidos los requisitos que debe satisfacer el sistema, se procede al diseño de la arquitectura de software encargada de darles soporte. Entendiendo la arquitectura de software, en línea con la definición clásica de Bass, Clements y Kazman [37], como la estructura o estructuras del sistema, que comprenden sus componentes, las propiedades visibles externamente de dichos componentes y las relaciones entre ellos, se ha optado por una organización en capas claramente diferenciadas, cada una de ellas con una responsabilidad concreta y acotada.

3.2.1. Vista de Alto Nivel

La Figura 3.4 representa los tres grandes bloques que componen el sistema y las comunicaciones principales entre ellos:

La Capa de Presentación agrupa las dos interfaces disponibles para interactuar con el sistema. La interfaz de consola se mantiene como una vía directa de ejecución y depuración, mientras que la interfaz web, implementada con Gradio, se organiza en tres páginas (Inicio, Ajustes y Chat), incorporando en la página de Ajustes la gestión de documentos, credenciales y configuración de modelos descrita en el Objetivo 8. Ambas interfaces continúan compartiendo la misma responsabilidad de fondo: traducir las respuestas del investigador en llamadas `Command(resume=...)` y presentar de vuelta los mensajes generados por el sistema.

El Núcleo del Agente se articula en torno al grafo de estados, el estado compartido y el *checkpointer* de persistencia, a los que se suma el Motor de Configuración: el componente responsable de resolver en tiempo de ejecución qué modelo concreto debe emplearse en cada una de las dos categorías de tarea del sistema, a partir de la configuración persistida por el investigador. Este componente se apoya en un registro de proveedores admitidos y en un mecanismo de construcción diferida de los modelos, cuyo detalle de implementación se abordará en el Capítulo 4.

La Capa de Modelos, por último, representa el conjunto de proveedores admitidos por el sistema (Ollama, Google Gemini, OpenAI y Anthropic), de entre los cuales el Motor de Configuración selecciona, para cada *slot*, el que el investigador haya indicado. Esta generalización no es un simple cambio cosmético del diagrama: refleja una decisión de diseño real orientada a la extensibilidad (RNF-08), en línea con el principio de asignación de tareas a modelos según su complejidad discutido en la Sección 2.1.2 [13].

Esta separación en capas, inspirada en el patrón arquitectónico de arquitectura en capas ampliamente documentado en la literatura de ingeniería del software [38], aporta al sistema una ventaja práctica relevante: la capa de presentación puede evolucionar de forma independiente (por ejemplo, sustituyendo o ampliando la interfaz Gradio) sin necesidad de modificar la lógica de negocio del núcleo del agente, ya que ambas capas se comunican

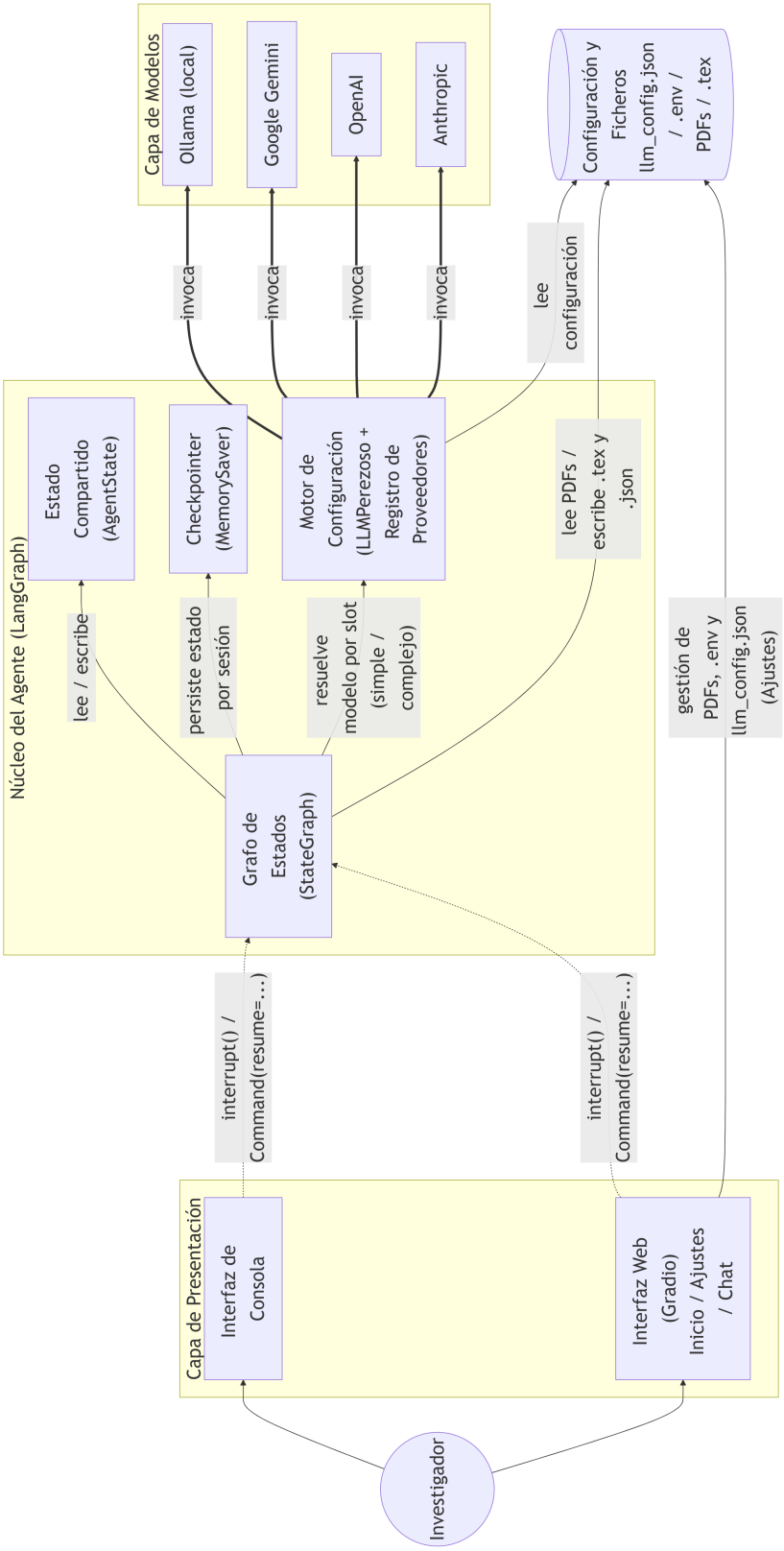


Figura 3.4: Vista de alto nivel de la arquitectura del sistema.

exclusivamente a través del contrato bien definido que ofrece la API de ejecución de LangGraph (`stream`, `interrupt`, `Command`).

3.2.2. El Estado Compartido del Agente

Un elemento central en el diseño del núcleo del agente es la estructura de estado compartido, `AgentState`, que actúa como memoria única de toda la ejecución y que es leída y escrita de forma incremental por los distintos nodos del grafo. Sus campos pueden agruparse conceptualmente en los bloques funcionales recogidos en la Tabla 3.1.

Tabla 3.1: Bloques funcionales del estado compartido `AgentState`.

Bloque	**Campos principales**	**Propósito**
Control y configuración	`idioma_salida`, `error`, `messages`	Configuración global de la sesión y registro conversacional
Contexto del *paper* propio	`tema_paper`	Ficha técnica estructurada del artículo del investigador
Ingesta y análisis	`trabajos_pdf`, `textos_trabajos`, `trabajos_analizados`	Rutas, contenido y fichas técnicas de los trabajos relacionados
Categorización	`categorizar_activo`, `categorias_propuestas`, `categorias_confirmadas`, `accion_categorias`, `instrucciones_modificacion`	Estado del subproceso de clasificación temática
Redacción	`introduccion_related_works`, `related_works_section`, `conclusion_related_work`	Bloques de texto generados por el motor de redacción

Continúa en la página siguiente

(Continuación de la Tabla 3.1)

Bloque	**Campos principales**	**Propósito**
Tabla comparativa	`recomendacion_tabla`, `tabla_comparativa_activa`, `estructura_tabla_propuesta`, `estructura_tabla_confirmada`, `accion_estructura`, `instrucciones_tabla`, `tabla_comparativa_generada`, `descripcion_tabla`	Estado del subproceso de generación de la tabla comparativa
Cierre	`related_works_document`	Documento final ensamblado en LaTeX

Este estado se ha diseñado como un único diccionario tipado (`TypedDict`) compartido por todos los nodos, en lugar de como mensajes independientes intercambiados entre agentes autónomos. Al no existir negociación entre entidades independientes, no resulta necesario un protocolo de comunicación inter-agente, sino que basta con que cada nodo lea del estado la información que necesita y escriba en él los resultados de su ejecución.

3.2.3. Vista del Grafo de Estados

Frente a la vista de alto nivel de la Sección 3.2.1, que representa los bloques arquitectónicos del sistema, conviene mostrar también la vista interna del grafo de estados, ya que es en ella donde se concreta el comportamiento dinámico del sistema y su capacidad de bifurcación y ciclado. La Figura 3.5 representa dicho grafo, agrupando los nodos por las fases funcionales que se detallarán en el Capítulo 4:

Como puede apreciarse, el grafo presenta dos bloques estructuralmente equivalentes (el de categorización y el de la tabla comparativa), ambos organizados en torno al mismo patrón: evaluación automática de conveniencia, propuesta inicial, y un bucle de confirmación que permite aceptar, regenerar o modificar la propuesta hasta la conformidad

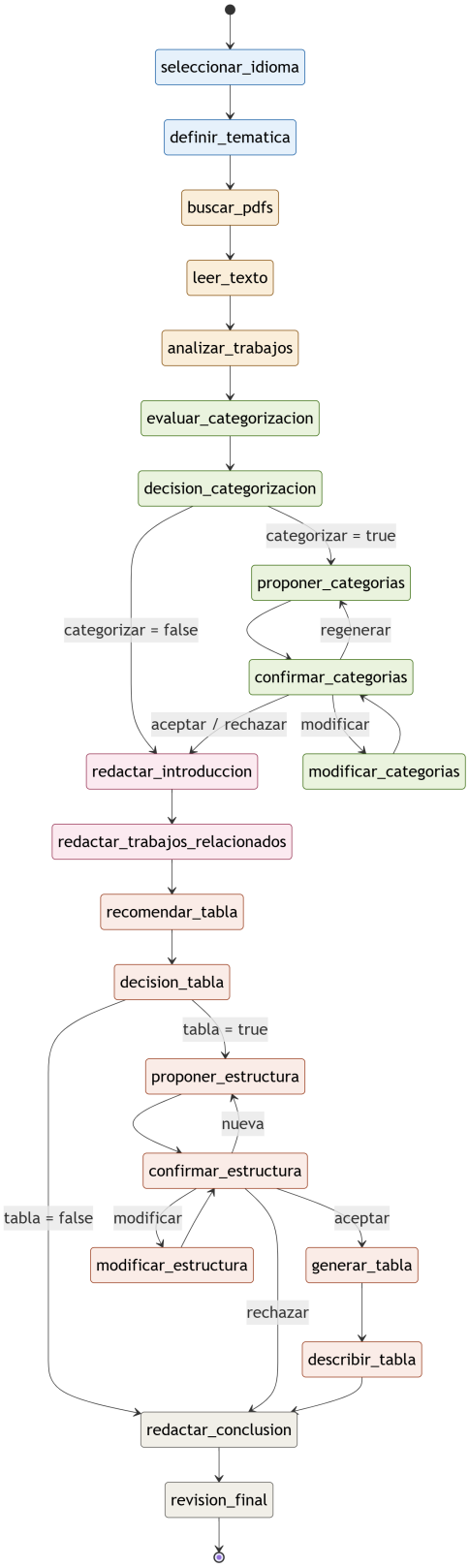


Figura 3.5: Vista interna del grafo de estados del agente.

del investigador. Esta simetría de diseño no es casual, sino que responde a la voluntad de aplicar de forma consistente el mismo mecanismo de control humano en los dos puntos del proceso en los que el sistema debe tomar una decisión estructural sobre el contenido de la sección (cómo se organiza la literatura y cómo se compara con la propuesta propia).

3.2.4. Justificación de las Decisiones Arquitectónicas

Para cerrar este capítulo, se recogen de forma sintética las principales decisiones de diseño adoptadas y su justificación, remitiendo a los argumentos ya desarrollados en el Estado del Arte:

- **Elección de un *workflow* agéntico frente a un sistema multiagente:** los sistemas multiagente basados en LLM sustituyen los protocolos de comunicación rígidos de los sistemas multiagente tradicionales por el lenguaje natural, organizándose habitualmente en topologías jerárquicas (un agente supervisor distribuye subtareas) o *peer-to-peer* (los agentes se autocoordinan entre iguales). Su adopción conlleva, sin embargo, tres costes que solo tiene sentido asumir cuando el problema realmente exige varias entidades autónomas negociando entre sí: la propagación en cascada de errores (un fallo o alucinación de un agente se traslada como contexto de entrada al siguiente), la sobrecarga de tokens derivada de las interacciones cruzadas entre agentes, y la dificultad de garantizar la consistencia de un estado global modificado de forma concurrente. El problema abordado en este TFG no requiere negociación entre entidades autónomas independientes, ya que cada fase puede modelarse como un nodo que lee y escribe sobre un único estado compartido, sin necesidad de un protocolo de comunicación inter-agente. Por ello se ha optado por un único agente orquestado mediante un grafo de estados, que obtiene las ventajas de la modularidad (nodos especializados por tarea) sin incurrir en los tres costes anteriores.

- **Elección de LangGraph frente a CrewAI y LlamaIndex:** de los frameworks de orquestación de agentes más extendidos, CrewAI propone un enfoque de

alto nivel centrado en la definición de roles y tareas secuenciales, eficiente para automatizaciones lineales pero con soporte limitado para flujos no lineales en los que el proceso deba retroceder, pivotar o repetirse ante una modificación del investigador; LlamaIndex, por su parte, centra su diseño en la indexación y recuperación de grandes volúmenes de datos (arquitecturas RAG), lo que puede introducir una sobrecarga de memoria innecesaria en flujos puramente conversacionales. LangGraph, en cambio, modela el flujo de trabajo como una máquina de estados explícita y cíclica en la que cada nodo puede alcanzarse más de una vez, y ofrece soporte nativo para el paradigma *Human-in-the-Loop* mediante `interrupt()`. Esta combinación de control determinista, gestión nativa de ciclos y soporte de primer nivel para la interrupción humana motiva su elección: la necesidad de modelar bucles de confirmación y modificación no lineales, presentes tanto en la categorización como en la tabla comparativa (Objetivos 3 y 4), encaja de forma natural con este modelo y resultaría considerablemente más costosa de implementar sobre CrewAI o LlamaIndex.

- **Separación entre presentación y núcleo del agente:** siguiendo el patrón de arquitectura en capas, se ha evitado que la lógica de negocio dependa de los detalles de una interfaz concreta, permitiendo que el mismo grafo compilado se exponga indistintamente por consola o por una interfaz web, tal y como se evidencia en el hecho de que `gui.py` no contiene lógica de negocio propia.

- **Configurabilidad del motor de modelos frente a una combinación fija:** los modelos de lenguaje disponibles se agrupan en dos categorías con compromisos opuestos. Los modelos cerrados en la nube (GPT, Claude, Gemini) ofrecen una capacidad de razonamiento superior sin requerir infraestructura propia, pero introducen coste por token consumido y obligan a transferir información a servidores de terceros. Los modelos abiertos ejecutados localmente mediante Ollama garantizan la confidencialidad de los datos y no tienen coste por token, a cambio de trasladar toda la exigencia de cómputo (RAM/VRAM) al hardware del investigador. Ninguna de las dos opciones es categóricamente superior: la elección óptima depende

del presupuesto, el hardware disponible y la sensibilidad de los datos de cada investigador concreto. Por ello el sistema no impone una combinación fija, sino que aplica el principio de *LLM routing* [13] de forma simplificada, organizando sus tareas en dos categorías (ligera y pesada) y permitiendo asignar de forma independiente a cada una cualquier proveedor disponible, local o en la nube.

- **Ausencia de RAG:** el paradigma de Generación Aumentada por Recuperación (RAG) conecta un LLM con fuentes de conocimiento externas mediante indexación, recuperación semántica y generación condicionada a los fragmentos recuperados, y resulta especialmente útil cuando el volumen documental a procesar es extenso. El volumen de trabajos relacionados que maneja un investigador al redactar su propia sección *Related Works* es, por naturaleza, mucho más acotado (habitualmente unas pocas decenas de referencias como mucho), lo que permite optar por la lectura directa del texto completo (con truncamiento configurable (RF-07)) en lugar de una arquitectura de recuperación vectorial completa, sin renunciar a mitigar la alucinación mediante otros mecanismos, como la extracción estructurada por esquema (Capítulo 4). La incorporación de RAG queda planteada como línea de trabajo futuro si el sistema se escalase a corpus documentales de mayor tamaño (Capítulo 7).

- **Ausencia de comunicación mediante MCP:** el sistema accede en la actualidad de forma directa al sistema de ficheros local para la lectura de PDFs, sin integrar el *Model Context Protocol* (MCP) como capa de abstracción entre el agente y sus fuentes documentales. Al operar sobre un único directorio local de trabajos relacionados, la complejidad adicional de un protocolo de interoperabilidad pensado para fuentes heterogéneas y distribuidas no está justificada en esta primera versión del sistema; su adopción se plantea como línea de trabajo futuro si el sistema se ampliase para acceder a fuentes bibliográficas externas (Capítulo 7).

- **Resolución diferida de los modelos (`LLMPerezoso`):** para evitar que una configuración incompleta o inválida (por ejemplo, una clave de API todavía no

introducida) impida arrancar la aplicación, la construcción del modelo real asociado a cada *slot* se retrasa hasta el primer uso efectivo, en lugar de resolverse al importar el módulo. Esta decisión, que se detallará en el Capítulo 4, es consecuencia directa del requisito de extensibilidad (RNF-08) y de la necesidad de que la página de Ajustes pueda ofrecerse de forma fiable incluso antes de que el investigador haya completado su configuración de modelos.

Capítulo 4

Implementación y desarrollo

Contenidos

4.1. Metodología de Trabajo 50

4.2. Visión General del Sistema 52

4.3. Capa de Configuración e Ingesta de Datos 54

4.3.1. Seleccionar Idioma y Definición de Temática 54

4.3.2. Búsqueda y Lectura de Documentos PDF 57

4.4. Capa de Análisis 58

4.5. Capa de Clasificación 61

4.5.1. Recomendación y Decisión 62

4.5.2. Propuesta, Confirmación y Modificación 62

4.6. Capa de Redacción 64

4.7. Capa de Tabla Comparativa 66

4.7.1. El Mecanismo de Anti-Repetición 67

4.7.2. Generación de la Tabla y Descripción 69

4.8. Capa de Cierre 70

4.8.1. Redacción de Conclusión 70

4.8.2. Creación del Documento LaTeX 70

4.8.3. Red de Seguridad Post-Generación 72
4.9. Capa de Orquestación 74
4.9.1. Definición del Grafo 74
4.9.2. Persistencia del Estado 74
4.9.3. Bucle Principal de Ejecución 75
4.9.4. Motor de Configuración de Modelos 78
4.10. Interfaz de Usuario 82
4.10.1. Página de Chat 82
4.10.2. Página de Ajustes 84
4.11. Funciones Auxiliares 87

Una vez definida en el capítulo anterior la arquitectura de software que sustenta el sistema, el presente capítulo se centra en describir en detalle cómo se ha llevado a la práctica cada uno de los componentes que la integran[1]. Dado el elevado grado de acoplamiento que existe entre los nodos que forman parte de un mismo subproceso del grafo, se ha optado por organizar la explicación de la implementación por capas funcionales en lugar de nodo por nodo. A lo largo de la explicación de cada capa se hará referencia explícita a los identificadores de requisitos (RF-XX, RNF-XX) definidos en la Sección 3.1.1, con el objetivo de que quede patente la trazabilidad entre lo especificado y lo finalmente implementado.

4.1. Metodología de Trabajo

El desarrollo del sistema no se ha ajustado a una metodología ágil formal (como Scrum o Kanban) en sentido estricto, dado que se trata de un proyecto individual sin necesidad de coordinar a un equipo de desarrollo. No obstante, sí se ha seguido de manera consistente

[1]El código fuente completo del sistema, incluyendo tanto el núcleo del agente (`app1.py`) como la interfaz web (`gui.py`), está disponible públicamente en: https://github.com/SergioDP2003/TFG_Agente_AsistenteRedaccion

un proceso de trabajo iterativo e incremental, inspirado en varias prácticas propias de los marcos ágiles, que merece la pena describir brevemente por haber condicionado buena parte de las decisiones de implementación que se explican en este capítulo.

En primer lugar, los ocho objetivos específicos definidos en la Sección 1.3 actuaron de facto como una *backlog* de producto, una lista priorizada de incrementos de funcionalidad, ordenada siguiendo la dependencia lógica entre ellos (no tiene sentido implementar la categorización antes que la ingesta de documentos que la alimenta), sobre la que se fue trabajando de forma secuencial en lugar de abordar el sistema completo de una sola vez.

El desarrollo arrancó con la construcción de un *walking skeleton*[2], un primer prototipo del grafo con un único nodo capaz de invocar a un modelo local a través de Ollama, cuyo único propósito era validar el funcionamiento básico de LangGraph antes de abordar la complejidad completa del sistema, evitando así descubrir tarde problemas de integración entre componentes. A partir de ese esqueleto, el desarrollo avanzó mediante incrementos funcionales sucesivos (cada uno correspondiente a un nodo o grupo de nodos del grafo (ingesta y análisis, categorización, motor de redacción, tabla comparativa y, por último, ensamblado final en LaTeX)), siguiendo el mismo principio de entrega incremental que estructura un *sprint* en Scrum, donde cada incremento debía quedar en un estado funcional y verificable antes de comenzar el siguiente. En línea con esto, cada nuevo nodo se sometió a un ciclo de prueba aislado mediante ejecuciones por consola antes de integrarse en el grafo completo lo que permitió detectar de forma temprana buena parte de los problemas de formato en las respuestas de los modelos que se describen más adelante en este capítulo.

Una vez el flujo completo funcionaba de extremo a extremo, el trabajo se concentró en un ciclo de inspección y adaptación sobre los *prompts* de cada nodo, donde cada versión del *prompt* se ejecutaba, se observaban las desviaciones del modelo respecto al formato esperado, y se ajustaba la instrucción en consecuencia, repitiendo el ciclo hasta estabilizar el comportamiento. Este ciclo empírico resultó considerablemente más largo de lo inicialmente previsto, ya que los modelos locales de menor tamaño empleados tienden

[2]Un *walking skeleton* es una implementación mínima que conecta de extremo a extremo las piezas arquitectónicas principales de un sistema, aunque cada una de ellas resuelva por el momento una versión trivial de su función, con el objetivo de validar la integración antes de invertir esfuerzo en la lógica de negocio completa.

a desviarse con facilidad de las instrucciones de formato si estas no se especifican de manera extremadamente explícita, lo que llevó a adoptar el estilo de *prompting* altamente restrictivo que se puede apreciar en los ejemplos incluidos a lo largo de este capítulo.

El último incremento del desarrollo incorporó la interfaz Gradio como capa de presentación alternativa a la consola, junto con los mecanismos de persistencia del estado y de exportación del documento final, cerrando así el conjunto de objetivos definidos en la *backlog* inicial.

4.2. Visión General del Sistema

Antes de descender al detalle de cada componente, conviene presentar de forma resumida el recorrido completo que realiza una ejecución del sistema, organizado desde ya en las siete capas funcionales en torno a las cuales se estructura el resto de este capítulo:

- **Capa de Configuración e Ingesta de Datos** (Sección 4.3): recoge el idioma de salida y la descripción del artículo propio del investigador, y localiza y prepara los documentos PDF de los trabajos relacionados (RF-01 a RF-04, RNF-01).
- **Capa de Análisis** (Sección 4.4): extrae de cada documento PDF una ficha técnica estructurada equivalente a la del artículo propio (RF-06).
- **Capa de Clasificación** (Sección 4.5): evalúa si conviene organizar los trabajos analizados en categorías temáticas y gestiona el ciclo de propuesta, confirmación y modificación con el investigador (RF-08 a RF-11, RNF-03).
- **Capa de Redacción** (Sección 4.6): redacta la introducción y el cuerpo de la sección *Related Works* a partir de las fichas técnicas generadas por la capa de análisis (RF-18, RF-19, RNF-05).
- **Capa de Tabla Comparativa** (Sección 4.7): reproduce el mismo patrón de propuesta, confirmación y modificación de la capa de clasificación, aplicado esta vez a la estructura y al contenido de la tabla comparativa (RF-12 a RF-17).

- **Capa de Cierre** (Sección 4.8): redacta la conclusión de la sección y ensambla todos los bloques de texto en el documento LaTeX final, con citas y bibliografía generadas automáticamente (RF-20 a RF-24).

- **Capa de Orquestación** (Sección 4.9): no procesa contenido por sí misma, sino que constituye la infraestructura que coordina la ejecución del resto de capas, gestionando la definición del grafo, la persistencia del estado entre interrupciones y el bucle principal de ejecución (RF-25, RF-26, RNF-07).

A estas siete capas del grafo se suman dos elementos transversales que también se describen en este capítulo: la Interfaz de Usuario (Sección 4.10), responsable de traducir el modelo de ejecución pausable del grafo en una experiencia de *chatbot* web (RF-27), y un conjunto de Funciones Auxiliares (Sección 4.11) empleadas de forma transversal por varias de las capas anteriores.

A grandes rasgos, y siguiendo el orden en que se acaban de presentar, una ejecución completa del sistema recorre estas capas de principio a fin: arranca solicitando al investigador el idioma de salida y una descripción de su propio artículo, tras lo cual localiza y analiza los documentos PDF disponibles; a continuación evalúa la conveniencia de categorizar los trabajos y, en su caso, gestiona el ciclo de confirmación correspondiente; con esa información redacta la introducción y el cuerpo de la sección, evalúa la conveniencia de incluir una tabla comparativa y gestiona un segundo ciclo de confirmación sobre su estructura; y, finalmente, redacta la conclusión y ensambla el documento resultante, que queda disponible tanto por consola como por la interfaz web.

Todo este recorrido queda materializado en el objeto `AgentState`, que actúa como memoria única de la ejecución (véase Sección 3.2.2), y en el grafo de LangGraph, que determina en cada momento qué nodo debe ejecutarse a continuación en función del propio estado.

4.3. Capa de Configuración e Ingesta de Datos

Esta capa agrupa los nodos `seleccionar_idioma_node`, `definir_tematica_node`, `buscar_pdfs_node` y `leer_texto_node`, y da soporte a los siguientes requisitos: seleccionar el idioma académico en el que se redactará el documento final (RF-01), describir en lenguaje natural la temática, metodología y aportaciones del artículo propio (RF-02), transformar dicha descripción en una ficha técnica estructurada (RF-03), localizar automáticamente los documentos PDF disponibles en el directorio de trabajos relacionados (RF-04), e informar de forma clara al investigador si dicho directorio no existe o está vacío, sin finalizar de forma abrupta (RNF-01).

4.3.1. Seleccionar Idioma y Definición de Temática

El primer nodo del grafo, `seleccionar_idioma_node`, es el responsable de dar cumplimiento a RF-01, cuyo propósito es presentar al investigador las dos opciones de idioma disponibles y esperar su respuesta antes de continuar. Puesto que el sistema no puede simplemente "adivinar" esta preferencia ni asumir un valor por defecto sin consultarla, la ejecución del grafo debe detenerse por completo en este punto hasta recibir una respuesta humana explícita. Este es precisamente el papel de la función `interrupt()` de LangGraph, que pausa la ejecución del grafo a la espera de dicha respuesta. `seleccionar_idioma_node` resulta representativo del patrón de interacción que se repetirá a lo largo de todo el sistema en cada uno de los puntos de decisión identificados en el Capítulo 3 (RF-25): a diferencia de una llamada de entrada estándar (como un `input()` en un *script* tradicional), `interrupt()` permite que la pausa se produzca de forma transparente tanto si el sistema se ejecuta por consola como si se ejecuta a través de la interfaz Gradio, delegando en la capa de presentación la responsabilidad de mostrar el mensaje y recoger la respuesta:

```
1 def seleccionar_idioma_node(state: AgentState) -> AgentState:
2     mapa_idiomas = {
3         "1": "Espanol academico",
```

```
        "2": "Ingles academico (English)"
    }
    mensaje_interrupcion = ""
    while True:
        opcion = interrupt(mensaje_interrupcion).strip()
        if opcion in mapa_idiomas:
            idioma_elegido = mapa_idiomas[opcion]
            ...
            return {"idioma_salida": idioma_elegido, ...}
        else:
            mensaje_interrupcion = "Opcion invalida. Por favor, introduce un
            ↪  numero del 1 al 2."
```

Un detalle de implementación que merece la pena destacar es el bucle `while True` que envuelve la llamada a `interrupt()`: en lugar de delegar la validación de la opción introducida por el investigador en un nodo adicional del grafo, se ha optado por resolverla dentro del propio nodo, reintentando la interrupción tantas veces como sea necesario hasta obtener una opción válida. Esta decisión simplifica considerablemente el grafo, evitando tener que modelar explícitamente los estados de error de validación como nodos o aristas independientes.

El nodo `definir_tematica_node` recoge, también mediante `interrupt()`, la descripción en lenguaje natural que el investigador hace de su propio artículo (RF-02), y su responsabilidad es transformar ese texto libre en una ficha técnica estructurada y homogénea (RF-03): un mismo esquema fijo de campos (título, autores, año, problema abordado, metodología, aportaciones y limitaciones) que se rellena a partir de la descripción del investigador y que, como se verá en la Sección 4.4, se reutilizará también para representar de forma equivalente cada uno de los trabajos relacionados. Para garantizar que esa ficha técnica cumple siempre este formato exacto, el nodo no se limita a pedir al modelo que redacte un resumen libre, sino que define de antemano el esquema de

datos esperado mediante Pydantic[3] y delega en la funcionalidad de salida estructurada de LangChain la responsabilidad de que la respuesta del modelo se ajuste a dicho esquema:

```
class PaperEstructurado(BaseModel):
    titulo: str = Field(description="Titulo ORIGINAL del paper...")
    autores: List[str] = Field(description="Lista de autores.")
    anio: str = Field(description="Anio (4 digitos).")
    problema_especifico: str = Field(description="Cual es el vacio o limitacion
    ↪ tecnica exacta...?")
    metodologia_detallada: str = Field(description="Menciona componentes
    ↪ tecnicos especificos...")
    aportaciones_clave: str = Field(description="Cual es la novedad
    ↪ principal?")
    limitaciones_criticas: str = Field(description="Debilidades tecnicas
    ↪ mencionadas o inferidas.")
    casos_uso: List[str] = Field(description="Lista de aplicaciones.")

llm_estructurado = llm_simple.with_structured_output(PaperEstructurado)
res_pydantic = llm_estructurado.invoke(prompt)
```

Cada uno de estos ocho campos recoge, respectivamente el título original del trabajo, la lista de autores, el año de publicación, el vacío o limitación técnica concreta que el trabajo aborda (*research gap*), los componentes metodológicos empleados, la aportación principal reivindicada, las limitaciones técnicas explícitas o inferidas y una lista de posibles casos de uso o aplicaciones. Este mismo esquema es, como ya se ha adelantado, el que estructura tanto la ficha técnica del artículo propio del investigador como la de cada trabajo relacionado analizado en la capa siguiente.

Este mecanismo de salida estructurada es clave para la fiabilidad del sistema. En lugar de confiar en que el modelo devuelva un bloque de texto libre que después haya que

[3]Pydantic es una biblioteca de Python para la definición y validación de esquemas de datos mediante clases tipadas: https://docs.pydantic.dev/.

analizar y validar manualmente, se define de antemano el esquema de datos exacto que se espera obtener, delegando en el propio *framework* la responsabilidad de forzar (o, en el caso de modelos que no soportan *function calling* nativo, de solicitar de forma insistente) que la respuesta se ajuste a dicho esquema. Esta técnica, conocida en la literatura como generación estructurada o *constrained/structured output generation*, se ha consolidado en los últimos años como una de las estrategias más efectivas para mejorar la fiabilidad de los sistemas que integran LLMs dentro de flujos de software más amplios, precisamente porque reduce la necesidad de un análisis (*parsing*) posterior propenso a errores. Además, se ha implementado un bloque de `try/except` con un valor de contingencia (*fallback*) que garantiza que, incluso si la estructuración automática fallase, el grafo pueda continuar su ejecución con una ficha técnica mínima construida a partir del texto original del investigador, en línea con el requisito de robustez señalado de forma transversal en la Sección 3.1.1.

4.3.2. Búsqueda y Lectura de Documentos PDF

Los nodos `buscar_pdfs_node` y `leer_texto_node` implementan, respectivamente, los requisitos RF-04 (localización de PDFs) y RF-05 (extracción de texto). El primero de ellos revisa la existencia del directorio `trabajos_relacionados`, creándolo automáticamente si no existe y devolviendo un mensaje informativo al investigador (dando cumplimiento a RNF-01), mientras que el segundo emplea la librería `pypdf` para extraer el contenido textual de cada documento, página a página:

```
1 reader = PdfReader(ruta)
2 contenido = " ".join([page.extract_text() for page in reader.pages if
  ↪  page.extract_text()])
```

Ambos nodos comparten un mismo patrón defensivo: el procesamiento de cada archivo se envuelve en un bloque `try/except` independiente, de modo que un fallo en la lectura de un PDF concreto (por ejemplo, un documento protegido o corrupto) no interrumpe

el procesamiento del resto de documentos, sino que simplemente se registra como una incidencia que se comunica al investigador al final del nodo. Esta decisión responde directamente al requisito de tolerancia a fallos (RNF-02) definido en el diseño.

4.4. Capa de Análisis

Esta capa está formada por un único nodo, `analizar_trabajos_node`, que da soporte al requisito RF-06 (generar, para cada trabajo relacionado, una ficha técnica estructurada equivalente a la definida para el artículo propio del investigador) y constituye uno de los puntos más sensibles del sistema desde el punto de vista de la fiabilidad, dado que sobre su resultado se apoya prácticamente todo el resto del flujo de redacción.

A diferencia de los nodos anteriores de la capa de ingesta, que procesan todos los documentos como un bloque conjunto, `analizar_trabajos_node` itera de manera independiente sobre cada uno de los textos extraídos en `leer_texto_node` (`state["textos_trabajos"]`), invocando una vez el mecanismo de salida estructurada por cada documento:

```
structured_llm = llm_simple.with_structured_output(PaperEstructurado)

for i, texto in enumerate(state["textos_trabajos"]):
    fragmento = texto if limite_caracteres <= 0 else texto[:limite_caracteres]
    try:
        res = structured_llm.invoke(prompt)
        data = res.model_dump()
        analizados.append(data)
    except Exception as e:
        print(f"Error en trabajo {i+1}: {e}")
```

Esta invocación uno a uno, en lugar de una única llamada que reciba todos los documentos de golpe, resulta necesaria porque el esquema `PaperEstructurado` está

pensado para representar un único trabajo, ya que pedirle al modelo que devolviera varias fichas técnicas en una sola respuesta multiplicaría el riesgo de que el resultado se desalinee (por ejemplo, mezclando el problema de un trabajo con la metodología de otro) y, además, impediría aislar el fallo de un documento concreto del resto. Precisamente por esto último, cada iteración envuelve la invocación en su propio bloque `try/except`. Si la extracción estructurada de un trabajo concreto falla (por ejemplo, porque el texto es demasiado corto o ambiguo para completar el esquema con confianza), el error se registra por consola y ese documento simplemente no se añade a la lista `analizados`, pero el bucle continúa sin interrumpirse con el resto de trabajos. A diferencia de `buscar_pdfs_node` y `leer_texto_node` (Sección 4.3.2), donde los documentos que fallan se comunican explícitamente al investigador en el mensaje de salida, aquí el fallo queda únicamente en el registro interno del servidor, por lo que el investigador solo lo advertirá de forma indirecta, al comprobar que el número de fichas técnicas generadas es menor que el número de PDFs localizados.

El nodo reutiliza el mismo esquema `PaperEstructurado` empleado en la definición de la temática propia (Sección 4.3.1), aplicándolo esta vez a cada uno de los trabajos relacionados localizados en la capa anterior: para cada documento PDF, extrae su texto y lo transforma en una ficha técnica con los mismos ocho campos ya descritos (título, autores, año, problema abordado, metodología, aportaciones, limitaciones y casos de uso), de modo que, al finalizar esta capa, tanto el artículo propio como cada trabajo relacionado quedan representados de forma homogénea y comparable entre sí (la base de información sobre la que se apoyarán tanto la clasificación temática como la redacción final). Como se anticipó al justificar el requisito R-02 en el Capítulo 3, el análisis no se realiza por defecto sobre el texto completo del documento, sino sobre un fragmento inicial acotado a 12.000 caracteres, límite que además es configurable por el investigador desde la página de Ajustes (RF-07), pudiendo desactivarse por completo (valor 0) para enviar el texto íntegro:

```
1 limite_caracteres = cargar_configuracion_llms().get(
2     "limite_caracteres_analisis", LIMITE_CARACTERES_ANALISIS_DEFECTO
```

```
3 )
4 fragmento = texto if limite_caracteres <= 0 else texto[:limite_caracteres]
```

Como se justificó en el Capítulo 3 (R-02 y Sección 3.2.4), este límite responde a un compromiso entre el riesgo de *Lost in the Middle* y la pérdida de información al truncar. En la práctica, 12.000 caracteres resultó un valor razonable dado que la información más determinante para completar el esquema `PaperEstructurado` (título, problema, metodología) suele concentrarse en las secciones iniciales del artículo (resumen, introducción y planteamiento del problema).

El *prompt* de este nodo incorpora, además, un conjunto de reglas de calidad explícitas orientadas a evitar respuestas genéricas o vacías de contenido técnico:

```
1  prompt = f"""
2  Extrae la ficha tecnica del paper.
3
4  REGLAS DE CALIDAD:
5  - TITULO: Extrae solo el nombre del paper, limpio.
6  - PROBLEMA: Define el 'Research Gap' (ej: 'Incapacidad de X para lograr Y').
7  - EVITA RELLENO: No uses frases como 'El trabajo utiliza...', ve directo al
   ↪  grano tecnico.
8  - IDIOMA: Responde siempre en Espanol.
9
10 TEXTO:
11 {{fragmento}}
12 """
```

Este tipo de instrucciones explícitas, en las que se especifican tanto ejemplos de lo que se espera como frases concretas que se deben evitar, se enmarca dentro de las técnicas de *prompt engineering* discutidas en la Sección 2.2.2, y resulta especialmente necesario al trabajar con modelos locales de pocos parámetros, cuya capacidad de seguir instrucciones implícitas es sensiblemente menor que la de los grandes modelos comerciales.

Una vez completado el bucle, el nodo no se limita a devolver la lista de fichas técnicas al estado del agente, ya que construye también dos mensajes de salida pensados específicamente para la interfaz conversacional. El primero recopila, a modo de reporte de progreso, el título de cada trabajo correctamente analizado; el segundo reutiliza la misma estrategia de renderizado en Markdown ya empleada en `definir_tematica_node` (Sección 4.3.1) para presentar cada ficha técnica desglosada por campos (título, autores, año, problema, metodología, aportaciones, limitaciones y casos de uso), en lugar de mostrar al investigador el `repr()` en crudo de la lista de diccionarios que produce `model_dump()`. Es precisamente esta lista de fichas técnicas (`trabajos_analizados`), homogénea y comparable entre trabajos, la que queda disponible en el estado compartido `AgentState` (Sección 3.2.2) para que la consuman directamente tanto la capa de clasificación (Sección 4.5) como la capa de redacción (Sección 4.6).

4.5. Capa de Clasificación

Una vez que la capa anterior ha dejado a los trabajos relacionados representados mediante fichas técnicas homogéneas, el sistema se encuentra en condiciones de decidir cómo organizarlos de cara a la redacción final. Esta capa implementa el ciclo completo de categorización dinámica y está formada por los nodos evaluar_categorizacion_node, decision_categorizacion_node, proponer_categorias_node, confirmar_categorias_node y modificar_categorias_node, junto con las funciones de enrutamiento condicional `gateway_categorizacion` y `gateway_categorias`. Da soporte a los requisitos de evaluar si conviene organizar los trabajos en categorías y comunicar dicha recomendación (RF-08), permitir al investigador decidir si aplica o no la categorización con independencia de la recomendación (RF-09), proponer un conjunto de categorías con su descripción y la asignación de trabajos correspondiente (RF-10), y permitir aceptar, regenerar o modificar manualmente la propuesta tantas veces como sea necesario, sin superar las tres categorías por legibilidad (RF-11, RNF-03).

4.5.1. Recomendación y Decisión

El nodo `evaluar_categorizacion_node` solicita al modelo una recomendación razonada (sí/no) sobre la conveniencia de categorizar los trabajos analizados, mientras que `decision_categorizacion_node` recoge, de nuevo mediante `interrupt()`, la decisión final del investigador. El resultado de esta decisión se traduce en el campo booleano `categorizar_activo` del estado, y es evaluado por la función `gateway_categorizacion`, que determina si el grafo continúa hacia `proponer_categorias` o salta directamente a `redactar_introduccion`:

```
1 def gateway_categorizacion(state: AgentState) -> str:
2     if state.get("categorizar_activo"):
3         return "proponer_categorias"
4     else:
5         return "redactar_introduccion"
```

Esta función es un ejemplo representativo de cómo LangGraph traduce las decisiones lógicas del diagrama de la Sección 3.2.3 en código: cada arista condicional del grafo se corresponde con una función de este tipo, registrada mediante `graph.add_conditional_edges(...)`.

4.5.2. Propuesta, Confirmación y Modificación

En caso de que el investigador, siguiendo o no la recomendación del agente, decida efectivamente organizar la sección en categorías, el nodo `proponer_categorias_node` solicita al modelo una propuesta de un máximo de tres categorías (RNF-03), especificando en el *prompt* un estilo de nomenclatura concreto para evitar nombres de categoría demasiado largos o genéricos, y realizando después una validación programática adicional sobre la respuesta del modelo (verificación de que los trabajos asignados existen realmente entre los analizados, y reasignación de cualquier trabajo que hubiera quedado sin categoría a la primera de ellas). Esta combinación de restricciones expresadas en el *prompt* junto

con una validación posterior en código es un patrón que se repite en varios puntos del sistema, y refleja una estrategia de diseño defensiva: no delegar por completo en el modelo la garantía de corrección de la salida, sino tratar dicha salida como una propuesta que el propio software revisa y corrige antes de continuar.

El nodo `confirmar_categorias_node` implementa el ciclo de decisión del investigador (RF-11), con cuatro posibles acciones (aceptar, regenerar, modificar o rechazar) que se traducen en el campo `accion_categorias` del estado y son interpretadas por la función `gateway_categorias`. La opción de modificación es particularmente interesante desde el punto de vista de la interacción, ya que introduce un segundo `interrupt()` anidado dentro del mismo nodo para solicitar al investigador, en lenguaje natural, qué cambios concretos desea aplicar:

```
1 if decision == "3":
2     instrucciones = interrupt(
3         "INSTRUCCIONES DE MODIFICACION\n"
4         "Indica que deseas cambiar (ej: 'Cambia el nombre de la categoria 1 a
          ↪  Modelos de Lenguaje' "
5         "o 'Mueve el paper X a la categoria 2')."
6     ).strip()
7     ...
```

Dichas instrucciones se envían después al nodo `modificar_categorias_node`, que las incorpora al *prompt* junto con la estructura de categorías actual, solicitando al modelo que devuelva únicamente el bloque JSON modificado. Este es uno de los puntos del sistema en los que la salida del modelo no se solicita mediante el mecanismo de salida estructurada de LangChain (como sí ocurre en la Sección 4.3.1), sino como texto libre que posteriormente se analiza mediante expresiones regulares (`extraer_json`), dado que, al tratarse de una tarea de modificación sobre una estructura JSON ya existente a partir de una instrucción en lenguaje natural completamente abierta, resulta más robusto tratar la respuesta como texto libre y validarla después en código, que confiar en que el mecanismo

de *function calling* del modelo capture correctamente una instrucción de edición arbitraria (una decisión independiente del modelo concreto empleado en el nodo).

Precisamente por la exigencia de esta tarea, interpretar con fidelidad una instrucción de modificación libre y aplicarla sin corromper el resto de la estructura, el nodo se asigna al *slot* `llm_complejo` (Sección 4.9.4) en lugar de a `llm_simple`, a diferencia de otros nodos de esta misma capa. Este cambio de estrategia de control de formato ilustra, en cualquier caso, una de las decisiones de ingeniería más pragmáticas tomadas durante el desarrollo: adaptar la técnica de control de formato a la naturaleza de cada tarea, en lugar de aplicar una única estrategia de forma homogénea a todo el sistema.

4.6. Capa de Redacción

Con los trabajos ya analizados y, opcionalmente, clasificados en categorías, el sistema dispone de toda la información necesaria para comenzar a redactar el texto final de la sección. Esta capa, formada por los nodos `redactar_introduccion_node` y `redactar_trabajos_relacionados_node`, da soporte a los requisitos de redactar un texto de introducción coherente con la temática del artículo propio y con la existencia o no de categorías (RF-18), redactar un párrafo descriptivo por cada trabajo relacionado analizado (RF-19), y respetar el idioma seleccionado por el investigador manteniendo un registro académico formal (RNF-05), constituye el núcleo del "Motor de Redacción" descrito en la Sección 1.3. Conviene aclarar, para evitar confusiones, que la introducción generada por `redactar_introduccion_node` es la introducción de la propia sección *Related Works* que se está construyendo (un párrafo breve que enmarca al lector antes de entrar en el cuerpo de trabajos relacionados), y no la introducción general del artículo del investigador, que queda fuera del alcance del sistema.

Ambos nodos comparten una misma estrategia de diseño: el *prompt* se bifurca en dos variantes claramente diferenciadas en función de si la categorización está activa (`categorizar_activo`) o no, generando en el primer caso un texto organizado por subsecciones temáticas y, en el segundo, una redacción lineal y continua de todos los trabajos. Esta bifurcación explícita dentro del propio nodo, en lugar de modelarse

como dos nodos independientes del grafo, se justifica porque ambas variantes comparten prácticamente toda la lógica de post-procesado del texto generado, evitando así una duplicación de código innecesaria.

Un aspecto especialmente cuidado en los *prompts* de esta capa es el control estricto del formato de salida, dado que estos textos se destinan a integrarse directamente en el documento LaTeX final sin apenas intervención humana adicional. El siguiente extracto del *prompt* de `redactar_introduccion_node` ilustra el nivel de detalle empleado para forzar una estructura exacta de dos párrafos:

```
prompt = f"""
Tu unica tarea es escribir una introduccion academica muy breve para abrir la
↪ seccion "Related Works".
Debes generar EXACTAMENTE DOS PARRAFOS cortos. Esta TERMINANTEMENTE PROHIBIDO
↪ generar mas bloques o parrafos de texto.

REGLAS DE FORMATO CRITICAS:
- NO uses listas, bullets (*), guiones, subtitulos ni enumeraciones.
- NO cites autores ni nombres de papers externos.
...
"""
```

Tras la invocación del modelo, el texto devuelto se somete además a un post-procesado programático que elimina líneas vacías accidentales y cualquier residuo de sintaxis Markdown que el modelo pudiera haber introducido pese a las instrucciones (asteriscos, guiones, almohadillas), reforzando de nuevo la estrategia defensiva de no confiar exclusivamente en el cumplimiento del *prompt* por parte del modelo.

Merece la pena destacar una decisión de diseño relativa al idioma de trabajo interno de esta capa. Con independencia del idioma de salida que el investigador seleccione en `seleccionar_idioma_node` (RF-01), los nodos `redactar_introduccion_node`, `redactar_trabajos_relacionados_node` y `redactar_conclusion_node` (Sección 4.8.1) redactan

siempre su borrador en español. La traducción y homogeneización al idioma finalmente solicitado por el investigador no se realiza en estos nodos, sino que se centraliza en un único punto del *pipeline*, el nodo de ensamblado final, `revision_final_node` (Sección 4.8.2), cuyo *prompt* sí incorpora una directiva explícita de idioma en su última línea.

Esta decisión responde a una consideración práctica sobre la fiabilidad de los modelos de menor capacidad asignados al *slot* `llm_simple` (Sección 4.9.4). Exigir simultáneamente a estos modelos el cumplimiento de las reglas de formato estrictas descritas en esta sección (número exacto de párrafos, prohibición de listas, ausencia de subtítulos, etc.) y la generación del texto en un idioma distinto al de sus propias instrucciones incrementa el riesgo de incumplimiento de dichas reglas, tal y como cabría esperar de la menor capacidad de estos modelos para seguir instrucciones complejas de forma simultánea (Sección 2.1.3). Al mantener el idioma de trabajo interno fijo y delegar la traducción en un único nodo posterior, el sistema evita propagar este riesgo a lo largo de todo el *pipeline* de redacción, concentrándolo en un único punto de control más fiable. El caso de uso descrito en el Capítulo 5 confirma sobre el sistema real que esta estrategia funciona correctamente de principio a fin, entregando siempre el documento final en el idioma solicitado por el investigador con independencia del idioma de trabajo interno de los nodos de redacción.

4.7. Capa de Tabla Comparativa

Con el cuerpo de la sección ya redactado, el agente puede proponer opcionalmente un último elemento de valor añadido como es una tabla comparativa de capacidades. Esta capa gestiona dos decisiones sucesivas por parte del investigador. La primera es si el investigador desea incluir dicha tabla de síntesis de los trabajos relacionados, y, en caso afirmativo, qué estructura de columnas debe tener, es decir, qué campos permitirán homogeneizar y comparar los distintos trabajos incluidos. Además, reproduce, de forma casi simétrica, el mismo patrón de propuesta-confirmación-modificación descrito en la Sección 4.5 para la categorización, aplicado esta vez a dicha estructura. Está formada por los nodos `recomendar_tabla_node`, `decision_tabla_node`, `proponer_estructura_node`, `confirmar_estructura_node`, `modificar_estructura_node`, `generar_tabla_node` y

`describir_tabla_node`, y da soporte a los requisitos de evaluar si la tabla aporta valor y comunicarlo (RF-12), permitir decidir su inclusión (RF-13), proponer una estructura de columnas relevante (RF-14), permitir aceptar, regenerar o modificar dicha estructura (RF-15), generar la tabla completa sin celdas vacías (RF-16) y redactar el texto que explica los criterios de comparación empleados (RF-17), evitando además proponer una estructura idéntica o muy similar a una ya descartada en la misma sesión (RNF-04).

Los tres primeros nodos de esta capa (`recomendar_tabla_node`, `decision_tabla_node` y `confirmar_estructura_node`) replican de forma prácticamente literal el patrón ya descrito en detalle en la Sección 4.5 (concretamente en la Sección 4.5.2) para la categorización, por lo que aquí solo se resumen brevemente en lugar de repetir su explicación completa. `recomendar_tabla_node` desempeña el mismo papel que `evaluar_categorizacion_node`, invocando al modelo para evaluar, a partir de las fichas técnicas de los trabajos analizados y de la ficha del trabajo propio, si la inclusión de una tabla comparativa aporta valor metodológico a la sección, y traslada esa recomendación al investigador (RF-12). `decision_tabla_node`, por su parte, cumple la misma función que `decision_categorizacion_node`, deteniéndose mediante `interrupt()` a la espera de la respuesta afirmativa o negativa del investigador y fija en el estado el indicador `tabla_comparativa_activa` en consecuencia (RF-13), determinando si el flujo continúa hacia `proponer_estructura_node` o salta directamente a la redacción de la conclusión. Finalmente, `confirmar_estructura_node` reproduce el mismo menú de cuatro opciones (aceptar, pedir una nueva propuesta, modificar manualmente o rechazar) que `confirmar_categorias_node`, encaminando la ejecución hacia `generar_tabla_node`, de vuelta a `proponer_estructura_node`, hacia `modificar_estructura_node` o hacia la conclusión, respectivamente (RF-15).

4.7.1. El Mecanismo de Anti-Repetición

El elemento más singular de esta capa, y probablemente uno de los mecanismos de mayor valor añadido de todo el sistema, es la lógica implementada en `proponer_estructura_node` para evitar que, ante una solicitud de regeneración (RNF-04),

el modelo proponga una estructura de columnas prácticamente idéntica a la ya descartada por el investigador. Para resolver este problema, se ha implementado una función de similitud estructural entre dos propuestas de tabla, basada en el coeficiente de Jaccard sobre el conjunto de nombres de columna de cada propuesta:

```
def estructuras_similares(e1, e2, umbral=0.7):
    if not e1 or not e2:
        return False
    cols1 = set([c.lower() for c in e1.get("columnas", [])])
    cols2 = set([c.lower() for c in e2.get("columnas", [])])
    if not cols1 or not cols2:
        return False
    inter = len(cols1.intersection(cols2))
    union = len(cols1.union(cols2))
    similitud = inter / union
    return similitud >= umbral
```

El coeficiente de Jaccard, definido clásicamente como el cociente entre el tamaño de la intersección y el tamaño de la unión de dos conjuntos, constituye una de las métricas de similitud estructural más extendidas en tareas de minería de texto y recuperación de información precisamente por su sencillez de cálculo y su independencia del orden de los elementos comparados. En este caso concreto, se aplica sobre el conjunto de nombres de columna normalizados a minúsculas de cada propuesta, de modo que dos estructuras que comparten un 70 % o más de sus columnas (umbral configurado en la función) se consideran demasiado similares y, por tanto, no válidas como nueva propuesta.

Este cálculo se combina con una lógica de reintento acotado: el nodo invoca al modelo hasta un máximo de tres veces, comparando cada nueva propuesta con la estructura anterior mediante `estructuras_similares`, y aceptando la primera que resulte suficientemente distinta:

```
for _ in range(3):
```

```
    res = llm_complejo.invoke(prompt)
    nueva = extraer_json(res.content)
    if not nueva:
        continue
    if not estructura_anterior or not
    ↪ estructuras_similares(estructura_anterior, nueva):
        ...
        return {"estructura_tabla_propuesta": nueva, ...}
```

Si tras los tres intentos no se logra una propuesta suficientemente diferenciada, el sistema no bloquea la ejecución, sino que informa al investigador de la dificultad encontrada y le sugiere recurrir directamente a la opción de modificación manual de columnas, dando así continuidad al flujo sin comprometer la experiencia de uso.

De forma análoga a `modificar_categorias_node` (Sección 4.5.2), el nodo `modificar_estructura_node` recibe las instrucciones de modificación en lenguaje natural introducidas por el investigador a través de `confirmar_estructura_node`, y las incorpora al *prompt* junto con la estructura de columnas actual, solicitando al modelo que devuelva únicamente el bloque JSON modificado. La salida se trata igualmente como texto libre analizado mediante `extraer_json`, por el mismo motivo expuesto en la Sección 4.5.2: al tratarse de una edición abierta sobre una estructura ya existente a partir de una instrucción arbitraria, resulta más robusto validar la respuesta en código que confiar en el mecanismo de salida estructurada nativa. Por la misma exigencia de interpretación fiel de instrucciones abiertas, este nodo se asigna también al *slot* `llm_complejo` (Sección 4.9.4).

4.7.2. Generación de la Tabla y Descripción

Una vez confirmada la estructura, el nodo `generar_tabla_node` solicita al modelo que rellene la tabla completa en formato Markdown, imponiendo mediante el *prompt* restricciones muy estrictas sobre el contenido de cada celda (máximo de 8 a 12 palabras, prohibición expresa de dejar celdas vacías, uso obligatorio de la expresión "No especificado" ante la ausencia de un dato), de forma coherente con el requisito

RF-16. El nodo `describir_tabla_node`, por su parte, redacta un texto académico que explica cada uno de los criterios de comparación empleados (RF-17), reutilizando en su *prompt* la estructura de columnas ya confirmada para asegurar que la descripción se ciñe exactamente a las columnas realmente generadas.

4.8. Capa de Cierre

Esta capa está formada por los nodos `redactar_conclusion_node` y `revision_final_node`, y da soporte a los requisitos de redactar una conclusión que posicione la propuesta del investigador frente a los trabajos analizados (RF-20), consolidar en un único documento la introducción, el cuerpo, la descripción de la tabla, la propia tabla y la conclusión (RF-21), generar dicho documento en formato LaTeX (RF-22), insertar automáticamente los comandos de cita junto a cada mención de un trabajo relacionado (RF-23), y generar el entorno bibliográfico correspondiente (RF-24).

4.8.1. Redacción de Conclusión

El nodo `redactar_conclusion_node` sintetiza en un único párrafo el posicionamiento del trabajo del investigador frente al estado del arte analizado, integrando en el *prompt* tanto los trabajos analizados como el cuerpo de la sección y la tabla comparativa (si existe). El *prompt* exige de forma explícita evitar listas y repeticiones de contenido ya expuesto, y solicita el uso de conectores propios del discurso académico ("sin embargo", "no obstante", "cabe destacar que"), buscando reforzar la coherencia estilística con el resto de la sección redactada previamente en la capa anterior.

4.8.2. Creación del Documento LaTeX

El nodo `revision_final_node` es, con diferencia, el más complejo de todo el grafo, y da cumplimiento a los requisitos RF-21 a RF-24. Su *prompt* se estructura explícitamente en dos fases: una primera fase de revisión y mejora del contenido donde se le pide mejorar y pulir la introducción, la descripción de la tabla y, sobre todo, la conclusión, ancla esta

última a la ficha técnica del trabajo propio para que quede clara la posición del investigador frente al estado del arte, y una segunda fase de ensamblado, en la que fusiona todos los bloques ya mejorados (introducción, cuerpo, descripción de la tabla, tabla y conclusión) en un único documento LaTeX coherente, insertando de forma automática las citas cruzadas y generando el entorno bibliográfico correspondiente.

Ambas fases se construyen, además, de forma condicional en función de qué opciones haya activado el investigador durante la sesión. Si la tabla comparativa no está activa (RF-13) o la categorización no lo está (RF-09), el *prompt* omite por completo las instrucciones relativas a la tabla o a las categorías en lugar de mantenerlas fijas sobre un contenido que no existe, evitando así que el modelo reciba instrucciones sobre elementos ausentes del estado de la sesión (un riesgo de alucinación semántica análogo al descrito en la Sección 1.2).

Antes de invocar al modelo, el propio nodo construye de forma programática tanto la guía de mapeo entre cada trabajo analizado y su correspondiente clave de cita (`ref-1`, `ref-2`, ...) como el bloque completo del entorno `thebibliography`. Esta guía es, precisamente, lo que permite completar el requisito RF-23, donde el modelo no inventa ni elige libremente las claves de cita, sino que recibe ya resuelta la correspondencia entre cada trabajo relacionado y el identificador exacto que debe usar al citarlo, limitándose a decidir en qué punto del texto ya redactado resulta más natural insertar cada comando `\cite{}`:

```
1 for idx, t in enumerate(trabajos, 1):
2     titulo = t.get("titulo", "Sin titulo")
3     anio = t.get("anio") or t.get("year") or "s.f."
4     autores_str = ", ".join(t.get("autores", []))
5     guias_cita_latex.append(f"- Para el paper '{titulo}': Insertar el comando
      ↪  '\\cite{{ref-{idx}}}'")
6     items_bibliografia.append(f"\\bibitem{{ref-{idx}}} {autores_str}.
      ↪  ``{titulo},'' {anio}.")
```

Esta decisión de generar la bibliografía mediante código determinista, en lugar de delegar por completo en el LLM la generación de las claves de cita, es especialmente relevante desde el punto de vista de la fiabilidad del sistema, ya que, al tratarse de metadatos bibliográficos ya extraídos y validados en la capa de análisis (Sección 4.4), no tendría sentido exponerlos de nuevo al riesgo de alucinación descrito en la Sección 1.2 dejando que el modelo los reescriba libremente. El LLM se emplea, en cambio, para las tareas en las que su capacidad de generación de lenguaje natural resulta imprescindible: revisar y mejorar el contenido de introducción, descripción de tabla y conclusión (Fase 1 del *prompt*), decidir en qué punto exacto del texto ya redactado debe insertarse cada comando `\cite{}`, y transformar la tabla de Markdown a un entorno `table`/`tabular` de LaTeX bien formado.

El nodo incorpora, además, un mecanismo de contingencia (*fallback*) especialmente cuidado dado que se trata del último paso del flujo: si la invocación al modelo fallase por cualquier motivo, se construye igualmente un documento LaTeX mínimo concatenando los bloques de texto ya generados junto con el bloque de bibliografía (que, como se ha explicado, no depende del LLM), garantizando así que el investigador nunca se queda sin ningún resultado exportable, en cumplimiento del requisito RNF-06.

4.8.3. Red de Seguridad Post-Generación

Aunque el *prompt* de `revision_final_node` es extremadamente detallado en sus instrucciones de formato (orientación de la tabla, ancho de columnas, orden de las secciones o ausencia de bloques de código Markdown), la experiencia acumulada a lo largo del desarrollo mostró que ningún nivel de detalle en el *prompt* garantiza por sí solo que el modelo cumpla todas las reglas en todos los casos. Por ello, tras la invocación al modelo, el texto devuelto se somete a una cadena de funciones de post-procesado deterministas antes de darse por definitivo:

```
1 texto_final_latex = _limpiar_fences_markdown(texto_final_latex)
```

```
2 texto_final_latex = _forzar_orden_secciones(texto_final_latex,
  ↪ marcadores_esperados)
3 texto_final_latex = _forzar_parrafo_tras_titulos_categoria(texto_final_latex)
4 texto_final_latex = _eliminar_entorno_landscape(texto_final_latex)
5 texto_final_latex = _eliminar_longtable(texto_final_latex)
6 texto_final_latex = _forzar_wrap_columnas_tabla(texto_final_latex)
7 texto_final_latex = _ajustar_anchos_columnas_segun_contenido(texto_final_latex)
8 texto_final_latex = _espaciar_columnas_tabla(texto_final_latex)
9 texto_final_latex = _asegurar_resizebox_tabla(texto_final_latex)
```

Cada una de estas funciones corrige un fallo de formato concreto observado empíricamente durante el desarrollo, en lugar de una regla genérica anticipada de antemano. `_limpiar_fences_markdown` elimina los delimitadores de bloque de código (```` ```latex ... ``` ````) que el modelo añade a veces pese a la instrucción explícita de no hacerlo, `_forzar_orden_secciones` reconstruye el documento recortando y reordenando sus bloques a partir de una serie de marcadores de comentario LaTeX invisibles que el propio *prompt* le pide insertar al inicio de cada sección, para el caso en que el modelo entregue fragmentos de una sección intercalados con los de otra, y `_ajustar_anchos_columnas_segun_contenido` recalcula el ancho `p{Xcm}` de cada columna de la tabla a partir de la palabra más larga presente en cualquiera de sus celdas, ya que el modelo tiende a dimensionar las columnas según el contenido de las celdas de datos (por ejemplo, *"Sí"*/*"No"*) sin tener en cuenta que la cabecera de esa misma columna puede ser considerablemente más larga.

Esta capa de post-procesado es, en definitiva, la aplicación más extensa dentro del sistema del principio defensivo ya introducido en la Sección 4.5.2: tratar la salida del LLM como una propuesta que el propio software revisa y corrige, en lugar de confiar ciegamente en que el modelo cumpla el *prompt* al pie de la letra, algo especialmente relevante en `revision_final_node`, dado que es el último paso del flujo y sus fallos de formato son los que con más probabilidad llegarían intactos hasta el documento final entregado al investigador.

4.9. Capa de Orquestación

Esta capa no se corresponde con un nodo de procesamiento de contenido, sino con la infraestructura que hace posible el funcionamiento coordinado de todas las capas anteriores. Agrupa dos aspectos complementarios: por un lado, el control de la ejecución propiamente dicho (Secciones 4.9.1 a 4.9.3); por otro, la resolución de qué modelo concreto emplea cada nodo en tiempo de ejecución (Sección 4.9.4). Da soporte a los requisitos de detener la ejecución en cada punto de decisión relevante a la espera de una respuesta del investigador (RF-25), reanudar la ejecución exactamente desde el punto en que fue detenida (RF-26), y conservar el estado completo de la interacción durante toda la sesión, sin perder ninguna decisión previa al reanudar la ejecución (RNF-07).

4.9.1. Definición del Grafo

La construcción del grafo se realiza mediante la clase `StateGraph` de LangGraph, parametrizada con el `TypedDict AgentState` descrito en la Sección 3.2.2. Cada nodo se registra mediante `graph.add_node(...)`, las transiciones deterministas mediante `graph.add_edge(...)`, y las transiciones condicionales mediante `graph.add_conditional_edges(...)`, especificando en este último caso tanto la función de enrutamiento (como `gateway_categorias`, ya mostrada en la Sección 4.5.1) como el diccionario de mapeo entre sus posibles valores de retorno y los nodos destino. Esta correspondencia directa entre el código y el diagrama de la Sección 3.2.3 es, precisamente, una de las ventajas prácticas del paradigma de grafos de estado señalada en el Estado del Arte (Sección 2.2.4): el propio código del grafo actúa como documentación viva de su comportamiento.

4.9.2. Persistencia del Estado

El grafo se compila indicando un *checkpointer*, en este caso una instancia de `MemorySaver`:

```
app = graph.compile(checkpointer=MemorySaver())
```

Este componente es el que hace posible que, tras cada `interrupt()`, el estado acumulado hasta ese momento (documentos ya analizados, categorías ya propuestas, textos ya redactados) no se pierda, sino que quede asociado a un identificador de sesión (`thread_id`) y pueda recuperarse en la siguiente llamada mediante `Command(resume=...)`, dando cumplimiento al requisito RNF-07. Conviene señalar que, al tratarse de un `MemorySaver`, la persistencia es volátil (en memoria), lo que resulta adecuado para el alcance de este TFG, pero que constituye una limitación conocida de cara a un uso en producción con sesiones de muy larga duración o con necesidad de recuperación ante reinicios del servidor, aspecto que se retomará como posible línea de trabajo futuro en el Capítulo 7.

4.9.3. Bucle Principal de Ejecución

`app.stream(entrada, config, stream_mode="values")` no devuelve de golpe el resultado final del grafo. Devuelve un generador de Python que va produciendo, uno por uno, el estado completo del `AgentState` cada vez que un nodo termina de ejecutarse. Por eso se usa el modo `"values"`. Cada elemento producido es el diccionario de valores del estado en ese instante, no solo el nombre del nodo o los campos que acaba de modificar. A cada uno de estos diccionarios se le llama informalmente un *chunk*. Por ejemplo, justo después de que `buscar_pdfs_node` (Sección 4.3) termine su ejecución, el *chunk* correspondiente contendría, entre otras claves del estado, algo equivalente a esto:

```
{
    "trabajos_pdf": ["trabajos_relacionados/paper1.pdf",
    ↪ "trabajos_relacionados/paper2.pdf"],
    "messages": [AIMessage(content="Bienvenido a..."), ...,
    ↪ AIMessage(content="Exploración completada...")],
    ...
```

```
5 }
```

Cuando el grafo llega a un nodo que invoca `interrupt()`, por ejemplo `seleccionar_idioma_node`, el *stream* no se corta silenciosamente. Produce un último *chunk* especial que añade al estado una clave adicional que no forma parte del `AgentState` definido por el desarrollador, `"__interrupt__"`. Esta clave la inyecta automáticamente LangGraph. Su valor es una lista con un único objeto `Interrupt`, cuyo atributo `.value` contiene exactamente el argumento que el nodo pasó a `interrupt(...)`. En el caso de `seleccionar_idioma_node`, ese valor es una cadena vacía en la primera llamada, o el mensaje de error si la opción introducida no fue válida. Es precisamente la presencia de esta clave la que la lógica de control usa para distinguir si el grafo se ha pausado o si simplemente ha terminado un nodo intermedio.

Tanto la versión de consola (incluida al final de `app1.py`) como la interfaz Gradio (Sección 4.10) comparten la misma lógica de control alrededor del grafo compilado. Ambas invocan `app.stream(...)` para obtener los mensajes generados hasta la siguiente pausa, comprueban si el último elemento del *stream* contiene la clave especial `"__interrupt__"` y, en tal caso, solicitan una respuesta al investigador y reanudan la ejecución mediante `Command(resume=respuesta)`. Este patrón se repite en un bucle `while True` hasta que el grafo alcanza el nodo `END`. Por ejemplo, cuando el grafo llega a `seleccionar_idioma_node` (Sección 4.3.1), el *stream* se detiene y expone la clave `"__interrupt__"` con el mensaje a mostrar. El bucle lo detecta, presenta las dos opciones de idioma al investigador y recoge su respuesta, por ejemplo `"1"`. Esa respuesta se reinyecta en el grafo mediante `Command(resume="1")`, y en ese momento la ejecución se reanuda exactamente en el punto en que quedó pausada.

Sobre el código real de la versión de consola, este patrón se traduce en el siguiente fragmento, que sirve como ejemplo concreto de todo lo anterior.

```
1 while True:
2     for chunk in app.stream(entrada, config, stream_mode="values"):
```

```
3           ultimo_chunk = chunk
4           # ... (impresion de mensajes nuevos por consola)
5
6       if "__interrupt__" not in ultimo_chunk:
7           break  # El grafo ha llegado a END
8
9       valor_interrupcion = ultimo_chunk["__interrupt__"][0].value
10      if valor_interrupcion:
11          print(f"\n Agente AI:\n{valor_interrupcion}")
12
13      respuesta_usuario = input("\n> ")
14      entrada = Command(resume=respuesta_usuario)
```

Siguiendo el ejemplo de `seleccionar_idioma_node`. En la primera vuelta del bucle `for`, el último *chunk* recibido es el que contiene `"__interrupt__"`. Como `valor_interrupcion` está vacío (la primera llamada a `interrupt()` del nodo, Sección 4.3.1), no se imprime nada adicional y se solicita directamente la respuesta del investigador mediante `input()`. Si el investigador escribe `"1"`, la variable `entrada` deja de ser el `AgentState` inicial y pasa a ser `Command(resume="1")`. En la siguiente vuelta del bucle exterior, ese objeto se pasa de nuevo a `app.stream(entrada, config, ...)`. Es en este punto donde interviene el *checkpointer* ya descrito anteriormente en este capítulo. Como `config` conserva el mismo `thread_id` durante toda la sesión, LangGraph recupera el estado exacto que había quedado persistido en el momento de la pausa. La ejecución de `seleccionar_idioma_node` se reanuda justo en la línea donde se invocó `interrupt()`, haciendo que esa llamada devuelva ahora la cadena `"1"` en lugar de bloquearse de nuevo, y el nodo continúa su ejecución con normalidad a partir de ahí, sin que el resto del grafo ni los nodos ya ejecutados se vean afectados.

4.9.4. Motor de Configuración de Modelos

El núcleo del agente incorpora el Motor de Configuración, responsable de resolver qué modelo concreto debe emplearse en cada uno de los dos *slots* de tarea (simple y complejo), dando soporte a los requisitos RF-28 a RF-32 y RNF-08. Antes de entrar en el detalle de su implementación, conviene dejar constancia explícita de cómo se reparten los nodos del grafo entre ambos *slots*:

- `llm_simple`: `definir_tematica_node`, `analizar_trabajos_node`, `evaluar_categorizacion_node`, `redactar_introduccion_node`, `recomendar_tabla_node`, `describir_tabla_node`, `redactar_conclusion_node`.
- `llm_complejo`: `proponer_categorias_node`, `modificar_categorias_node`, `redactar_trabajos_relacionados_node`, `proponer_estructura_node`, `modificar_estructura_node`, `generar_tabla_node`, `revision_final_node`.

Este reparto no es arbitrario: los nodos asignados a `llm_complejo` son, en todos los casos, aquellos cuya tarea exige razonar sobre un volumen de información considerablemente mayor, generar una salida estructuralmente más exigente, o interpretar una instrucción abierta en lenguaje natural para aplicarla correctamente sobre una estructura de datos ya existente (como ocurre en `modificar_categorias_node` y `modificar_estructura_node`, que deben interpretar peticiones de modificación arbitrarias formuladas libremente por el investigador (Secciones 4.5.2 y 4.7.1) y aplicarlas con precisión sin alterar el resto de la estructura), mientras que los asignados a `llm_simple` resuelven tareas más acotadas y con un espacio de respuesta más restringido (clasificar, evaluar, redactar un bloque breve). Este criterio de asignación es una aplicación directa, aunque decidida manualmente por el desarrollador en lugar de por un clasificador automático, del principio de *LLM routing* discutido en la Sección 2.1.2 [13].

El primer elemento de este mecanismo es un registro de proveedores admitidos, `PROVEEDORES_LLM`, que centraliza en un único diccionario toda la información necesaria para instanciar un modelo de cada proveedor: su etiqueta legible, el nombre de la variable

de entorno donde se espera su clave de API (o `None` si el proveedor es local y no requiere clave) y su modelo por defecto:

```
PROVEEDORES_LLM = {
    "ollama": {
        "etiqueta": "Ollama (modelo local)",
        "api_key_env": None,
        "modelo_defecto": "llama3.2:3b",
    },
    "google_genai": {
        "etiqueta": "Google Gemini (API)",
        "api_key_env": "GOOGLE_API_KEY",
        "modelo_defecto": "gemini-2.5-flash",
    },
    "openai": {
        "etiqueta": "OpenAI (API)",
        "api_key_env": "OPENAI_API_KEY",
        "modelo_defecto": "gpt-4o-mini",
    },
    "anthropic": {
        "etiqueta": "Anthropic Claude (API)",
        "api_key_env": "ANTHROPIC_API_KEY",
        "modelo_defecto": "claude-3-5-haiku-latest",
    },
}
```

Sobre este registro se apoya la función `crear_llm`, que emplea la utilidad `init_chat_model` de LangChain para instanciar la clase concreta correspondiente a cada proveedor (`ChatOllama`, `ChatGoogleGenerativeAI`, `ChatOpenAI` o `ChatAnthropic`) a partir únicamente de un nombre de proveedor, un nombre de modelo y una temperatura, resolviendo también de forma automática la clave de API desde las variables de entorno

cuando el proveedor la requiere:

```
def crear_llm(config_slot: dict):
    proveedor = config_slot.get("proveedor", "ollama")
    info_proveedor = PROVEEDORES_LLM.get(proveedor, PROVEEDORES_LLM["ollama"])
    kwargs = {
        "model": config_slot.get("modelo") or info_proveedor["modelo_defecto"],
        "model_provider": proveedor,
        "temperature": config_slot.get("temperature", 0.2),
    }
    clave_env = info_proveedor.get("api_key_env")
    if clave_env:
        valor = os.getenv(clave_env)
        if valor:
            kwargs["api_key"] = valor
    return init_chat_model(**kwargs)
```

Gracias a este diseño, incorporar un nuevo proveedor al sistema se reduce a añadir una entrada más al diccionario `PROVEEDORES_LLM` (siempre que el paquete de integración correspondiente de LangChain esté instalado), sin necesidad de tocar ni un solo nodo del grafo ni la lógica de negocio del resto del sistema, dando cumplimiento directo al requisito de extensibilidad RNF-08.

El segundo elemento, y el más relevante desde el punto de vista de las decisiones de ingeniería tomadas, es la clase `LLMPerezoso`, que envuelve cada *slot* y retrasa la construcción real del modelo hasta su primer uso efectivo:

```
class LLMPerezoso:
    def __init__(self, slot: str):
        self._slot = slot
        self._modelo = None
```

```
5
6       def _resolver(self):
7           if self._modelo is None:
8               config = cargar_configuracion_llms()[self._slot]
9               self._modelo = crear_llm(config)
10          return self._modelo
11
12      def invoke(self, *args, **kwargs):
13          return self._resolver().invoke(*args, **kwargs)
14
15      def with_structured_output(self, *args, **kwargs):
16          return self._resolver().with_structured_output(*args, **kwargs)
17
18  llm_simple = LLMPerezoso("simple")
19  llm_complejo = LLMPerezoso("complejo")
```

Esta clase es un ejemplo directo de aplicación del patrón *Proxy*, y más concretamente de su variante de proxy virtual orientada a la inicialización perezosa (*lazy initialization*): en lugar de que el cliente (en este caso, cada nodo del grafo) interactúe directamente con el objeto real y costoso de construir (el modelo de chat concreto), interactúa con un sustituto ligero que comparte su misma interfaz (`invoke`, `with_structured_output`) y que solo construye el objeto real la primera vez que resulta estrictamente necesario [39]. La motivación de esta decisión fue, en este caso, resolver un problema muy concreto detectado durante el desarrollo: en una versión anterior del sistema, los modelos se construían directamente a nivel de módulo (es decir, en el momento de hacer `import app1`), de modo que si faltaba, por ejemplo, la clave de API de Gemini, la simple carga del módulo lanzaba una excepción y el programa era incapaz de arrancar, incluso si esa clave concreta no fuera a usarse en la ejecución en curso. Con `LLMPerezoso`, un proveedor mal configurado solo produce un error en el momento en que un nodo intenta usarlo realmente, permitiendo que el resto del sistema (incluida la propia página de Ajustes, que es precisamente donde el investigador corregiría dicha configuración) funcione con normalidad.

Una consecuencia práctica de este diseño, ya anticipada como restricción R-04 en el Capítulo 3, es que la configuración se relee de `llm_config.json` en el momento de la primera resolución de cada *slot*, y no en el arranque de la aplicación: esto permite que los cambios guardados desde Ajustes surtan efecto sin reiniciar el proceso, siempre que ningún nodo haya usado ya ese *slot* concreto durante la sesión en curso. Se trata de una limitación conocida y asumida, derivada directamente de mantener la implementación de `LLMPerezoso` simple (el objeto solo se resuelve una vez, no se vuelve a comprobar en cada invocación), y que se comunica explícitamente al investigador desde la propia interfaz, como se detalla en la siguiente sección.

4.10. Interfaz de Usuario

Descrito ya el funcionamiento interno del grafo y de su motor de configuración, resta explicar cómo el investigador interactúa realmente con el sistema. Como se adelantó en la Sección 3.2.1, el módulo `gui.py` no contiene lógica de negocio propia, sino que actúa exclusivamente como una traducción entre el modelo de ejecución pausable del grafo descrito en la sección anterior y el modelo de interacción de un *chatbot* web, dando soporte al requisito RF-27.

4.10.1. Página de Chat

La función `ejecutar_hasta_pausa` constituye el núcleo de esta capa. Antes de ver su código conviene entender por qué está construida como un generador de Python, es decir, una función que usa la palabra clave `yield` en lugar de `return`. Una función normal ejecutaría todo el bucle sobre `app.stream(...)` de golpe y solo devolvería un resultado al final, dejando al investigador viendo una pantalla en blanco durante todo el tiempo que tarde el grafo en llegar a la siguiente pausa o al final. Un generador, en cambio, puede ir devolviendo resultados parciales cada vez que llega a una instrucción `yield`, sin terminar su ejecución. Esto encaja de forma natural con el modelo de actualización incremental de Gradio, que puede refrescar el componente `chatbot` cada vez que recibe un nuevo

valor del generador, en lugar de esperar a que termine toda la función. Así, si un nodo produce varios `AIMessage` seguidos, por ejemplo `analizar_trabajos_node` (Sección 4.4), el investigador ve aparecer cada ficha técnica en el chat según se van generando, no todas de golpe al final del nodo.

Se trata de un generador de Python que va emitiendo el historial de conversación acumulado a medida que el grafo produce nuevos mensajes, hasta detectar que la ejecución se ha pausado (nuevamente, comprobando la presencia de `"__interrupt__"` en el último elemento del *stream*) o ha finalizado. Cada valor emitido es una tupla `(historial, terminado)`, donde `historial` es la lista de mensajes ya traducida al formato que espera `gr.Chatbot` (mediante `mensajes_a_historial`) y `terminado` es un booleano que indica si el grafo ha llegado a `END`. Esta segunda componente permite a las funciones que consumen el generador, `comenzar_y_iniciar` y `manejar_envio`, saber si deben dejar el campo de texto activo a la espera de una nueva respuesta del investigador o si, por el contrario, deben desactivarlo porque el proceso ha concluido.

```
1  def ejecutar_hasta_pausa(entrada, config):
2      ultimo_chunk = {}
3      for chunk in app.stream(entrada, config, stream_mode="values"):
4          ultimo_chunk = chunk
5          if "__interrupt__" not in chunk and "messages" in chunk:
6              yield mensajes_a_historial(chunk["messages"]), False
7
8      if "__interrupt__" in ultimo_chunk:
9          valor = ultimo_chunk["__interrupt__"][0].value
10         historial = mensajes_a_historial(ultimo_chunk.get("messages", []))
11         if valor:
12             historial.append({"role": "assistant", "content": str(valor)})
13         yield historial, False
14         return
15
16     guardar_state(ultimo_chunk, "state_guardado.json")
```

```
17      guardar_documento_latex(ultimo_chunk, "related_works.tex")
18      historial = mensajes_a_historial(ultimo_chunk.get("messages", []))
19      historial.append({"role": "assistant", "content": MENSAJE_FINALIZADO})
20      yield historial, True
```

Esta comprobación se apoya deliberadamente en el propio *stream* y no en el estado general del grafo. La razón es que un mismo nodo puede lanzar más de una interrupción durante su ejecución, como ocurre en `confirmar_categorias_node` (Sección 4.5) cuando el investigador elige modificar manualmente una propuesta. En ese tipo de casos, consultar el estado general del grafo deja de reflejar con fiabilidad si la ejecución sigue pausada, mientras que inspeccionar directamente el *stream* sí lo hace. Es un ejemplo más de cómo una decisión aparentemente menor responde, en la práctica, a un caso límite concreto detectado durante el desarrollo.

Por lo demás, la interfaz Gradio (`gui.py`) se limita a mantener un identificador de sesión (`thread_id`) por conversación en un objeto `gr.State`, y a conectar los eventos de la interfaz (envío de un mensaje, pulsación del botón "Enviar") con las funciones `iniciar_conversacion` y `manejar_envio`, ambas construidas como generadores sobre `ejecutar_hasta_pausa` para permitir que la interfaz se actualice progresivamente a medida que el grafo produce nuevos mensajes, en lugar de bloquearse hasta la siguiente pausa.

4.10.2. Página de Ajustes

Frente a la página de Chat, centrada en la interacción conversacional propia del flujo de redacción, la página de Ajustes concentra en un único lugar toda la configuración que el investigador necesitaría tocar solo ocasionalmente, por ejemplo, qué documentos analizar, qué credenciales usar y qué modelos emplear, evitando así que tenga que editar manualmente ficheros o variables de entorno del sistema. Agrupa tres bloques de gestión independientes, cada uno respaldado por su propio conjunto de funciones.

Gestión de documentos PDF (RF-31). Las funciones `anadir_pdfs` y

`eliminar_pdfs` permiten al investigador incorporar o retirar documentos del directorio de trabajos relacionados directamente desde la interfaz, sin necesidad de acceder al sistema de archivos:

```
def anadir_pdfs(archivos):
    if not archivos:
        return gr.update(choices=_listar_pdfs()), gr.update(), None
    os.makedirs(CARPETA_PDFS, exist_ok=True)
    copiados = []
    for ruta in archivos:
        destino = os.path.join(CARPETA_PDFS, os.path.basename(ruta))
        shutil.copy(ruta, destino)
        copiados.append(os.path.basename(ruta))
    mensaje = f"Anadido(s) {len(copiados)} PDF(s): {', '.join(copiados)}"
    return gr.update(choices=_listar_pdfs(), value=[]), mensaje, None
```

Esta funcionalidad complementa, sin sustituirlo, al mecanismo de detección automática de PDFs ya descrito en la Sección 4.3.2 (`buscar_pdfs_node`). Dicho nodo sigue localizando los documentos presentes en el directorio en el momento de ejecutarse, con independencia de si estos se colocaron manualmente en el sistema de archivos o se subieron a través de esta página de Ajustes.

Gestión de credenciales (RF-32). La función `guardar_env` persiste en el fichero `src/.env` las claves de API introducidas por el investigador para los proveedores en la nube que decida utilizar, y además las refleja de inmediato en las variables de entorno del proceso en curso (`os.environ`), de modo que, si el *slot* correspondiente todavía no se ha resuelto (véase `LLMPerezoso`, Sección 4.9.4), la nueva clave se recoja sin necesidad de reiniciar la aplicación:

```
def guardar_env(*valores):
    contenido = "\n".join(f"{clave}={valor}" for clave, valor in
    ↪  zip(CLAVES_ENV, valores)) + "\n"
```

```
    with open(RUTA_ENV, "w", encoding="utf-8") as f:
        f.write(contenido)
    for clave, valor in zip(CLAVES_ENV, valores):
        if valor:
            os.environ[clave] = valor
    return (
        "Variables guardadas en `src/.env` (...). "
        "Si algun modelo ya se habia usado en esta sesion, reinicia la
        ↪ aplicacion "
        "(`python src/gui.py`) para que recoja la clave nueva."
    )
```

El propio mensaje de confirmación mostrado al investigador advierte de la limitación descrita en la Sección 4.9.4 (un *slot* ya resuelto no recogerá el cambio hasta reiniciar), lo que constituye una decisión deliberada de comunicar honestamente al usuario final una restricción técnica conocida del sistema, en lugar de ocultarla.

Gestión de modelos LLM (RF-28, RF-29, RF-30). La página de Ajustes ofrece, para cada uno de los dos *slots*, un desplegable de proveedor (construido dinámicamente a partir de las claves de `PROVEEDORES_LLM`, por lo que un nuevo proveedor añadido al registro aparece automáticamente en la interfaz sin cambios adicionales), un campo de texto para el nombre del modelo y un control numérico para la temperatura; a estos se suma un único control numérico adicional, compartido por ambos *slots*, para el límite de caracteres de análisis descrito en la Sección 4.4 (RF-07), dado que dicho límite solo afecta al uso que `analizar_trabajos_node` hace de `llm_simple`. La función `guardar_modelos_llm` persiste toda esta configuración en `src/llm_config.json`:

```
def guardar_modelos_llm(proveedor_simple, modelo_simple, temp_simple,
                        proveedor_complejo, modelo_complejo, temp_complejo,
                        limite_caracteres_analisis):
    config = {
```

```
5         "simple": {"proveedor": proveedor_simple, "modelo":
          ↪  modelo_simple.strip(), "temperature": temp_simple},
6         "complejo": {"proveedor": proveedor_complejo, "modelo":
          ↪  modelo_complejo.strip(), "temperature": temp_complejo},
7         "limite_caracteres_analisis": int(limite_caracteres_analisis),
8     }
9     guardar_configuracion_llms(config)
10    return "Configuracion de modelos guardada en `src/llm_config.json`. ..."
```

4.11. Funciones Auxiliares

Por último, y de forma más breve dado su carácter de soporte, cabe mencionar un conjunto de funciones auxiliares que, sin constituir nodos del grafo, son empleadas de forma transversal por varias de las capas ya descritas:

- `extraer_json` / `extraer_json_puro`: funciones basadas en expresiones regulares empleadas para recuperar un bloque JSON válido de una respuesta de texto libre del modelo, utilizadas en los nodos que no emplean el mecanismo de salida estructurada nativa (Secciones 4.5.2 y 4.7.1).

- `_normalizar_columnas`: función de blindaje empleada en la capa de tabla comparativa (Sección 4.7.1) que reduce a una lista plana de *strings* el campo `columnas` de una propuesta de estructura, para el caso en que el modelo devuelva en su lugar una lista de objetos (por ejemplo, {`"nombre": ..., "tipo": ...`}), garantizando así que el resto del *pipeline* (`estructuras_similares`, mensajes al investigador, `generar_tabla_node`) siempre reciba el formato que espera.

- `limpiar_texto_academico`: función de normalización de texto empleada para corregir artefactos de espaciado ocasionales en el texto extraído de los PDF.

- `guardar_state` y `guardar_documento_latex`: funciones de persistencia en disco del estado final de la sesión y del documento LaTeX generado, reutilizadas de forma

idéntica tanto por la versión de consola como por la interfaz Gradio, lo que evidencia de nuevo la separación de responsabilidades entre la lógica de negocio (definida una única vez en `app1.py`) y las distintas capas de presentación que la reutilizan.

Capítulo 5

Caso de Uso

Contenidos

5.1. Definición del Caso 90

5.2. Ejecución de la Herramienta 92

5.3. Resultados 104

5.3.1. Introducción de Related Works 105

5.3.2. Cuerpo de Related Works 106

5.3.3. Descripción y Tabla Comparativa 107

5.3.4. Conclusión 109

5.4. Detalles y Hallazgos Adicionales del Caso de Uso 110

5.4.1. Comparación de estrategias de categorización 110

5.4.2. Confirmación de una decisión de diseño: redacción intermedia en español y traducción final 111

5.4.3. El LLM complejo resolviendo fallos del LLM simple 112

5.4.4. Otros detalles destacados 113

5.4.5. Síntesis 114

Con el objetivo de validar el funcionamiento de la herramienta desarrollada en un escenario realista, en este capítulo se aplica el sistema al proceso de redacción de la sección *Related Works* de un artículo científico real, publicado por miembros de un grupo de investigación de la Universidad de Extremadura. Esta elección no es casual, ya que al disponer del artículo original completo, incluyendo su propia sección *Related Works* redactada manualmente por los autores y su tabla comparativa, es posible contrastar de forma directa el resultado producido por el agente con un trabajo de referencia elaborado por investigadores expertos en la materia, en lugar de limitarse a una evaluación puramente subjetiva del texto generado. La ejecución descrita en este capítulo se documentó mediante capturas de pantalla de cada paso del proceso, lo que permite analizar no solo el documento final, sino también el comportamiento intermedio del sistema.

5.1. Definición del Caso

El artículo elegido como caso de uso es "SimulateIoT: Domain Specific Language to Design, Code Generation and Execute IoT Simulation Environments" [40], publicado en la revista *IEEE Access* en 2021 por José A. Barriga, Pedro J. Clemente, Encarna Sosa-Sánchez y Álvaro E. Prieto, del grupo de investigación Quercus Software Engineering de la Universidad de Extremadura. El artículo propone un enfoque de Desarrollo Dirigido por Modelos (*Model-Driven Development*) para diseñar, generar código y ejecutar entornos de simulación de sistemas IoT, articulado en torno a un metamodelo propio, un editor gráfico y una transformación modelo-a-texto que genera automáticamente microservicios desplegables mediante contenedores Docker.

Para reproducir la interacción de un investigador real con la herramienta, en el nodo `definir_tematica_node` (véase Sección 4.3.1) se introdujo una descripción en lenguaje natural del artículo, elaborada a partir del resumen (*abstract*) e introducción del propio *paper*: se describió SimulateIoT como un lenguaje específico de dominio y una metodología asociada que permite modelar entornos de simulación IoT a alto nivel

de abstracción (sensores, actuadores, nodos *fog* y *cloud*), generar automáticamente el código necesario mediante transformaciones modelo-a-texto, y desplegar la simulación resultante como microservicios *Dockerizados* comunicados mediante un protocolo de publicación-suscripción, sin que el usuario tenga que escribir código manualmente.

Como conjunto de trabajos relacionados, se emplearon 14 de los 17 trabajos citados originalmente en la sección *Related Works* del artículo. Los tres trabajos excluidos fueron:

- **Bevywise IoT Simulator:** se trata de la página web comercial de un producto de simulación, no de un artículo científico con estructura analizable (sin autores, año de publicación ni metodología descrita de forma académica), por lo que no resultaba adecuado para el tipo de ficha técnica que extrae `analizar_trabajos_node`.
- **COMFIT** (de Farias et al., *Future Generation Computer Systems*, 2017): el artículo se encuentra detrás de un muro de pago en Elsevier/ScienceDirect y no fue posible localizar una versión de acceso abierto en el momento de realizar este caso de uso.
- **Siafu:** al igual que Bevywise, es la página de un proyecto de software de código abierto, no un artículo académico, por lo que carecía del contenido estructurado necesario para el análisis.

Esta exclusión de fuentes no académicas o inaccesibles es, en sí misma, una observación interesante sobre las limitaciones prácticas del sistema, ya que al depender de la extracción de texto de un PDF (Sección 4.3.2), la herramienta requiere necesariamente que el trabajo relacionado exista como documento académico convencional, quedando fuera de su alcance recursos como páginas de producto o repositorios de código, que sin embargo sí aparecen citados con normalidad en artículos científicos del ámbito de la ingeniería del software.

La configuración de modelos empleada (Figura 5.3) fue: `llm_simple` → Ollama, `llama3.2:3b`, temperatura 0.1; `llm_complejo` → Google Gemini, `gemini-2.5-flash`, temperatura 0.3; truncamiento de análisis a 12.000 caracteres por documento. Esta configuración confirma de forma directa, sobre el sistema real en ejecución, la arquitectura híbrida local/nube descrita en la Sección 3.2.1 y la justificación de *LLM routing* de la Sección 2.1.2 [13]. La Figura 5.4 confirma, además, que la clave de API de

Google (`GOOGLE_API_KEY`) estaba correctamente configurada para dar servicio al *slot* `llm_complejo`.

5.2. Ejecución de la Herramienta

La ejecución arranca en la pantalla de Inicio de la interfaz web, que ofrece dos accesos: "Comenzar", que inicia el flujo conversacional sobre el grafo de LangGraph, y "Ajustes", que da acceso a la página de configuración descrita en la Sección 4.10.2.



Figura 5.1: Pantalla de Inicio de la interfaz web.

Antes de arrancar el flujo, se utiliza la página de Ajustes para dejar preparado todo lo que el sistema necesitaría durante la ejecución, en lugar de recurrir a la manipulación manual del sistema de archivos que exigían versiones anteriores de la herramienta. En primer lugar, se gestionan desde la interfaz los 14 documentos PDF seleccionados como trabajos relacionados (Sección 5.1), comprobándose que la herramienta permite tanto visualizar el listado completo como añadir o eliminar documentos mediante la zona de arrastre, dando cumplimiento directo al requisito RF-31:

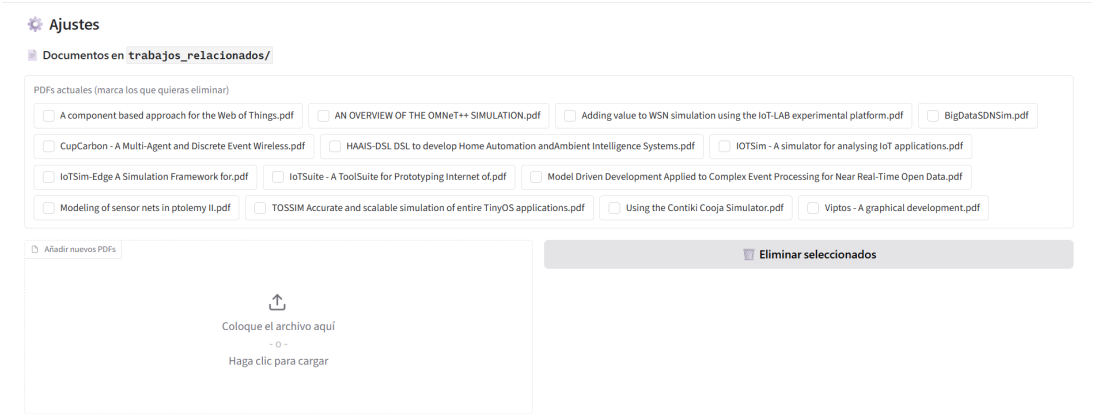


Figura 5.2: Gestión de documentos PDF desde la página de Ajustes.

A continuación, se configuran los dos *slots* de modelo que gobiernan el comportamiento del Motor de Configuración descrito en la Sección 4.9.4. El `llm_simple` se deja asignado a un modelo local servido mediante Ollama (`llama3.2:3b`, temperatura 0.1), mientras que `llm_complejo` se asigna a un modelo en la nube (Google Gemini `gemini-2.5-flash`, temperatura 0.3), manteniendo además en 12.000 el número de caracteres enviados por documento al modelo simple durante el análisis. Esta pantalla materializa de forma tangible la arquitectura híbrida y configurable descrita en las Secciones 2.1.2, 3.2.1 y 4.9.4:



Figura 5.3: Configuración de los dos *slots* de modelo LLM (simple y complejo).

Por último, se verifica que la clave de API de Google (`GOOGLE_API_KEY`) está correctamente almacenada en `src/.env`, condición necesaria para que el *slot* `llm_complejo` pueda resolverse sin errores en el momento de su primer uso (RF-32):



Figura 5.4: Verificación de la clave de API de Google en la página de Ajustes.

Con la configuración lista, se pulsa “Comenzar”. El sistema muestra el mensaje de bienvenida y solicita el idioma de salida del documento, primer punto de interacción humana del grafo (`seleccionar_idioma_node`, RF-01):

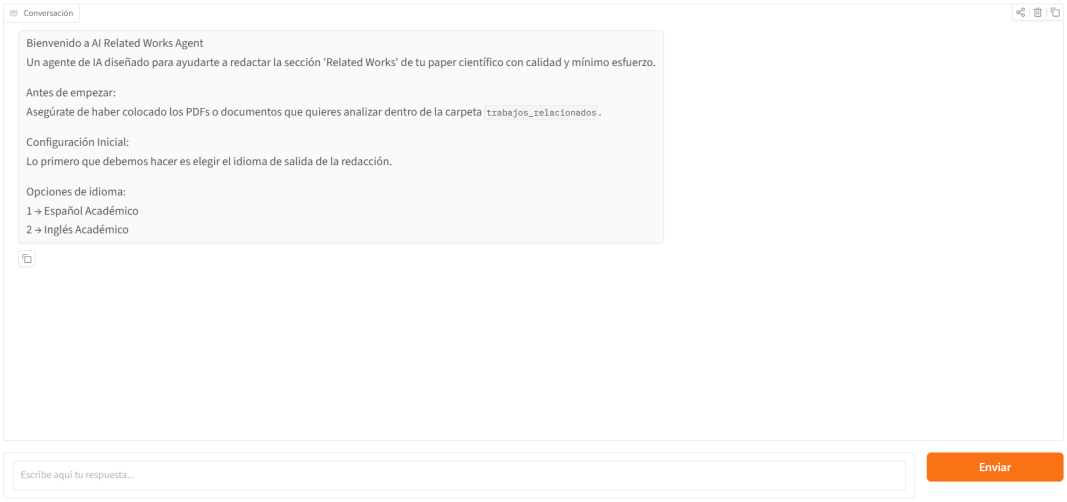


Figura 5.5: Mensaje de bienvenida y selección del idioma de salida.

Se selecciona la opción de inglés académico. El sistema confirma la elección y solicita a continuación, en el mismo turno, una descripción en lenguaje natural del artículo propio del investigador:

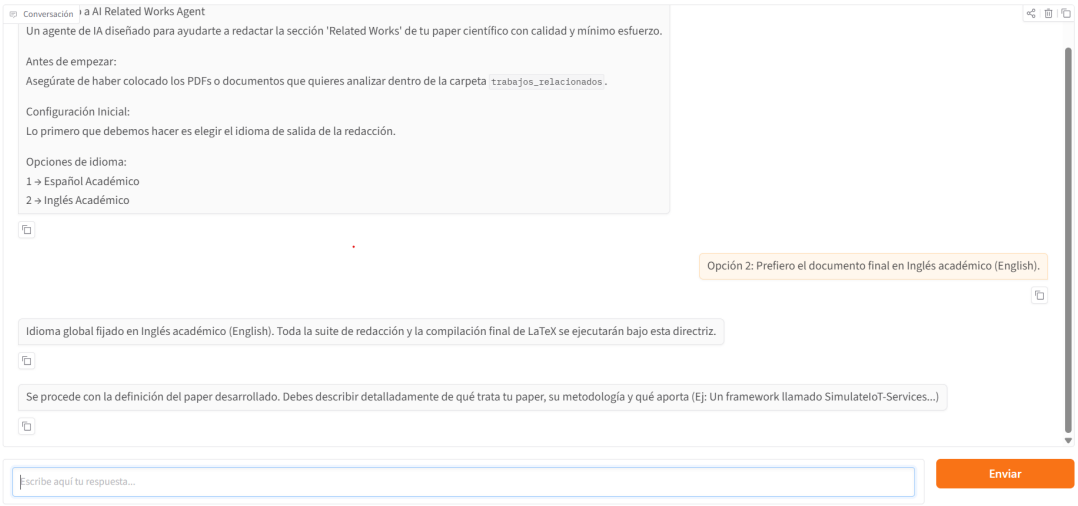


Figura 5.6: Confirmación del idioma seleccionado y solicitud de la descripción del artículo propio.

Se introduce una descripción de SimulateIoT elaborada a partir de su resumen e

introducción (Sección 5.1), que el nodo `definir_tematica_node` recoge para iniciar su estructuración (RF-02, RF-03):

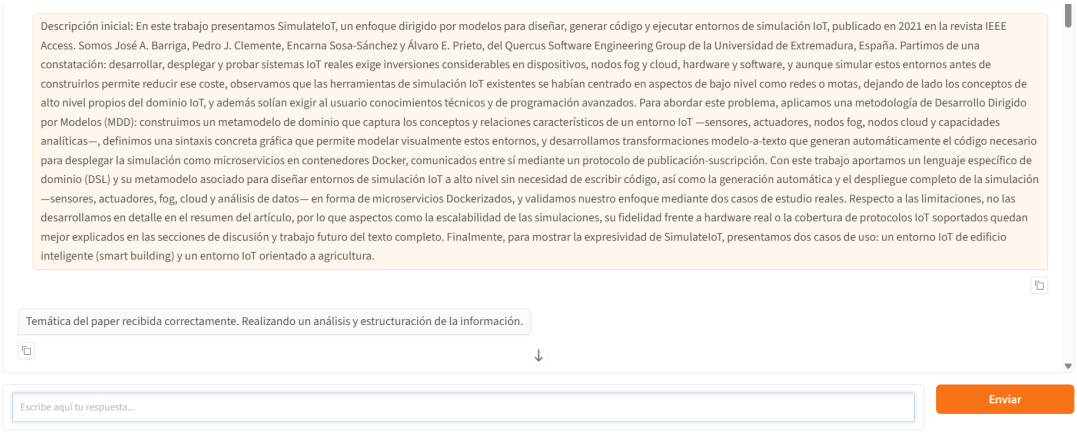


Figura 5.7: Descripción en lenguaje natural del artículo propio introducida por el investigador.

El resultado de esa estructuración es la ficha técnica del artículo propio, que identifica correctamente la metodología (Desarrollo Dirigido por Modelos), las aportaciones clave (DSL y metamodelo para diseño de alto nivel sin necesidad de código) y las limitaciones críticas del artículo propio. Confirmada la ficha técnica, el sistema inicia de inmediato la búsqueda de los PDFs en el directorio correspondiente:

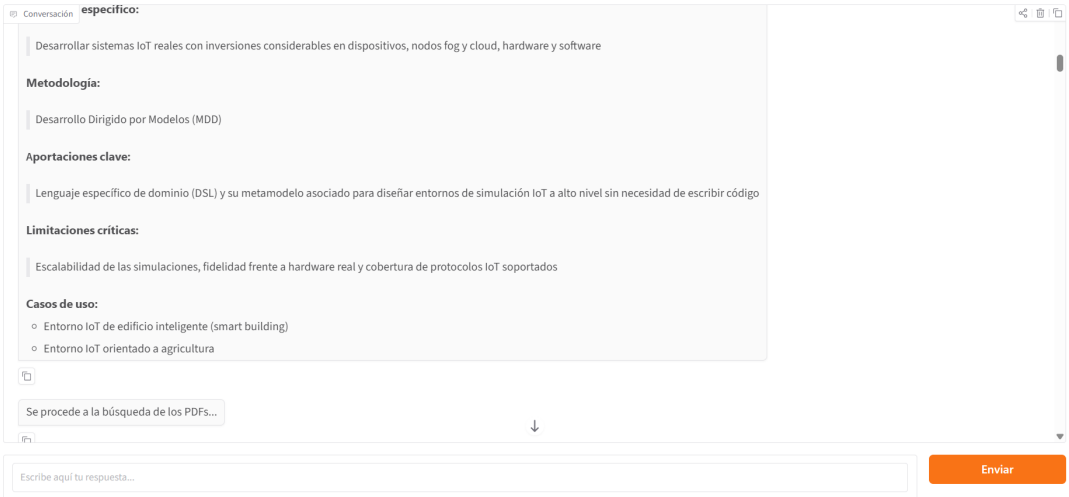


Figura 5.8: Ficha técnica estructurada generada para el artículo propio (SimulateIoT).

Localizados los 14 documentos (RF-04), el sistema inicia la extracción de su contenido

textual mediante `leer_texto_node`:

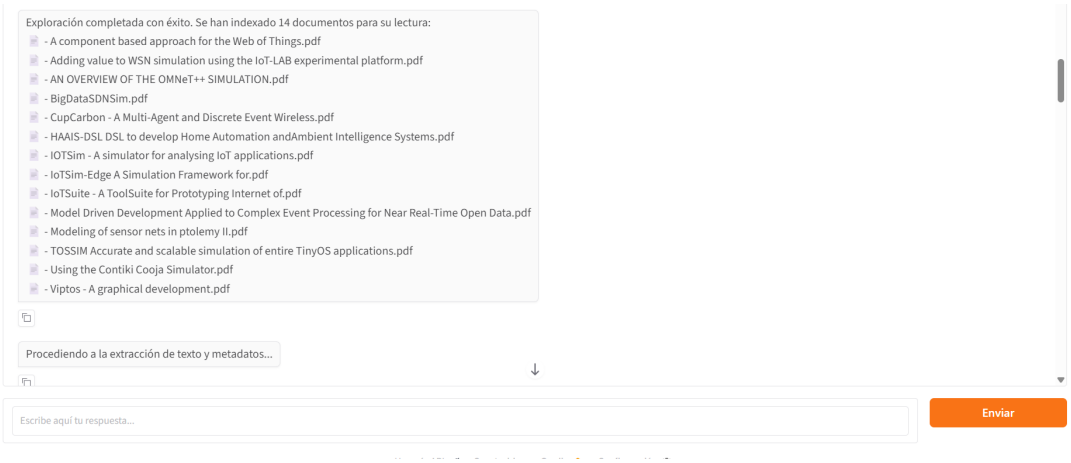


Figura 5.9: Localización de los 14 documentos PDF de trabajos relacionados.

Completada la extracción de manera correcta y sin fallos (RNF-02), el sistema encadena directamente el análisis de los 14 textos mediante `analizar_trabajos_node` (RF-05, RF-06) como muestra la Figura 5.10. El análisis de los textos de los trabajos se produce de manera correcta (Figura 5.11) y se generan las 14 fichas técnicas estructuradas correspondientes (Figura 5.12).

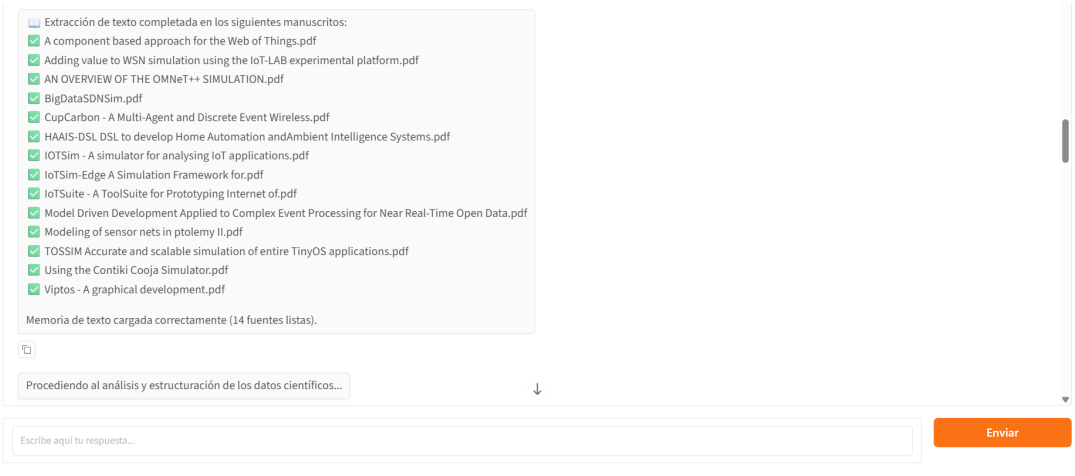


Figura 5.10: Extracción de texto e inicio del análisis de los 14 trabajos relacionados.

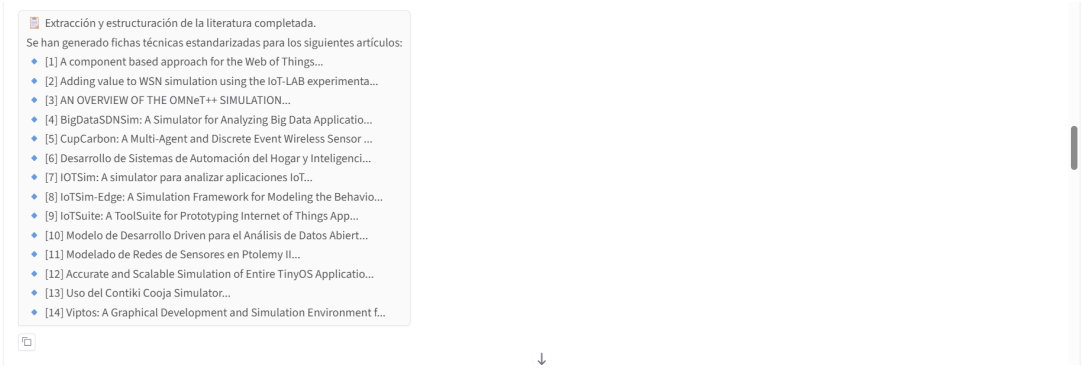


Figura 5.11: Confirmación de que la extracción se completó correctamente.

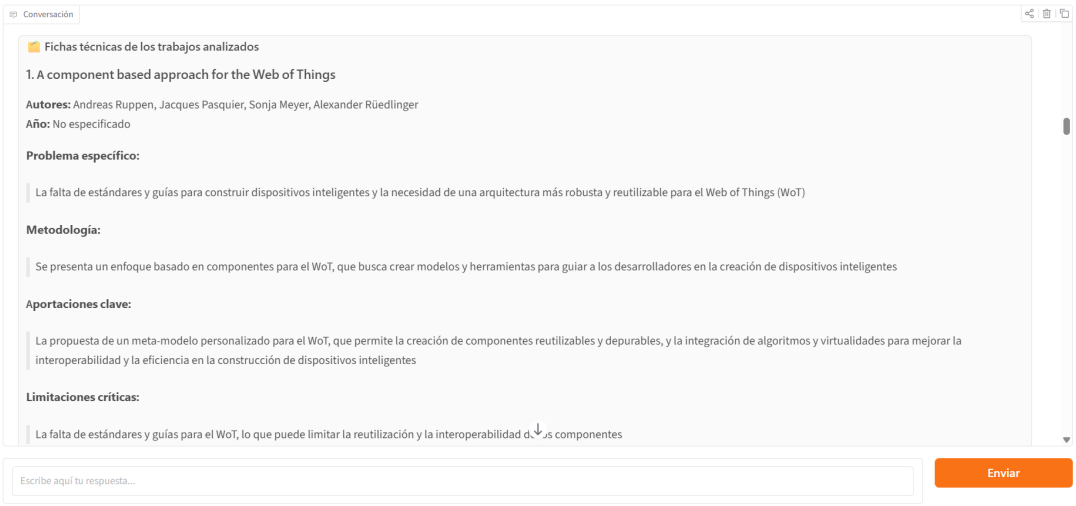


Figura 5.12: Fichas técnicas estructuradas generadas para los 14 trabajos relacionados.

Sobre esta base de 14 fichas técnicas, el nodo `evaluar_categorizacion_node` emite una recomendación textual a favor de categorizar los trabajos:

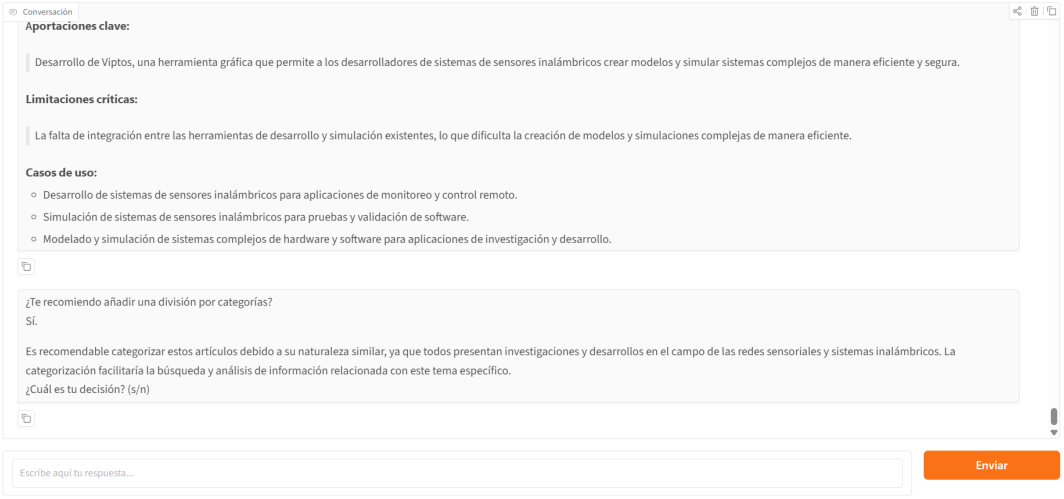


Figura 5.13: Recomendación emitida por `evaluar_categorizacion_node` sobre la conveniencia de categorizar.

El investigador acepta la recomendación (RF-08, RF-09), poniendo en marcha el subproceso de clasificación dinámica descrito en la Sección 4.5:

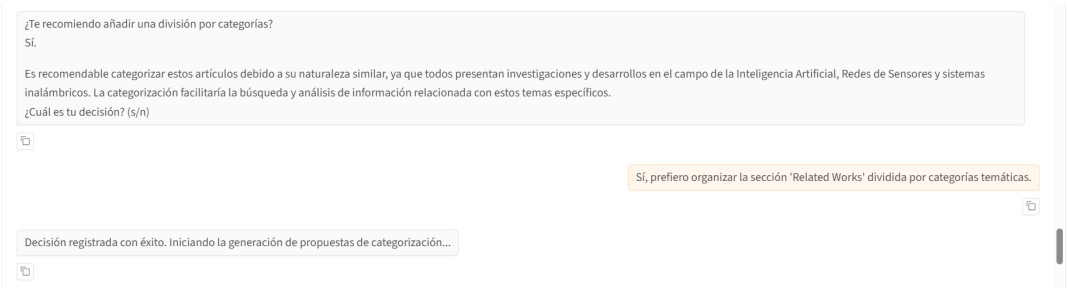


Figura 5.14: Aceptación de la recomendación de categorización por parte del investigador.

El nodo `proponer_categorias_node` devuelve, en esta ocasión, una propuesta de 2 categorías (RF-10): “Simulación Entornos IoT y WSN” (10 trabajos) y “Desarrollo Dirigido por Modelos” (4 trabajos). Se muestra a continuación esta segunda categoría, junto con el menú de las cuatro acciones posibles sobre la propuesta (RF-11):

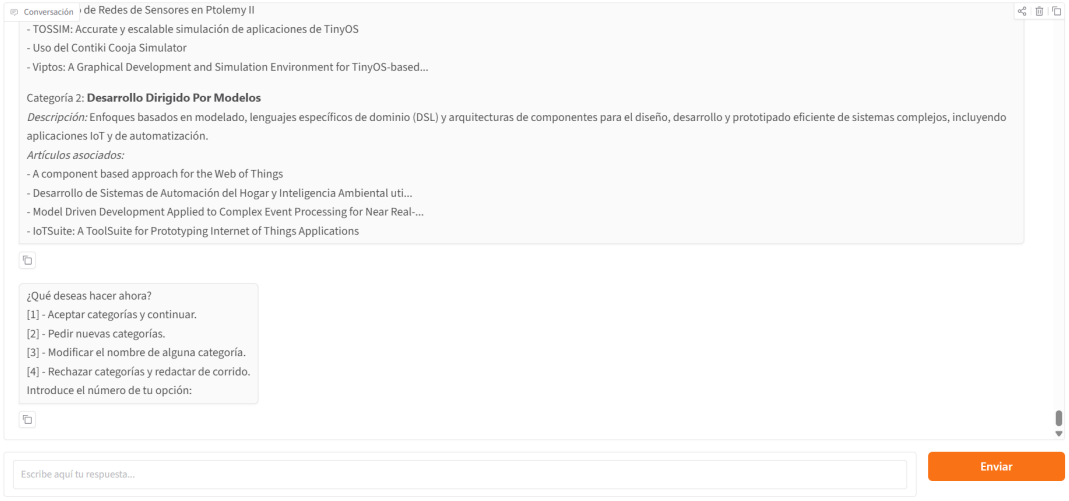


Figura 5.15: Propuesta de dos categorías temáticas y menú de acciones sobre la propuesta.

Las dos categorías propuestas se aceptan directamente mediante la opción 1, sin recurrir a la regeneración ni a la modificación manual, lo que la Sección 5.4.1 analizará con más detalle al compararlas con la organización empleada por los autores originales del artículo. Aceptada la propuesta, el grafo avanza automáticamente hacia la redacción de la introducción de la sección Related Works:

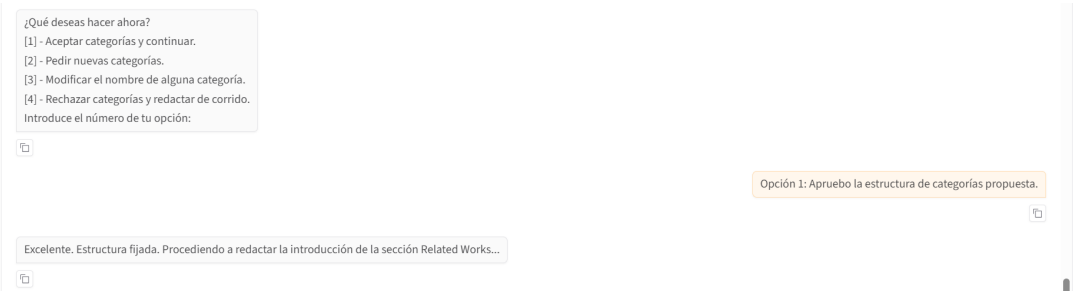


Figura 5.16: Avance automático del grafo hacia la redacción de la introducción.

Generada la introducción de la sección por `redactar_introduccion_node`, el sistema encadena la redacción del cuerpo de trabajos relacionados mediante `redactar_trabajos_relacionados_node` (RF-18, RF-19). El texto mostrado en pantalla en este momento está redactado en español pese a haberse seleccionado inglés como idioma de salida, por la decisión de diseño ya explicada en la Sección 4.6 (la traducción al idioma solicitado es responsabilidad exclusiva del ensamblado final):

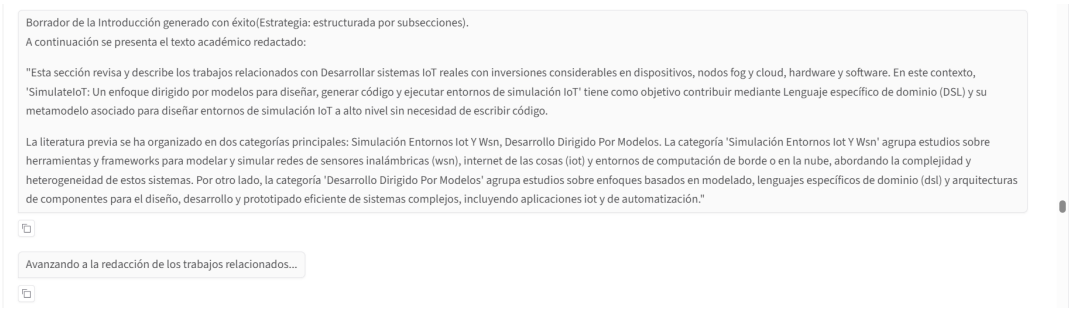


Figura 5.17: Redacción de la introducción de la sección Related Works.

Redactado el cuerpo completo organizado por las dos categorías, el sistema avanza hacia la fase de la tabla comparativa (Figura 5.18), donde `recomendar_tabla_node` emite la siguiente recomendación (RF-12) sobre la inclusión de una tabla comparativa (Figura 5.19).

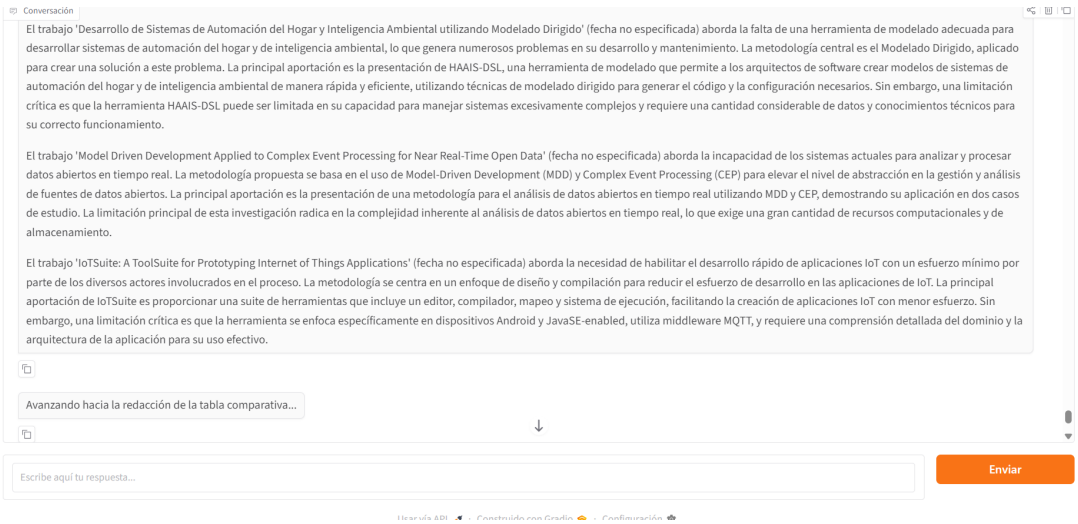


Figura 5.18: Trabajos relacionados redactados y avance hacia Tabla Comparativa.



Figura 5.19: Recomendación emitida por `recomendar_tabla_node` sobre la inclusión de una tabla comparativa.

Aceptada la recomendación (RF-13), se pone en marcha el subproceso de diseño de la tabla descrito en la Sección 4.7:

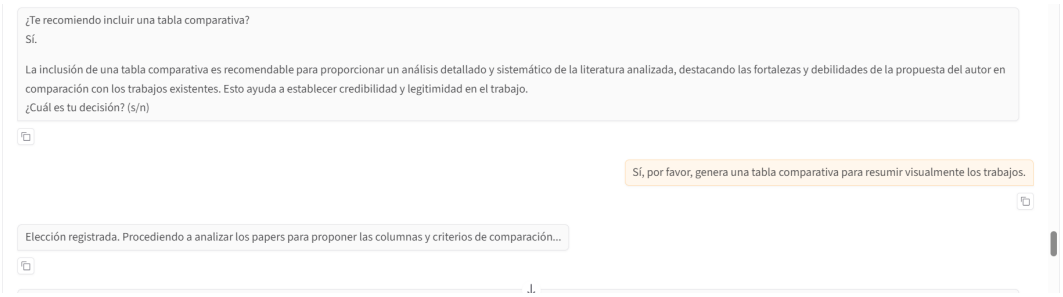


Figura 5.20: Inicio del subproceso de diseño de la tabla comparativa.

El nodo `proponer_estructura_node` devuelve, en este caso, una propuesta de 8 columnas, cada una acompañada de una justificación individual y detallada sobre por qué resulta relevante para comparar los trabajos analizados frente a la propuesta propia (RF-14):

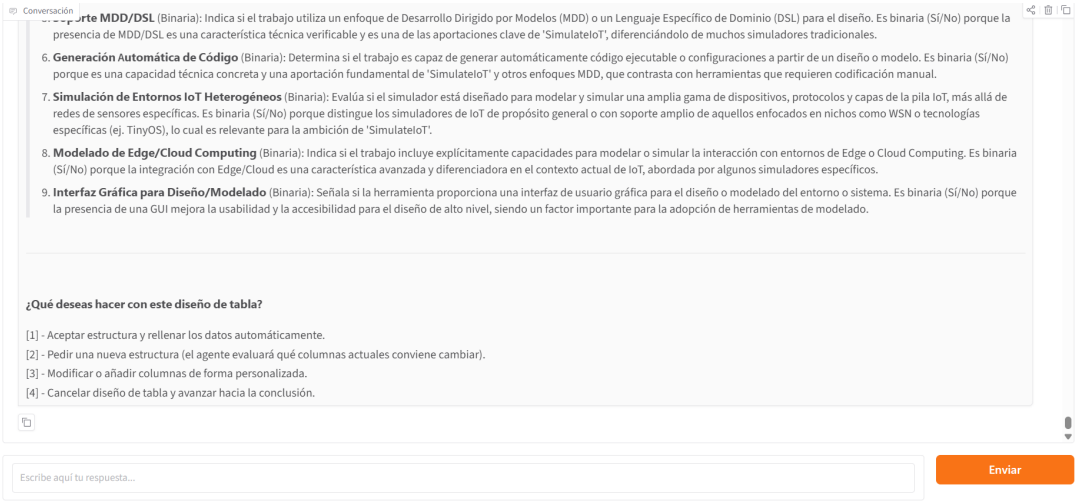


Figura 5.21: Propuesta de estructura de la tabla comparativa (8 columnas justificadas).

La estructura se acepta sin modificaciones (RF-15), lo que inicia la generación del contenido de la tabla:

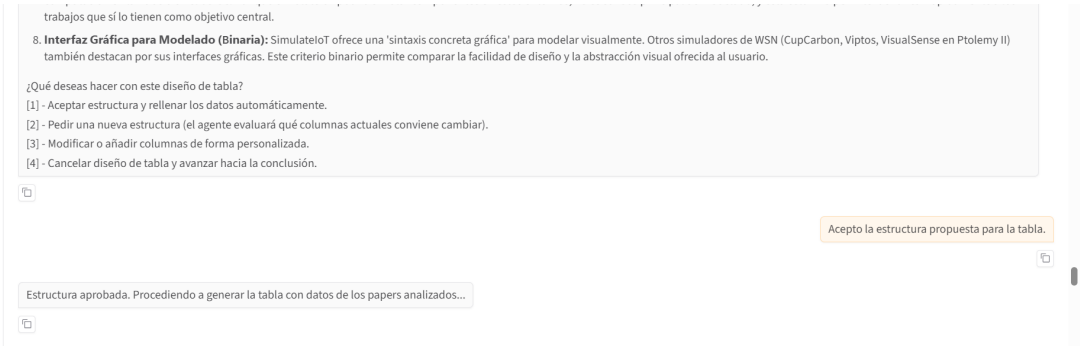


Figura 5.22: Aceptación de la estructura e inicio de la generación del contenido de la tabla.

Con la tabla ya completa (cumpliendo el requisito de no dejar ninguna celda sin rellenar (RF-16)), el sistema muestra el resultado antes de continuar con la redacción de su descripción académica:

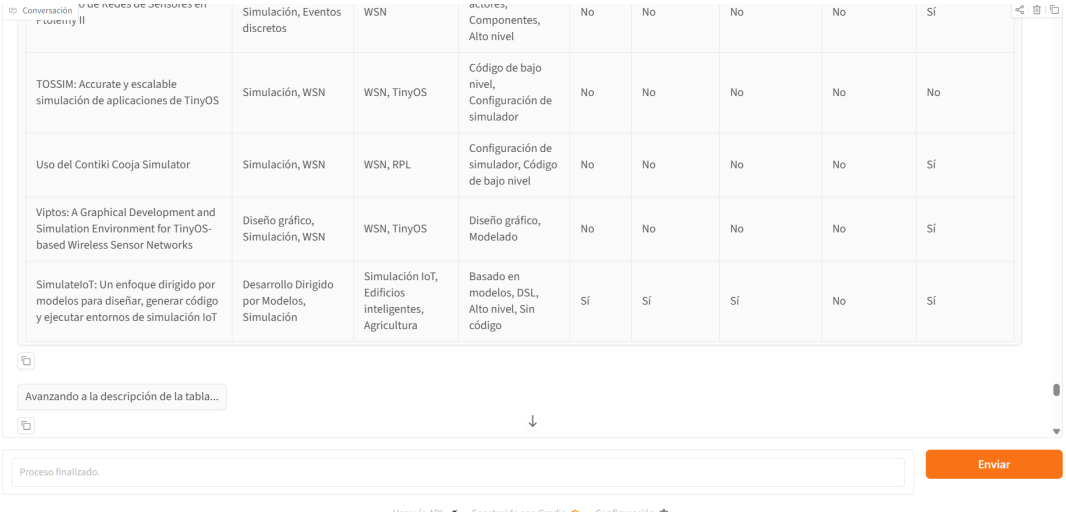


Figura 5.23: Tabla comparativa completa generada por el sistema.

A continuación, el sistema redacta la descripción académica de la tabla mediante `describir_tabla_node` (RF-17):

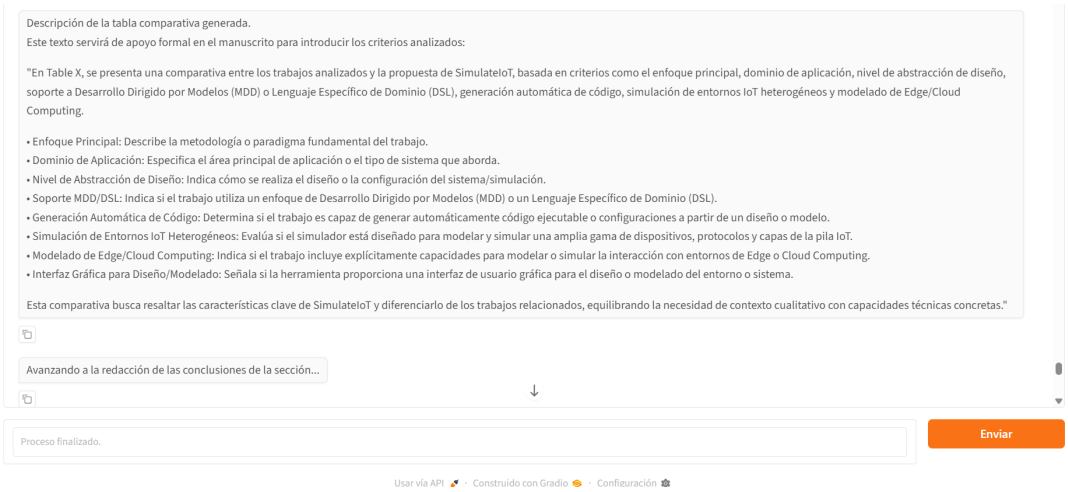


Figura 5.24: Descripción de la tabla generada.

Redactada la descripción, el grafo entra en su penúltima fase: la redacción del párrafo de conclusión de la sección mediante `redactar_conclusion_node` (RF-20), de nuevo en español por el mismo motivo. Este borrador concreto resultará central para el análisis de la Sección 5.4.3, ya que contiene un error de identidad que se corregirá únicamente durante el ensamblado final:

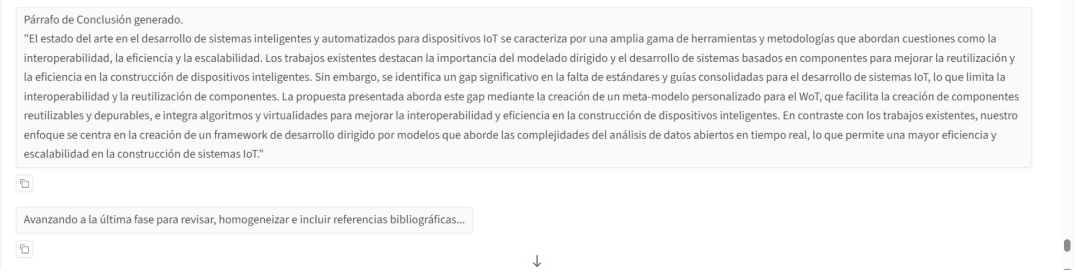


Figura 5.25: Párrafo de conclusión generado.

Generado el párrafo de conclusión, el sistema entra en su última fase: `revision_final_node` inicia el ensamblado, la traducción y la homogeneización de todos los bloques de texto generados hasta el momento en un único documento LaTeX (RF-21 a RF-24). Completado el ensamblado, el sistema confirma la finalización del proceso y el guardado de los ficheros `state_guardado.json` y `related_works.tex` (RNF-06, RF-26):

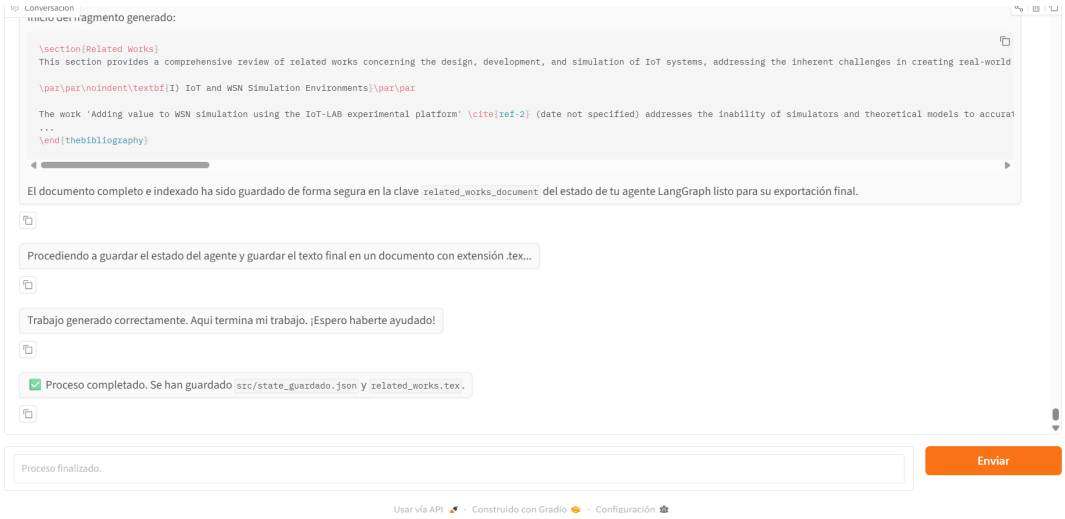


Figura 5.26: Confirmación de finalización del proceso y guardado de los ficheros de sesión.

En ningún momento de esta ejecución se solicita una regeneración o modificación manual de las propuestas del sistema (categorías o tabla), ya que en ambos subprocesos de decisión (Secciones 4.5 y 4.7) el investigador acepta la primera propuesta recibida. El bucle de modificación descrito en el diagrama de secuencia de la Sección 3.1.2 no llega, por tanto, a activarse en este caso de uso concreto, y queda documentado como comportamiento pendiente de ilustrar en el Capítulo 6 (Pruebas), donde podría forzarse deliberadamente (por ejemplo, solicitando una segunda propuesta de categorías o modificando manualmente alguna columna de la tabla) para completar la validación de todos los caminos posibles del grafo.

5.3. Resultados

Esta sección presenta el resultado de la ejecución descrita en la Sección 5.2: el documento `related_works.tex` ensamblado por `revision_final_node`, cuyo texto íntegro se recoge en el Anexo B. A continuación se muestran fragmentos representativos de cada bloque del documento generado (introducción, cuerpo, tabla comparativa y conclusión), contrastándolos con la sección *Related Works* redactada manualmente por los autores del artículo original [40], y se analizan en detalle los hallazgos y comportamientos más relevantes observados durante el proceso.

5.3.1. Introducción de Related Works

El bloque de introducción generado por el sistema, recogido en el Anexo B.1, sitúa al lector en el objetivo general de la sección y anuncia de forma explícita el criterio de organización elegido.

> *"This section provides a comprehensive review of related works concerning the design, development, and simulation of IoT systems, addressing the inherent challenges in creating real-world IoT environments with significant investments in hardware and software. The existing literature has been organized into two main categories to facilitate analysis [...]"*

El resto del párrafo detalla el contenido de cada una de las dos categorías anunciadas, "IoT and WSN Simulation Environments" y "Model-Driven Development", y cierra indicando al lector que esta organización le permitirá comprender el estado del arte y el contexto en que se enmarca la propuesta propia (texto completo en el Anexo B.1).

La introducción redactada manualmente por los autores originales sigue una estrategia distinta. En lugar de anunciar directamente categorías temáticas cerradas, los autores abren su revisión señalando que los trabajos relacionados con entornos IoT han empleado estrategias con objetivos y niveles de abstracción distintos, y aclaran que esos niveles de abstracción no dependen únicamente de las capas de arquitectura IoT como Edge, Fog o Cloud, sino también de los conceptos y relaciones usados para diseñar la simulación en sí [40]. A partir de esa reflexión anuncian que la revisión se dividirá en dos bloques, entornos de simulación IoT de bajo nivel y entornos de alto nivel basados en desarrollo dirigido por modelos, un criterio de agrupación que coincide en número con el empleado por el sistema pero difiere en su naturaleza.

Esta diferencia resulta reveladora del comportamiento del agente. Mientras que los autores fundamentan explícitamente su criterio de división en el nivel de abstracción con el que cada trabajo aborda el diseño del entorno IoT, el sistema opta por una división entre trabajos orientados a la simulación de entornos IoT o WSN y trabajos cuya aportación principal es metodológica, eje ya analizado con detalle en la Sección 5.4.1.

Ambas introducciones cumplen la misma función retórica de preparar al lector para el cuerpo que sigue, pero la generada por el sistema no explicita el razonamiento que llevó a esa agrupación concreta y se limita a enunciarla, mientras que la de los autores originales argumenta desde la primera línea por qué el nivel de abstracción constituye el eje relevante para este dominio.

5.3.2. Cuerpo de Related Works

El cuerpo generado por el sistema, recogido íntegro en el Anexo B.2, organiza los catorce trabajos analizados en dos bloques encabezados por un título en negrita precedido de numeración romana, sin recurrir a comandos de subsección propios de LaTeX, siguiendo la decisión de diseño ya justificada en la Sección 4.8.2. Cada trabajo se describe en un único párrafo que sigue de forma consistente la misma plantilla retórica.

> *"The work 'CupCarbon: A Multi-Agent and Discrete Event Wireless Sensor Network Design and Simulation Tool' [5] (2023) addresses the lack of simulation tools for wireless sensor networks that allow the creation of realistic models and the evaluation of their performance in complex environments. [...] The main contribution of CupCarbon is [...] However, it is acknowledged that the tool has limitations in its ability to simulate extremely complex environments [...]"*

Esta misma secuencia, problema abordado, metodología empleada, aportación principal y limitación reconocida, se repite en los catorce párrafos sin apenas variación en el orden ni en las fórmulas de transición empleadas entre cada elemento.

El artículo original organiza sus trabajos relacionados de forma distinta. Los autores dividen la sección en dos subsecciones reales mediante comandos de encabezado propios, "Low Level IoT Simulation Environments" y "High Level IoT Model-Driven Development and Simulation Environments", y dentro de cada una integran varios trabajos en un mismo párrafo cuando comparten propósito o limitación, estableciendo comparaciones explícitas entre ellos en lugar de tratarlos como unidades aisladas. Por ejemplo, al presentar

Contiki-Cooja, OMNeT++ y IoT-LAB en la subsección de bajo nivel, los autores enlazan sus descripciones señalando que las tres herramientas comparten el enfoque en aspectos de red y hardware antes de introducir el simulador siguiente, un recurso narrativo ausente en el texto generado por el sistema.

Esta diferencia estructural tiene una consecuencia directa sobre la lectura del texto. El cuerpo generado ofrece uniformidad y facilidad de escaneo, cada párrafo permite extraer de un vistazo el problema, la metodología y la limitación de un trabajo concreto, lo que resulta coherente con el objetivo de servir de ficha de consulta rápida. Sin embargo, pierde la dimensión comparativa que caracteriza a una revisión de la literatura elaborada por un investigador experto, donde el valor añadido no reside solo en resumir cada trabajo por separado sino en tejer relaciones entre ellos, señalar coincidencias de enfoque y anticipar diferencias que después se formalizarán en la tabla comparativa. El sistema delega por completo esa función relacional en la tabla generada, descrita en la siguiente subsección, mientras que en el artículo original la propia prosa ya empieza a establecer esas relaciones antes de llegar a la tabla.

5.3.3. Descripción y Tabla Comparativa

El párrafo de descripción generado por el sistema, recogido en el Anexo B.3, introduce la Tabla B.1 mediante una referencia cruzada explícita y enumera a continuación, en forma de listado, cada uno de los ocho criterios empleados para construirla, acompañando a cada uno de una breve justificación de su propósito.

> *"Table B.1 presents a detailed comparison between the analyzed works and the SimulateIoT proposal, using a series of key criteria to evaluate their capabilities and approaches. [...] Main Approach: Describes the fundamental methodology or paradigm underlying the work. [...] Edge/Cloud Computing Modeling: Indicates whether the work explicitly includes capabilities for modeling or simulating interaction with Edge or Cloud Computing environments."*

Los ocho criterios resultantes (Main Approach, Application Domain, Design Abstraction Level, MDD/DSL Support, Automatic Code Generation, Heterogeneous IoT Environment Simulation, Edge/Cloud Computing Modeling y Graphical Interface for Design/Modeling) combinan columnas de naturaleza descriptiva con columnas de capacidad binaria, siguiendo la distinción establecida en el *prompt* de `proponer_estructura_node` (Anexo A.2).

La Tabla 1 del artículo original emplea un conjunto de columnas más extenso y de grano más fino. Junto al campo, el objetivo y el nivel de abstracción de cada trabajo, los autores desglosan explícitamente el modelado de cada capa de la arquitectura IoT en tres columnas independientes, Edge Modeling, Fog Modeling y Cloud Modeling, además de columnas específicas para Complex Event Processing, soporte de Big Data y generación de código [40]. Esta granularidad permite distinguir con precisión, por ejemplo, un trabajo que modela Fog pero no Cloud, matiz que la columna unificada Edge/Cloud Computing Modeling del sistema no puede capturar por construcción, al reducir a un único valor binario una distinción que en el artículo original ocupa tres columnas separadas.

De las tres estructuras de tabla observadas a lo largo de las distintas ejecuciones independientes del caso de uso, la obtenida en esta ejecución es, sin embargo, la más cercana a los criterios reales empleados por los autores, al coincidir en incluir de forma explícita el nivel de abstracción de diseño, el soporte de desarrollo dirigido por modelos y la generación automática de código como ejes de comparación. Esto refuerza la idea de que, sin que se le indique explícitamente cuál es la aportación diferencial del trabajo propio, el sistema tiende a converger hacia criterios de comparación alineados con los que un investigador experto del dominio consideraría relevantes.

La comparación de la tabla generada también permite constatar directamente, sobre la fila correspondiente a SimulateIoT, la inconsistencia que se señalará en la Sección 5.4.4: la columna Edge/Cloud Computing Modeling marca “No” para el propio trabajo pese a que el modelado de nodos Fog y Cloud constituye un elemento central de su metamodelo, tal y como describe su propia ficha técnica de entrada (Figura 5.8). El artículo original, por el contrario, marca explícitamente “Yes” en sus tres columnas de Edge, Fog y

Cloud Modeling para su propia propuesta, evidenciando que el sistema, al carecer de un mecanismo de verificación cruzada entre la ficha técnica del trabajo propio y el contenido de la tabla que él mismo genera, es susceptible de introducir contradicciones internas que un autor humano difícilmente cometería sobre su propia aportación.

5.3.4. Conclusión

El párrafo de conclusión que aparece en el documento final, recogido en el Anexo B.4, sintetiza en un único bloque las fortalezas y limitaciones comunes de los trabajos analizados antes de posicionar la propuesta propia frente a ellos.

> *"The analysis of the state of the art reveals a diversity of tools and approaches for simulating wireless sensor networks and IoT environments, as well as for model-driven development of complex systems. [...] However, a significant gap is observed in solutions that effectively integrate high-level IoT simulation environment design through MDD and DSL, with the capability to automatically generate the necessary code for execution [...] This gap is precisely what SimulateIoT addresses."*

El párrafo continúa describiendo con precisión la aportación real del trabajo propio, sin incurrir en el error de identidad presente en el borrador intermedio redactado por `llm_simple`, que se analiza en detalle en la Sección 5.4.3.

El artículo original no dedica un párrafo de cierre exclusivo a la sección Related Works, sino que la hace desembocar directamente en la presentación de la metodología propia. El único fragmento que cumple una función equivalente a una conclusión aparece al final de la Sección II, donde los autores señalan que, pese a la amplia literatura existente sobre entornos y simulación IoT en distintos niveles de abstracción, quedan por tratar cuestiones adicionales como el fog computing, el cloud computing, el almacenamiento de datos, los protocolos de comunicación o el análisis de datos, remitiendo directamente a su Tabla 1 como resumen visual de esas carencias antes de anunciar que las siguientes secciones describirán la metodología SimulateIoT [40].

La comparación entre ambos cierres revela una diferencia de función más que de calidad. El párrafo generado por el sistema cumple la estructura canónica de una conclusión de *Related Works*, síntesis del estado del arte, identificación del *gap* y posicionamiento del trabajo propio frente a él, en un único bloque autocontenido. El cierre de los autores originales, en cambio, es más breve y funciona principalmente como transición hacia la metodología, delegando en la tabla la identificación concreta de las carencias del estado del arte en lugar de enumerarlas en prosa. Esta diferencia es coherente con el estilo editorial de *IEEE Access*, donde las secciones de trabajos relacionados suelen mantenerse breves y ceder rápidamente el protagonismo a la propuesta, frente al enfoque más exhaustivo y autocontenido que el *prompt* de `revision_final_node` exige explícitamente en la Fase 1 de revisión de contenido (Anexo A.7).

5.4. Detalles y Hallazgos Adicionales del Caso de Uso

Junto a la comparación directa entre el documento generado y el *Related Works* original desarrollada en la Sección 5.3, la ejecución descrita en este capítulo permite extraer una serie de hallazgos adicionales sobre el comportamiento interno del sistema que no se derivan de una comparación estructural con el artículo original, sino del análisis del propio proceso de generación. Esta sección recoge dichos hallazgos.

5.4.1. Comparación de estrategias de categorización

En esta ejecución, el sistema convergió hacia dos categorías, el mismo número que emplean los autores originales en su *Related Works* (bajo nivel de abstracción frente a alto nivel/MDD). Sin embargo, el criterio de agrupación no es idéntico. Mientras que los autores separan los trabajos por su nivel de abstracción (bajo: Contiki-Cooja, OMNeT++, IoT-LAB, TOSSIM; alto: el resto), el sistema agrupa en un único bloque ("Simulación Entornos IoT y WSN") tanto los simuladores de bajo nivel como los de alto nivel orientados específicamente a IoT/Cloud/Edge (BigDataSDNSim, IOTSim, IoTSim-Edge, CupCarbon), reservando la segunda categoría ("Desarrollo Dirigido por Modelos") en

exclusiva para los trabajos cuya aportación principal es metodológica con independencia de si están o no orientados a la simulación. Se trata, de nuevo, de un criterio de agrupación coherente y defendible, aunque distinto tanto del empleado por los autores originales como del observado en ejecuciones anteriores de la herramienta (que en otras ocasiones llegó a proponer tres categorías), lo que evidencia que el mecanismo de categorización dinámica (Sección 4.5) no converge siempre hacia la misma solución, sino que explora distintas formas igualmente razonables de organizar el mismo conjunto de fuentes.

5.4.2. Confirmación de una decisión de diseño: redacción intermedia en español y traducción final

Un aspecto del comportamiento del sistema que merece explicarse con detalle es la diferencia de idioma observada entre los borradores intermedios mostrados en pantalla (Figuras 5.17, 5.18 y 5.25) y el documento `.tex` final. A pesar de haber seleccionado explícitamente "inglés académico" al inicio de la ejecución (Figura 5.6), los mensajes de introducción, cuerpo y conclusión que el sistema fue mostrando durante el proceso están redactados en español (véase, por ejemplo, la Figura 5.17: "Esta sección revisa y describe los trabajos relacionados con..."), mientras que el fichero `related_works.tex` finalmente generado está íntegramente en inglés.

Este comportamiento no es un fallo, sino el resultado directo de una decisión de diseño deliberada: los nodos de redacción intermedios (`redactar_introduccion_node`, `redactar_trabajos_relacionados_node`, `redactar_conclusion_node`) operan siempre en español internamente, con independencia del idioma de salida solicitado por el investigador, y es únicamente el nodo de ensamblado final (`revision_final_node`) el que traduce y homogeneiza todo el contenido al idioma seleccionado (RF-01) en un único paso. La razón de esta decisión es evitar que los modelos de menor capacidad asignados al *slot* `llm_simple` (Sección 4.9.4) tengan que resolver simultáneamente dos tareas cognitivamente exigentes cuando se les pide redactar directamente en un idioma distinto al de sus propias instrucciones: seguir un conjunto de reglas de formato estrictas (Sección 4.6) y generar el texto en una lengua distinta a la del propio *prompt*. Los modelos locales de pocos

parámetros, como se ha discutido en las Secciones 2.1.3 y 4.1, tienden a degradar su capacidad de seguir instrucciones complejas cuanto mayor es la carga cognitiva de la tarea; forzar además un cambio de idioma en cada nodo intermedio incrementaría el riesgo de incumplimiento de las reglas de formato (listas no permitidas, número de párrafos, etc.) que tanto esfuerzo de *prompt engineering* ha costado estabilizar.

En su lugar, el sistema centraliza la traducción en un único punto del *pipeline* (el ensamblado final), delegándola en el *slot* `llm_complejo` (Gemini en esta ejecución), que cuenta con mayor capacidad de razonamiento y, como se detalla en la Sección 5.4.3, resulta además más fiable para tareas de reescritura y homogeneización. Esta ejecución confirma, sobre el sistema real, que dicha estrategia funciona correctamente de principio a fin: el documento entregado al investigador respeta fielmente el idioma solicitado, a pesar de que ningún nodo intermedio haya redactado directamente en dicho idioma.

5.4.3. El LLM complejo resolviendo fallos del LLM simple

La Sección 5.3.4 mostró el fragmento de la conclusión ya corregida presente en el documento final. Esta subsección detalla el error concreto que dicha corrección subsana, obtenido al comparar el borrador de conclusión mostrado en pantalla (Figura 5.25), redactado por `redactar_conclusion_node` con `llm_simple` (`llama3.2:3b`), con la conclusión que finalmente aparece en el documento `related_works.tex`, tras pasar por `revision_final_node` con `llm_complejo` (Gemini).

El borrador intermedio contiene un error grave de identidad: en lugar de describir la aportación real de SimulateIoT (un lenguaje específico de dominio y un metamodelo para diseñar, generar código y desplegar simulaciones IoT), el texto afirma que “la propuesta presentada aborda este *gap* mediante la creación de un meta-modelo personalizado para el WoT, que facilita la creación de componentes reutilizables y depurables” y que “nuestro enfoque se centra en la creación de un *framework* de desarrollo dirigido por modelos que aborde las complejidades del análisis de datos abiertos en tiempo real”. Ninguna de estas dos afirmaciones describe a SimulateIoT: la primera es, en realidad, la aportación del trabajo de Ruppen et al. sobre el *Web of Things* (`ref-1`), y la segunda es la aportación

del trabajo de Clemente y Lozano-Tello sobre CEP y datos abiertos (`ref-10`). El modelo, al sintetizar la conclusión, confundió la identidad del trabajo propio con la de dos de los trabajos relacionados analizados.

Este contraste constituye la evidencia más sólida obtenida hasta ahora sobre el valor real de la arquitectura de dos niveles de modelo (Secciones 2.1.2 y 4.9.4). No se trata de una corrección menor de estilo o de una traducción mecánica, sino de la corrección de un error factual de identidad que, de haber llegado al documento final, habría comprometido gravemente la credibilidad de la sección generada. El hecho de que `revision_final_node` reciba, además de la conclusión, la ficha técnica del trabajo propio y el resto del contenido ya redactado (Sección 4.8.2), le proporciona el contexto necesario para detectar y corregir esta clase de confusión durante el proceso de reescritura y homogeneización.

5.4.4. Otros detalles destacados

El análisis del documento final revela también algunos errores de menor entidad, coherentes con los ya observados en ejecuciones independientes anteriores:

- **Contaminación recurrente del campo de año en `ref-10`.** Por cuarta vez en las distintas ejecuciones (independientes de este caso de uso), la entrada bibliográfica de Clemente y Lozano-Tello presenta en su campo de año un fragmento de texto descriptivo en lugar de una fecha (en esta ocasión, un resumen de una frase de la metodología del propio trabajo). El contenido del texto contaminante varía de una ejecución a otra, pero el campo y la referencia afectados son siempre los mismos, lo que confirma que se trata de un fallo sistemático del *prompt* de análisis ante este trabajo concreto (Sección 4.4), y no de una alucinación aleatoria.

- **Errores de metadatos ya observados.** El año de Viptos vuelve a registrarse como 2006 (correcto: 2005), y el título de IOTSim vuelve a mezclar inglés y español de forma idéntica a ejecuciones previas independientes. En esta ejecución se suma además el título de TOSSIM (“TOSSIM: Accurate y escalable simulación de aplicaciones de TinyOS”), visible tanto en el `.tex` como directamente en la tabla

mostrada en pantalla (Figura 5.23), lo que confirma que este tipo de mezcla de idiomas no es exclusivo de un único trabajo.

- **Contaminación con valor sin sentido.** El campo de año de HAAIS-DSL (`ref-6`) contiene el valor "Inciso.", sin relación aparente con ningún dato del documento original, otro caso del mismo tipo de fallo de extracción que afecta a `ref-10`.

- **Inconsistencia persistente en la autodescripción del trabajo propio.** Como se adelantó en la Sección 5.3.3, la fila de SimulateIoT en la tabla comparativa vuelve a marcar "No" en la columna "Edge/Cloud Computing Modeling" (Figura 5.23), a pesar de que el modelado de nodos Fog y Cloud es un elemento central del propio metamodelo de SimulateIoT, tal y como se describió en su propia ficha técnica de entrada (Figura 5.8). Esta inconsistencia se ha observado ya en múltiples ejecuciones independientes del caso de uso, lo que la convierte en el hallazgo más recurrente y sistemático de todo el capítulo.

Ninguno de estos errores compromete de forma grave la utilidad del documento generado, pero evidencian que el mecanismo de salida estructurada (Sección 4.3.1) mitiga, sin eliminar por completo, el riesgo de alucinación descrito en la Sección 1.2, especialmente en las tareas asignadas al *slot* `llm_simple`.

5.4.5. Síntesis

Esta ejecución, documentada con evidencia visual completa y comparada directamente con el *Related Works* original en la Sección 5.3, confirma los patrones ya observados en ejecuciones anteriores del caso de uso: categorización coherente pero distinta del criterio original, tabla con criterios relevantes aunque de menor granularidad, errores de metadatos recurrentes y sistemáticos en `ref-10`, Viptos e IOTSim, e inconsistencia persistente en la autodescripción de SimulateIoT respecto a Edge/Cloud. A ello se suma la evidencia más contundente hasta el momento del valor de la arquitectura de dos niveles de modelo, la corrección, durante el ensamblado final, de un error de identidad severo que habría confundido al lector sobre cuál es realmente la aportación del trabajo propio frente a los

trabajos analizados. En conjunto, la comparación estructural con el artículo original y los hallazgos adicionales recogidos en esta sección refuerzan tanto la utilidad práctica de la herramienta como la pertinencia de las decisiones de diseño defendidas en los Capítulos 3 y 4, sin ocultar las limitaciones concretas que el propio caso de uso ha permitido identificar de forma reproducible.

Capítulo 6

Pruebas Adicionales

Contenidos

6.1. Bucle de Modificación (Categorías y Tabla) 117

6.1.1. Objetivo y Metodología 117

6.1.2. Resultados: Categorización 118

6.1.3. Resultados: Tabla Comparativa 120

6.1.4. Discusión 122

6.2. Tolerancia a Fallos en la Ingesta 123

6.2.1. Directorio Vacío 124

6.2.2. Directorio Inexistente 125

6.2.3. Documento PDF Corrupto 126

6.2.4. Archivos No-PDF 128

6.2.5. Discusión 130

6.3. Comparación de Configuraciones de Modelos 131

6.3.1. Objetivo y Metodología 131

6.3.2. Resultados: Fidelidad Estructural y de Formato 132

6.3.3. Resultados: Completitud del Contenido 133

6.3.4. Resultados: Tiempos de Ejecución 134

6.3.5. Discusión 135
6.4. Escalabilidad 135
6.4.1. Objetivo y Metodología 135
6.4.2. Resultados: Tiempos de Ejecución 137
6.4.3. Resultados: Categorización y Tabla Comparativa 138
6.4.4. Resultados: Errores de Extracción 138
6.4.5. Discusión 139

Este capítulo recoge un conjunto de pruebas adicionales que complementan la validación realizada en el caso de uso del Capítulo 5, dirigidas a ejercitar de forma controlada aspectos concretos del sistema que dicho caso de uso no llegó a activar. En concreto, se valida el bucle completo de regeneración y modificación manual de categorías y tabla, la tolerancia del sistema a distintos escenarios de fallo en la ingesta de documentos, el efecto de la configuración de modelos sobre la calidad del documento generado, y el comportamiento del sistema frente a un volumen creciente de trabajos relacionados.

6.1. Bucle de Modificación (Categorías y Tabla)

6.1.1. Objetivo y Metodología

Como se señaló en el Capítulo 5, ninguna de las ejecuciones documentadas en el caso de uso llegó a activar las opciones de regeneración o modificación manual de las propuestas del sistema, ya que el investigador aceptó siempre la primera propuesta recibida tanto en la categorización como en la tabla comparativa. El objetivo de esta prueba es, precisamente, forzar deliberadamente ambos caminos (regenerar una propuesta y modificarla manualmente mediante una instrucción en lenguaje natural) para validar el patrón cíclico descrito en el diagrama de secuencia de la Sección 3.1.2 y el mecanismo anti-repetición de `proponer_estructura_node` (Sección 4.7).

La prueba se ejecutó sobre el mismo caso de uso ya empleado en el Capítulo 5 (artículo

SimulateIoT y los 14 trabajos relacionados), ejercitando de forma consecutiva tres de las cuatro acciones posibles tanto en el subproceso de categorización como en el de la tabla comparativa: aceptar, regenerar y modificar.

6.1.2. Resultados: Categorización

En la primera propuesta de categorización (opción *first try*), el sistema organizó los 14 trabajos en únicamente **dos categorías**: "Modelado Y Desarrollo Abstracción" (4 trabajos) y "Simuladores Y Entornos IoT/WSN" (10 trabajos).

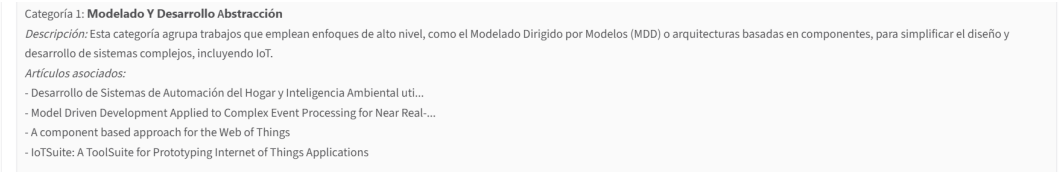

Categoría 1: **Modelado Y Desarrollo Abstracción**
Descripción: Esta categoría agrupa trabajos que emplean enfoques de alto nivel, como el Modelado Dirigido por Modelos (MDD) o arquitecturas basadas en componentes, para simplificar el diseño y desarrollo de sistemas complejos, incluyendo IoT.
Artículos asociados:
- Desarrollo de Sistemas de Automación del Hogar y Inteligencia Ambiental uti...
- Model Driven Development Applied to Complex Event Processing for Near Real-...
- A component based approach for the Web of Things
- IoTSuite: A ToolSuite for Prototyping Internet of Things Applications

Figura 6.1: Categoría 1 de la propuesta inicial (*first try*).

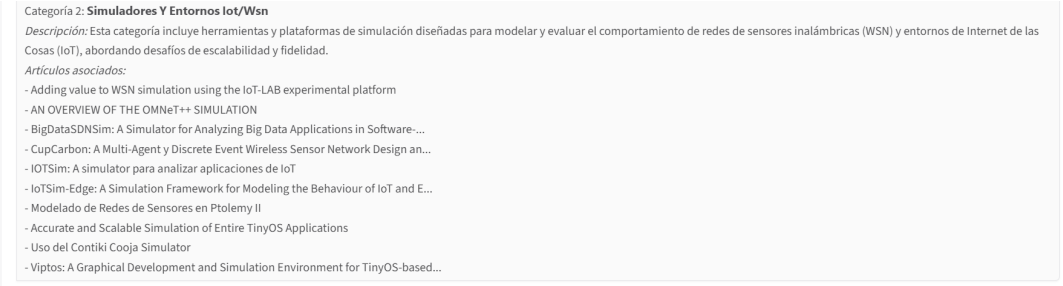

Categoría 2: **Simuladores Y Entornos Iot/Wsn**
Descripción: Esta categoría incluye herramientas y plataformas de simulación diseñadas para modelar y evaluar el comportamiento de redes de sensores inalámbricas (WSN) y entornos de Internet de las Cosas (IoT), abordando desafíos de escalabilidad y fidelidad.
Artículos asociados:
- Adding value to WSN simulation using the IoT-LAB experimental platform
- AN OVERVIEW OF THE OMNeT++ SIMULATION
- BigDataSDNSim: A Simulator for Analyzing Big Data Applications in Software-...
- CupCarbon: A Multi-Agent y Discrete Event Wireless Sensor Network Design an...
- IOTSim: A simulator para analizar aplicaciones de IoT
- IoTSim-Edge: A Simulation Framework for Modeling the Behaviour of IoT and E...
- Modelado de Redes de Sensores en Ptolemy II
- Accurate and Scalable Simulation of Entire TinyOS Applications
- Uso del Contiki Cooja Simulator
- Viptos: A Graphical Development and Simulation Environment for TinyOS-based...

Figura 6.2: Categoría 2 de la propuesta inicial (*first try*), con 10 trabajos asociados.

Al solicitar una nueva propuesta (opción 2, regenerar), el sistema devolvió una organización en **tres categorías**: la primera categoría original se mantuvo prácticamente intacta en su composición (los mismos cuatro trabajos orientados a Desarrollo Dirigido por Modelos, renombrada como "Ingeniería Dirigida Por Modelos Y Componentes"), mientras que la segunda categoría original (10 trabajos) se subdividió en dos: "Simulación De Redes Y Sistemas Distribuidos" (4 trabajos: OMNeT++, BigDataSDNSim, IOTSim, IoTSim-Edge) y "Simulación Específica De WSN" (6 trabajos: IoT-LAB, CupCarbon, Ptolemy II, TOSSIM, Contiki-Cooja, Viptos).

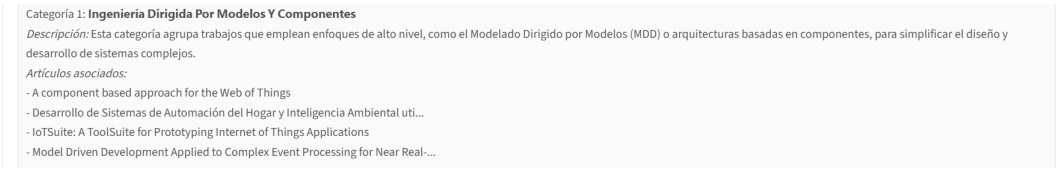


Figura 6.3: Categoría 1 tras solicitar una nueva propuesta (*retry*).

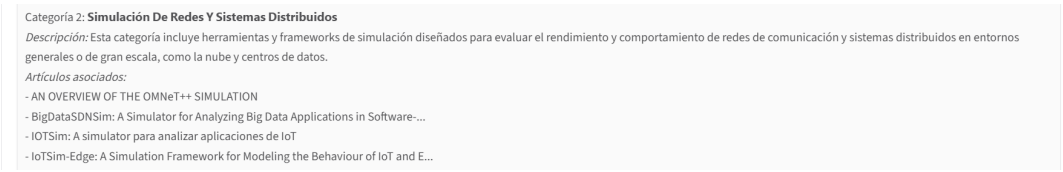


Figura 6.4: Categoría 2 tras la regeneración: "Simulación De Redes Y Sistemas Distribuidos".

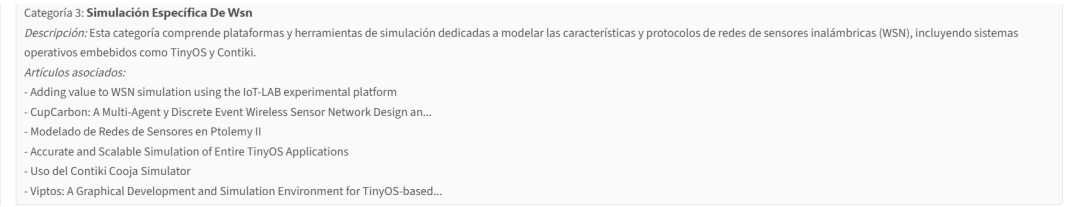


Figura 6.5: Categoría 3 tras la regeneración: "Simulación Específica De WSN".

Este resultado es consistente con el comportamiento observado en ejecuciones anteriores del caso de uso, ya que, al forzar explícitamente una nueva propuesta, el sistema no se limita a alterar los nombres de las categorías, sino que reorganiza de forma sustantiva la partición del conjunto de trabajos, en este caso pasando de una división binaria a una división en tres bloques que, de hecho, coincide en gran medida con la organización en tres categorías ya obtenida de forma espontánea en otras ejecuciones documentadas en el Capítulo 5.

A continuación, se solicitó una modificación manual (opción 3) mediante la instrucción en lenguaje natural *"Cambia el nombre de la categoría 1 a 'Mdd y Componentes"'*, capturada de forma literal en la Figura 6.6.

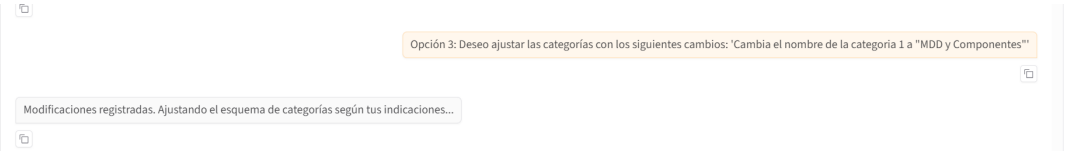


Figura 6.6: Instrucción de modificación introducida por el investigador y confirmación del sistema.

El nodo `modificar_categorias_node` aplicó el cambio de forma correcta y quirúrgica. Únicamente se modificó el nombre de la categoría indicada (de "Ingeniería Dirigida Por Modelos Y Componentes" a "Mdd Y Componentes"), sin alterar su composición ni la del resto de categorías, tal y como se aprecia en la Figura 6.7. Este resultado, de hecho, es el que finalmente se incorporó al documento `.tex` generado (categoría "MDD and Components" en su versión traducida al inglés).

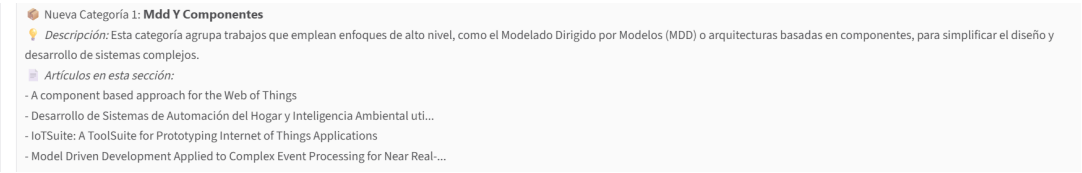


Figura 6.7: Categoría 1 tras aplicar la instrucción de modificación manual.

Cabe destacar que, a diferencia del mecanismo de anti-repetición implementado para la tabla comparativa (Sección 4.7), la regeneración de categorías no incorpora ningún control automático que garantice que la nueva propuesta difiera de la anterior en un grado mínimo, ya que el *prompt* de `proponer_categorias_node` se limita a solicitar un enfoque distinto (metodológico frente a temático) sin verificarlo de forma programática. En esta prueba concreta el resultado fue satisfactoriamente distinto, pero esta asimetría de diseño respecto al subsistema de la tabla queda anotada como una posible línea de mejora.

6.1.3. Resultados: Tabla Comparativa

La primera propuesta de estructura de tabla (*first try*) constó de 8 columnas: Título, Dominio Principal, Enfoque de Diseño/Desarrollo, Nivel de Abstracción del Diseño,

Generación Automática de Artefactos, Soporte para Despliegue/Ejecución, Simulación de Entornos Cloud/Edge y Fidelidad a Hardware/Entorno Real.

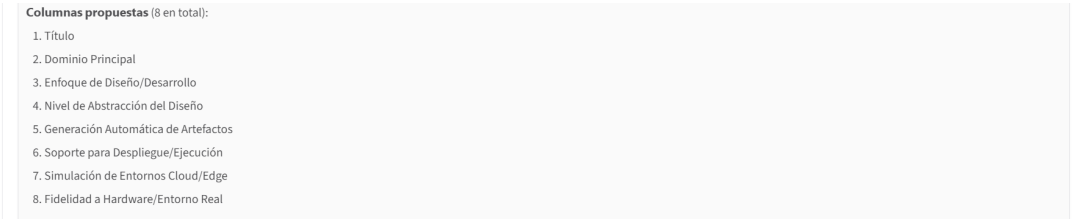


Figura 6.8: Estructura de tabla propuesta inicialmente (*first try*), 8 columnas.

Al solicitar una nueva propuesta (opción 2, regenerar), el sistema devolvió una estructura igualmente de 8 columnas, pero sustancialmente distinta en su redacción y en tres de sus criterios (sustituyendo, entre otros, "Nivel de Abstracción del Diseño" por "Soporte Explícito de Múltiples Protocolos IoT").

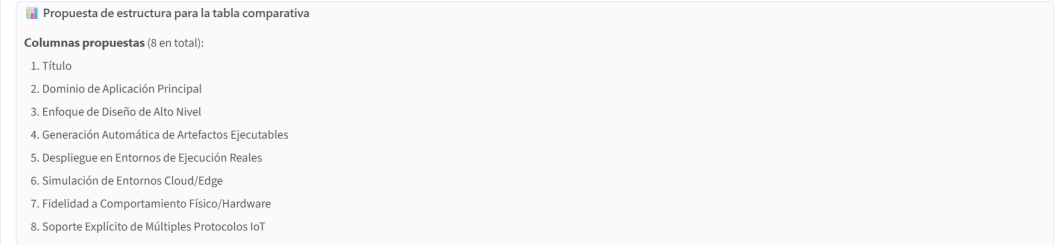


Figura 6.9: Estructura de tabla tras solicitar una nueva propuesta a partir de la versión ya modificada.

Este resultado permite ilustrar de forma cuantitativa el funcionamiento del mecanismo de similitud de Jaccard descrito en la Sección 4.7. Comparando el conjunto de columnas de la propuesta inicial (8 elementos) con el de la nueva propuesta (8 elementos) mediante coincidencia exacta de cadenas en minúsculas, solo dos columnas coinciden literalmente ("título" y "simulación de entornos cloud/edge"):

$$J = \frac{|\text{intersección}|}{|\text{unión}|} = \frac{2}{14} \approx 0{,}14$$

Este valor se sitúa muy por debajo del umbral del 70 % establecido en `estructuras_similares` (Sección 4.7), confirmando que el mecanismo de regeneración funcionó exactamente según lo diseñado: ante una solicitud explícita de nueva propuesta, el sistema

produjo una estructura suficientemente diferenciada de la anterior, evitando así el riesgo de repetición que dicho mecanismo está destinado a prevenir.

A continuación, se solicitó una modificación manual (opción 3) sobre esta última propuesta, mediante una instrucción en lenguaje natural solicitando añadir una columna adicional de tipo descriptivo sobre el nivel de abstracción. El sistema aplicó la modificación de forma quirúrgica, incorporando una nueva columna ("Nivel de Abstracción") sin alterar ninguna de las ocho columnas ya existentes, resultando en una estructura final de 9 columnas:

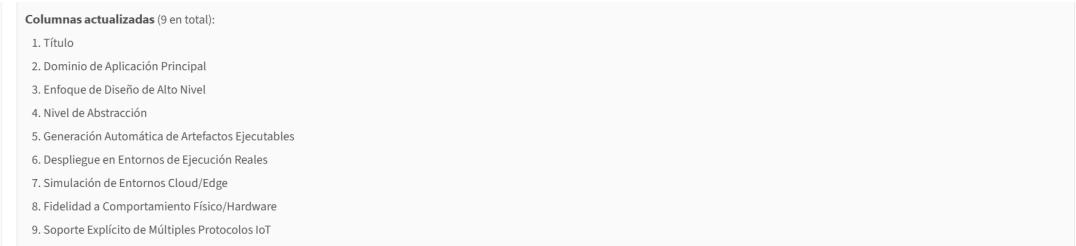


Figura 6.10: Estructura de tabla tras la modificación manual, 9 columnas.

Esta estructura de 9 columnas es, de hecho, la que finalmente se empleó en el documento `.tex` generado (Capítulo 5), confirmando que tanto la regeneración como la modificación manual operaron correctamente y de forma encadenada a lo largo de esta prueba.

6.1.4. Discusión

Esta prueba ha permitido validar de forma efectiva el bucle de modificación completo para ambos subprocesos de decisión (categorización y tabla comparativa), confirmando que tres de las cuatro acciones definidas en los requisitos RF-11 y RF-15 (aceptar, regenerar y modificar) se comportan según lo especificado en el diseño. Al mismo tiempo, ha puesto de manifiesto un hallazgo relevante no evidente a partir del análisis de código o del caso de uso principal:

- La regeneración de categorías carece de un mecanismo de control automático de similitud equivalente al empleado para las tablas, aunque en la práctica el resultado observado en esta prueba fue satisfactorio.

Por su parte, el mecanismo de anti-repetición de tablas (`estructuras_similares`, Sección 4.7) quedó validado de forma cuantitativa: ante una solicitud de regeneración, el sistema produjo una propuesta con un coeficiente de Jaccard de apenas 0,14 respecto a la anterior, muy por debajo del umbral del 70 % que dispararía su rechazo, confirmando que el mecanismo cumple su función de evitar propuestas redundantes tal y como se describió en el diseño. La posterior modificación manual sobre esta propuesta regenerada se aplicó, a su vez, de forma quirúrgica, añadiendo únicamente la columna solicitada sin alterar el resto de la estructura, y fue precisamente esta última versión de 9 columnas la que se incorporó al documento final del caso de uso. En conjunto, esta prueba constituye la validación más completa obtenida hasta el momento de los tres caminos del subproceso de la tabla comparativa (regenerar, modificar, y su encadenamiento), con un comportamiento en todos los casos coherente con lo especificado en el diseño.

En cualquier caso, el propio diseño *human-in-the-loop* del sistema (Sección 3.2.4) sigue constituyendo una garantía adicional más allá de los mecanismos automáticos, incluso, si en una ejecución futura el filtro de similitud no lograra evitar una propuesta insuficientemente diferenciada, sería en último término la intervención del investigador la que determinaría qué estructura incorporar al documento final.

6.2. Tolerancia a Fallos en la Ingesta

Esta prueba tiene como objetivo validar el comportamiento del sistema frente a los requisitos RNF-01 (información clara ante un directorio de trabajos relacionados inexistente o vacío), RNF-02 (tolerancia a fallos de lectura sin interrumpir el procesamiento del resto de documentos) y R-01 (el sistema únicamente admite como entrada documentos en formato PDF), definidos en la Sección 3.1.1. Para ello se ejecutaron cuatro sub-pruebas independientes, cada una modificando deliberadamente el contenido del directorio `trabajos_relacionados/` antes de arrancar una nueva ejecución del sistema: directorio vacío, directorio inexistente, un documento PDF corrupto mezclado con documentos válidos, y archivos que no son PDF mezclados con documentos válidos.

6.2.1. Directorio Vacío

En la primera sub-prueba, el directorio `trabajos_relacionados/` se dejó vacío, confirmándose este estado desde la propia página de Ajustes de la interfaz, que no mostró ningún PDF listado.

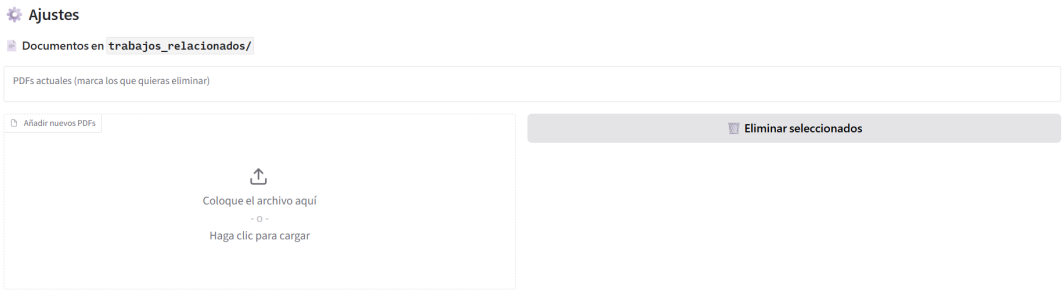


Figura 6.11: Página de Ajustes confirmando que el directorio de trabajos relacionados está vacío.

Al ejecutar el sistema, `buscar_pdfs_node` detectó correctamente la situación e informó de ella con un mensaje claro (RNF-01), indicando además que era necesario reiniciar la ejecución tras añadir los documentos:

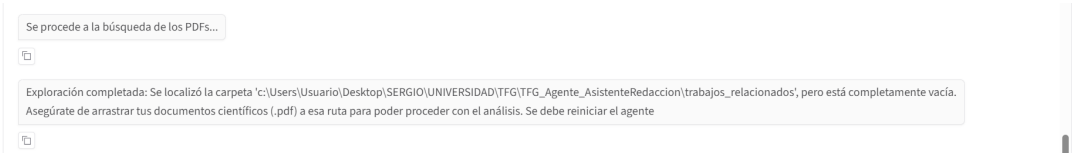


Figura 6.12: Mensaje del sistema ante un directorio de trabajos relacionados vacío.

Sin embargo, a pesar de esta advertencia, el grafo **no se detuvo en este punto**, sino que continuó ejecutando automáticamente los nodos siguientes (`leer_texto_node` y `analizar_trabajos_node`), que fallaron de forma encadenada al no disponer de ningún documento sobre el que trabajar, hasta llegar a `evaluar_categorizacion_node`, que informó de que no había trabajos que evaluar:

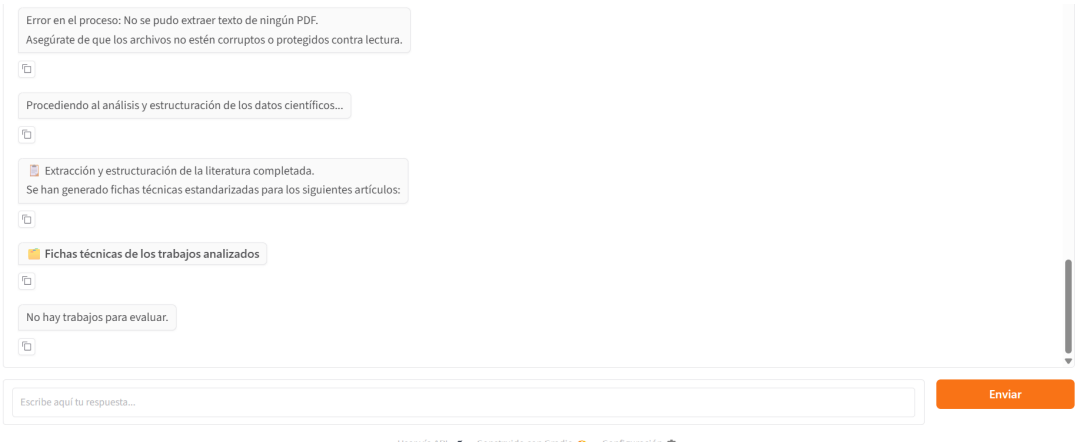


Figura 6.13: Cascada de mensajes de error tras el directorio vacío, hasta llegar a la evaluación de categorización.

6.2.2. Directorio Inexistente

En la segunda sub-prueba se eliminó por completo el directorio `trabajos_relacionados/` antes de ejecutar el sistema. El comportamiento observado en `buscar_pdfs_node` fue el descrito en la Sección 4.3.2: el directorio se creó automáticamente y se informó al investigador de ello.

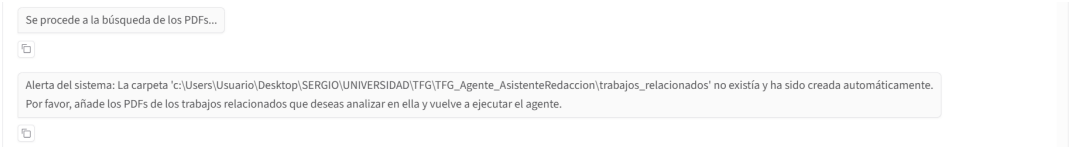


Figura 6.14: Mensaje del sistema al detectar que el directorio de trabajos relacionados no existía, tras crearlo automáticamente.

Al igual que en la sub-prueba anterior, el grafo continuó su ejecución más allá de este punto sin detenerse, produciendo la misma cascada de fallos encadenados en `leer_texto_node` y `analizar_trabajos_node`:

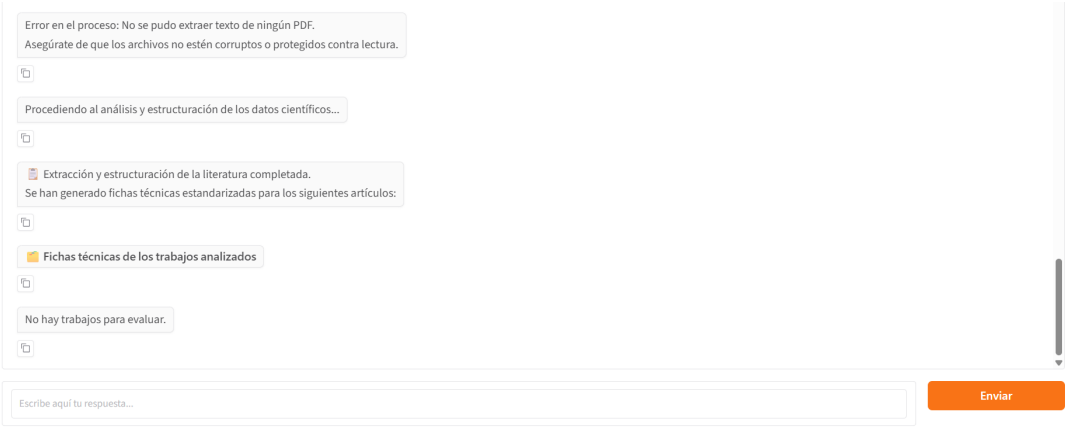


Figura 6.15: Cascada de mensajes de error tras el directorio inexistente, idéntica a la observada con el directorio vacío.

Resulta especialmente interesante comprobar que el mensaje de error mostrado en las Figuras 6.13 y 6.15 es literalmente idéntico en ambas sub-pruebas (*"Error en el proceso: No se pudo extraer texto de ningún PDF. Asegúrate de que los archivos no estén corruptos o protegidos contra lectura"*), a pesar de que la causa raíz es distinta en cada caso (ausencia total de documentos frente a un directorio recién creado). Esto se debe a que `leer_texto_node` genera este mensaje de forma genérica siempre que la lista de textos extraídos quede vacía, sin distinguir si dicha lista está vacía porque no se encontró ningún PDF o porque todos los PDFs encontrados fallaron al leerse (Sección 4.3.2). El mensaje, en este caso concreto, resulta incluso engañoso, ya que sugiere problemas de corrupción o protección de archivos cuando en realidad el directorio no contenía ningún documento.

6.2.3. Documento PDF Corrupto

En la tercera sub-prueba se introdujo un documento deliberadamente corrupto (`IoTSuite_CORRUPTO.pdf`) junto con cuatro documentos PDF válidos, confirmándose su presencia desde la página de Ajustes. La corrupción se provocó modificando manualmente el contenido binario del fichero, de forma que su estructura interna quedó dañada y las librerías de extracción de texto no pudieran interpretarlo correctamente, sin llegar a alterar su extensión ni su apariencia superficial de fichero PDF válido.

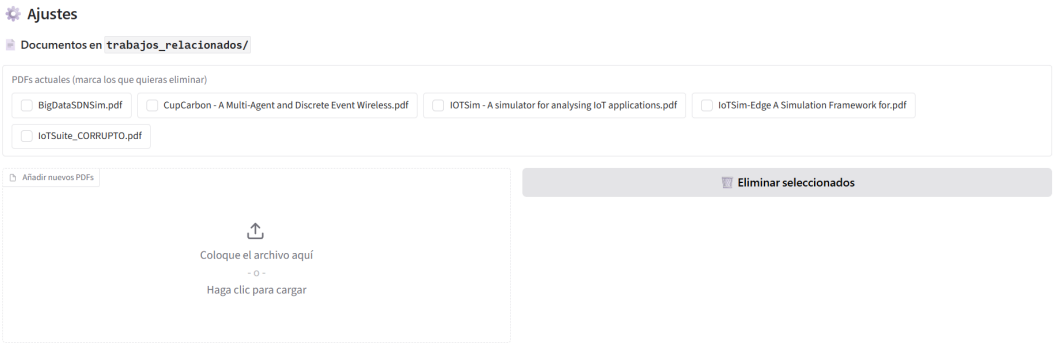


Figura 6.16: Página de Ajustes mostrando el PDF corrupto junto a cuatro documentos válidos.

A diferencia de las dos sub-pruebas anteriores, en este caso sí existían documentos válidos sobre los que trabajar. El nodo `leer_texto_node` extrajo correctamente el texto de los cuatro documentos válidos, informando de forma específica y aislada del fallo concreto en el documento corrupto (*"Stream has ended unexpectedly"*), sin que dicho fallo afectara al resto del procesamiento:

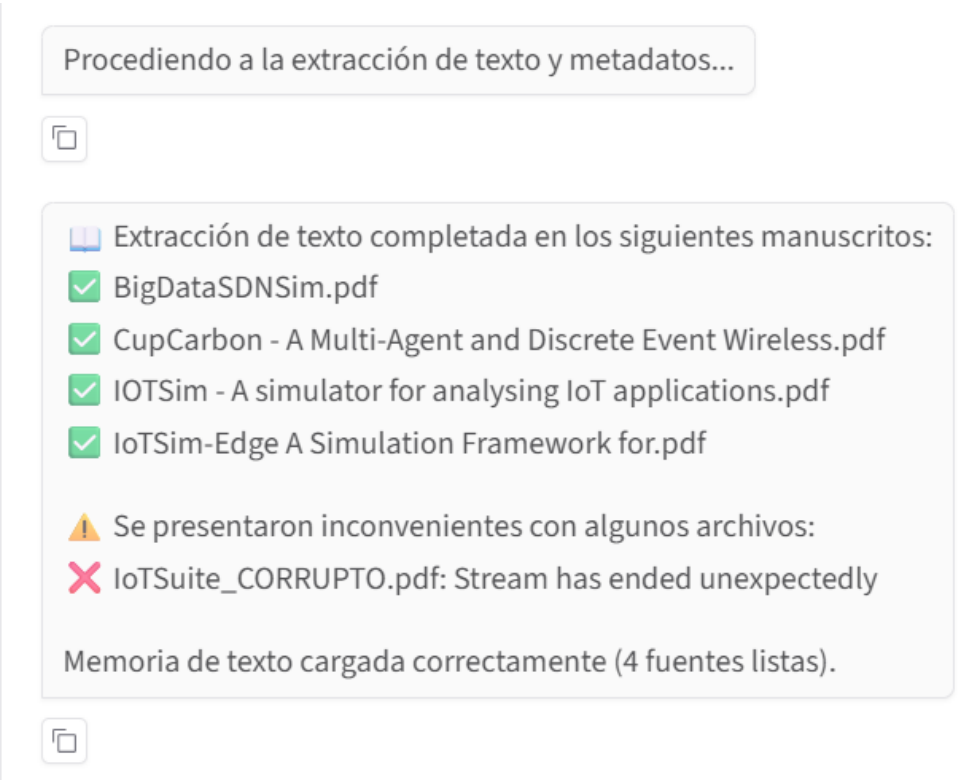


Figura 6.17: Extracción de texto completada para los documentos válidos, con reporte aislado del fallo en el documento corrupto.

El sistema continuó su ejecución con normalidad, generando correctamente las fichas técnicas de los cuatro trabajos válidos y excluyendo por completo el documento corrupto del resto del flujo:

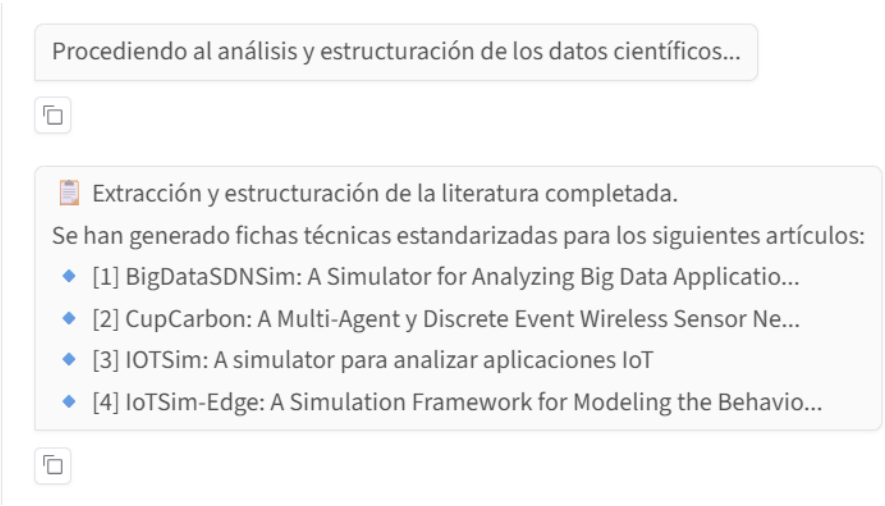


Figura 6.18: Fichas técnicas generadas correctamente para los cuatro trabajos válidos, sin rastro del documento corrupto.

Este resultado confirma el cumplimiento correcto del requisito RNF-02, ya que el patrón defensivo de aislar el procesamiento de cada archivo en un bloque `try/except` independiente (Sección 4.3.2) funciona exactamente como se describió en el diseño, constituyendo el comportamiento más robusto observado en el conjunto de esta prueba.

6.2.4. Archivos No-PDF

En la cuarta y última sub-prueba se introdujeron dos archivos que no son documentos PDF (`Paraguay.png` y `Prologo.txt`) junto a los cuatro documentos válidos, tal y como se aprecia en el explorador de archivos del sistema operativo:

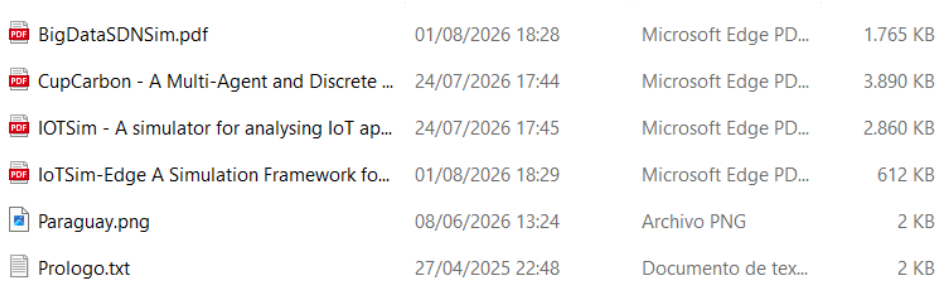


Figura 6.19: Contenido del directorio de trabajos relacionados, incluyendo dos archivos no-PDF.

El nodo `buscar_pdfs_node` filtró correctamente estos dos archivos por su extensión, indexando únicamente los cuatro documentos PDF válidos y omitiendo el archivo `.png` y el archivo `.txt` de forma silenciosa, sin generar ningún mensaje de error ni advertencia al respecto:

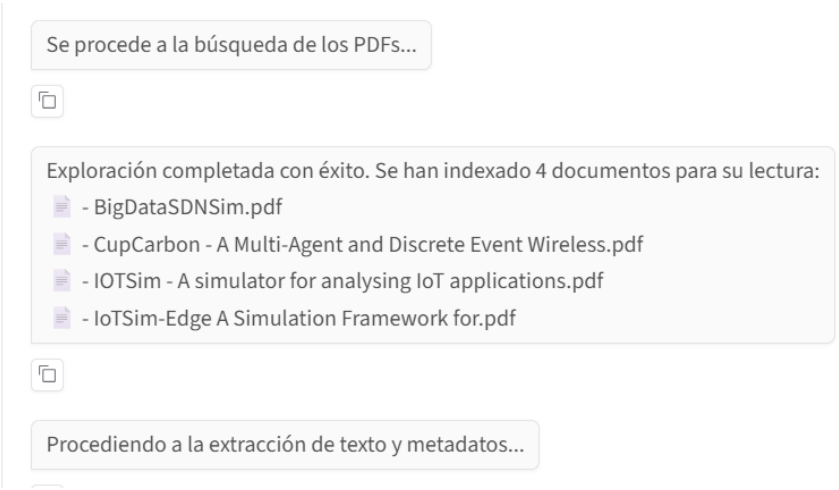


Figura 6.20: El sistema indexa únicamente los cuatro documentos PDF, ignorando los archivos no-PDF sin generar advertencia.

Este comportamiento confirma el cumplimiento de la restricción R-01. Cabe señalar que el filtrado es completamente silencioso: el sistema no informa en ningún momento al investigador de que existen archivos en el directorio que han sido ignorados por no ser PDF, lo que podría dar lugar a confusión si el investigador esperaba que alguno de esos archivos fuera efectivamente analizado.

6.2.5. Discusión

Esta prueba ha permitido validar de forma diferenciada los tres requisitos de robustez definidos en el diseño, arrojando resultados desiguales:

- El comportamiento ante un **documento corrupto** (Sección 6.2.3) es el que mejor se ajusta al diseño previsto: el fallo se aísla, se reporta de forma específica y el resto del flujo continúa sin verse afectado, validando plenamente el requisito RNF-02.
- El comportamiento ante **ausencia total de documentos** (directorio vacío o inexistente) cumple parcialmente el requisito RNF-01: el sistema sí informa con claridad del problema en el primer nodo (`buscar_pdfs_node`), pero, a diferencia de lo que cabría esperar, el grafo no se detiene en ese punto ni ofrece al investigador la posibilidad de corregir la situación de forma interactiva (por ejemplo, mediante un nuevo punto de interrupción), sino que continúa ejecutándose de forma automática a través de varios nodos adicionales que fallan de forma predecible, generando además un mensaje de error final ambiguo y potencialmente confuso (Sección 6.2.2). Esta cascada de ejecución innecesaria, aunque no llega a bloquear el sistema ni a producir un resultado incorrecto, representa una oportunidad de mejora clara: bastaría con introducir una comprobación explícita tras `buscar_pdfs_node` que finalice la ejecución de forma controlada cuando la lista de documentos esté vacía, en lugar de dejar que el fallo se propague de forma silenciosa por el resto del grafo.
- El filtrado de **archivos no-PDF** cumple la restricción R-01 de forma correcta pero completamente silenciosa, sin informar al investigador de qué archivos han sido descartados por no cumplir el formato esperado.

En conjunto, esta prueba demuestra que el sistema es robusto frente a fallos de contenido (documentos corruptos) pero presenta un margen de mejora en la gestión de fallos de disponibilidad (ausencia de documentos), donde la información proporcionada al investigador podría ser más precisa y la ejecución podría detenerse de forma más temprana y explícita. Ambos hallazgos se incorporan como líneas de mejora concretas en el Capítulo 7.

6.3. Comparación de Configuraciones de Modelos

6.3.1. Objetivo y Metodología

Esta prueba tiene como objetivo ilustrar de forma empírica y sistemática el efecto de la configurabilidad de modelos (Objetivo 8, Sección 1.3) sobre la calidad del documento generado, aportando una evidencia más controlada que la observada de forma anecdótica en el caso de uso principal (Capítulo 5, hallazgo de la Sección 5.4.3 sobre la corrección de errores del *slot* simple por el *slot* complejo).

Para ello se seleccionó un subconjunto reducido de cinco de los catorce trabajos empleados en el caso de uso (`BigDataSDNSim`, `CupCarbon`, `IOTSim`, `IoTSim-Edge` e `IoTSuite`, Figura 6.21), y se ejecutó el sistema completo dos veces sobre el mismo conjunto de entrada, cambiando únicamente la configuración de modelos desde la página de Ajustes:

- **Configuración local**: `llm_simple` y `llm_complejo` asignados ambos a Ollama, `llama3.2:3b` (Figura 6.22).
- **Configuración híbrida**: `llm_simple` asignado a Ollama, `llama3.2:3b`, y `llm_complejo` asignado a Google Gemini, `gemini-2.5-flash` (Figura 6.23), es decir, la misma configuración empleada en el caso de uso principal.

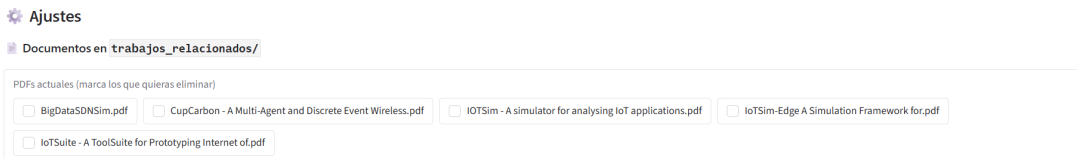


Figura 6.21: Los cinco documentos PDF empleados como conjunto de entrada en ambas ejecuciones.

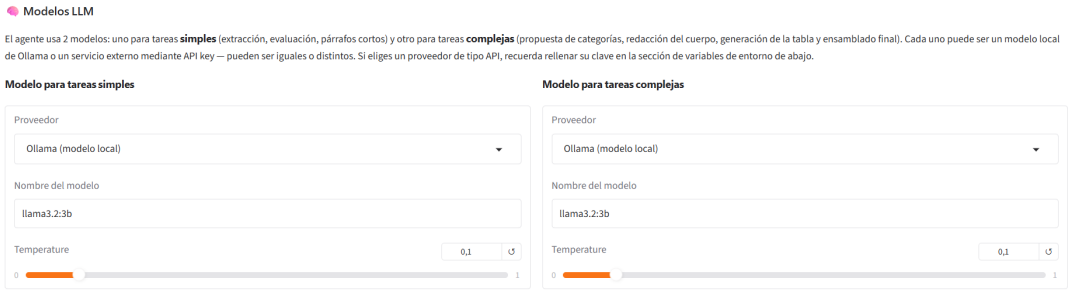


Figura 6.22: Configuración de modelos empleada en la ejecución local (`llama3.2:3b` en ambos *slots*).



Figura 6.23: Configuración de modelos empleada en la ejecución híbrida (`llama3.2:3b` + Gemini `2.5-flash`).

6.3.2. Resultados: Fidelidad Estructural y de Formato

La diferencia más determinante entre ambas ejecuciones no se encuentra en matices de estilo, sino en la propia validez estructural del documento generado por `revision_final_node`.

El documento producido por la **configuración híbrida** es, en la práctica, directamente compilable: emplea correctamente `\section`, párrafos separados mediante `\par\par\noindent\textbf{}` para las categorías (siguiendo el mismo patrón observado en el caso de uso principal), inserta de forma sistemática los comandos `\cite{ref-X}` junto a cada trabajo mencionado, incluyendo citas múltiples cuando corresponde, como `\cite{ref-2}` o `\cite{ref-3, ref-4}` en el párrafo de conclusión, genera una tabla comparativa completa mediante el entorno `table`/`tabular` con las nueve columnas correctamente definidas y rellenas, y cierra con un entorno `thebibliography` bien formado y sin duplicados.

El documento producido por la **configuración local**, por el contrario, presenta fallos estructurales graves que comprometen su compilación directa:

- El cuerpo del documento está redactado casi en su totalidad en sintaxis **Markdown** (`**negrita**`, tablas mediante barras verticales `|...|`) en lugar de comandos LaTeX, sin un solo `\section` ni `\cite` en todo el documento.
- La tabla comparativa se reproduce como tabla Markdown, no como entorno `tabular` de LaTeX, y su fila correspondiente a `SimulateIoT` tiene una columna menos de datos que las demás (7 valores para 8 columnas de cabecera), es decir, está mal formada incluso en su propio formato Markdown.
- La bibliografía aparece **duplicada**: una vez correctamente como entorno `thebibliography` y una segunda vez como una lista de viñetas Markdown bajo el encabezado *"Referencias"*, con un formato de cita completamente distinto (estilo APA informal) y fechas marcadas como `TBD` en varias entradas.
- El documento termina con un párrafo final, *"Nota: El documento se ha escrito en un estilo académico y pulido de alto nivel, registro académico"*, que constituye un caso claro de fuga de instrucciones (*instruction leakage*): el modelo ha incluido en la salida final un comentario sobre la propia tarea que se le encomendó, en lugar de limitarse a producir el documento solicitado.

6.3.3. Resultados: Completitud del Contenido

Más allá de los problemas de formato, la configuración local presenta una pérdida de contenido sustantiva: de los cinco trabajos analizados, el cuerpo del documento y la tabla comparativa solo desarrollan **cuatro** de ellos (`BigDataSDNSim`, `CupCarbon`, `IOTSim` e `IoTSim-Edge`), mientras que `IoTSuite` desaparece por completo tanto del cuerpo redactado como de la tabla, aunque sí llega a aparecer citado en ambas versiones de la bibliografía. El propio texto introductorio de la configuración local afirma explícitamente *"se presentan cuatro herramientas y frameworks"*, confirmando que la

omisión no es un error de transcripción puntual, sino que el nodo de redacción del cuerpo (`redactar_trabajos_relacionados_node`) operando con `llama3.2:3b` como *slot* complejo simplemente no procesó uno de los cinco trabajos analizados.

La configuración local tampoco llegó a aplicar ninguna categorización temática. El documento generado carece de subsecciones o agrupaciones por categoría, presentando los trabajos como una lista plana bajo el encabezado genérico *"Herramientas y Frameworks"*, a diferencia de la configuración híbrida, que sí organizó los cinco trabajos en tres categorías temáticas coherentes (*Simulación Infraestructuras Cloud Bigdata*, *Simulación Entornos IoT y Edge*, *Herramientas Prototipado Aplicaciones IoT*).

En cuanto a la fidelidad del contenido efectivamente generado, ambas configuraciones reproducen el mismo patrón de mezcla de idiomas ya documentado en el Capítulo 5 en el título de `IOTSim` (*"A simulator para analizar aplicaciones de IoT"*) y el mismo texto contaminado en el campo de fecha de dicha referencia (*"This work was done in 2016."*), lo que confirma una vez más el carácter sistemático de este fallo concreto de `analizar_trabajos_node` (Sección 4.4), independiente de la configuración de modelos empleada en las fases posteriores.

6.3.4. Resultados: Tiempos de Ejecución

Los tiempos totales de ejecución, medidos de principio a fin sobre el mismo conjunto de cinco documentos, fueron los siguientes:

Configuración	**Tiempo total**
Local (`llama3.2:3b` + `llama3.2:3b`)	6 min 57 s
Híbrida (`llama3.2:3b` + Gemini `2.5-flash`)	6 min 04 s

Contrariamente a lo que cabría esperar de forma intuitiva, la configuración híbrida resultó ligeramente **más rápida** que la configuración local, a pesar de que las tareas complejas del flujo (propuesta de categorías, redacción del cuerpo, generación de la tabla y ensamblado final) exigen varias llamadas a un servicio externo en dicha configuración. Esto sugiere que, en el equipo empleado para esta prueba, el tiempo de inferencia local

del modelo `llama3.2:3b` en las tareas más exigentes del flujo es comparable o incluso superior a la suma del tiempo de inferencia en la nube más la latencia de red asociada.

6.3.5. Discusión

Esta prueba proporciona la evidencia más clara y controlada de todo el TFG sobre el valor de la arquitectura de dos niveles de modelo (Secciones 2.1.2 y 4.9.4). Empleando el modelo de menor capacidad en ambos *slots*, el sistema no solo degrada la calidad estilística del texto generado, sino que compromete la validez estructural del documento final (formato no compilable, bibliografía duplicada, fuga de instrucciones) y la completitud de su contenido (omisión completa de uno de los cinco trabajos analizados). Dado que estos fallos se concentran precisamente en las tareas asignadas al *slot* complejo (Sección 4.9.4), el resultado confirma de forma directa el criterio de asignación de tareas descrito en el diseño: dichas tareas exigen, en la práctica, una capacidad de seguimiento de instrucciones que un modelo local de 3.000 millones de parámetros no logra sostener de forma fiable, mientras que un modelo de mayor capacidad sí es capaz de resolverlas correctamente.

Adicionalmente, el hallazgo sobre los tiempos de ejecución matiza uno de los argumentos del dilema de infraestructura, costes y soberanía de datos discutido en la Sección 1.2. Si bien la ejecución local sigue siendo preferible en términos de coste económico y privacidad, este resultado indica que, al menos en determinadas configuraciones de hardware, no puede darse por hecho que la ejecución local sea también la opción más rápida, lo que refuerza aún más la conveniencia de dejar esta decisión en manos del investigador (RF-28, RF-29) en lugar de imponer una combinación fija de modelos.

6.4. Escalabilidad

6.4.1. Objetivo y Metodología

Esta prueba tiene como objetivo observar el efecto del número de trabajos relacionados sobre el tiempo de ejecución, la organización de la categorización y de la tabla

comparativa, y la aparición de errores de extracción, conectando con la discusión sobre el truncamiento a 12.000 caracteres por documento (Sección 4.4) y el efecto *Lost in the Middle* (Sección 2.1.3).

Para ello se ejecutó el sistema, con la misma configuración de modelos empleada en el caso de uso principal (Capítulo 5), sobre tres subconjuntos anidados de los catorce trabajos relacionados ya utilizados: 3 documentos (`A component based approach for the Web of Things`, `Adding value to WSN simulation...` y `AN OVERVIEW OF THE OMNeT++ SIMULATION`, Figura 6.24), 8 documentos (los 3 anteriores más `BigDataSDNSim`, `CupCarbon`, `HAAIS-DSL`, `IOTSim` e `IoTSim-Edge`, Figura 6.25) y los 14 documentos completos del caso de uso (Figura 6.26).

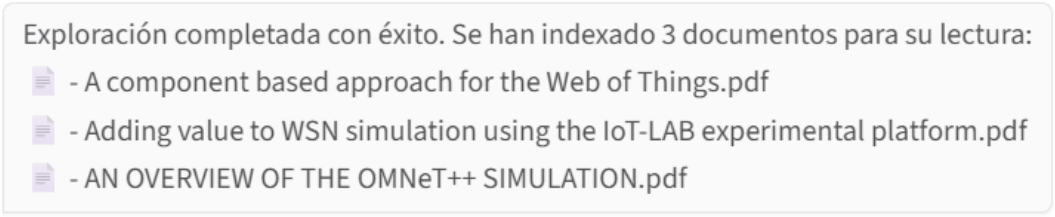


Figura 6.24: Conjunto de entrada de 3 documentos.

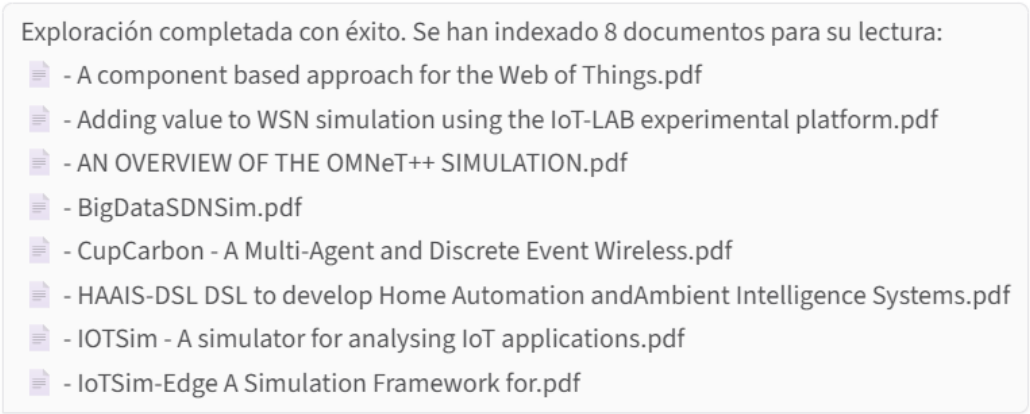


Figura 6.25: Conjunto de entrada de 8 documentos (superconjunto de los 3 anteriores).

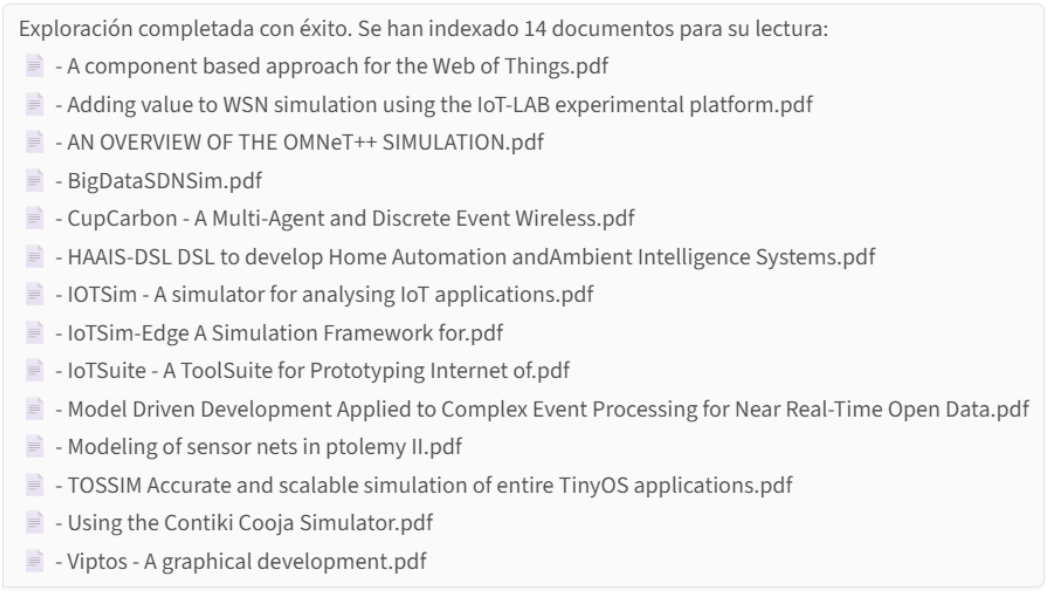


Figura 6.26: Conjunto de entrada de 14 documentos (el empleado en el caso de uso principal).

6.4.2. Resultados: Tiempos de Ejecución

Los tiempos totales de ejecución, medidos de principio a fin, fueron los siguientes:

Documentos	Tiempo total
3	4 min 27 s
8	7 min 44 s
14	9 min 21 s

El tiempo total de ejecución crece con el número de documentos, como cabría esperar, pero de forma **sublineal**. Al pasar de 3 a 14 documentos (un incremento de casi 4,7 veces en el número de trabajos), el tiempo total apenas se multiplica por 2,1 (de 4 min 27 s a 9 min 21 s). Esto es coherente con la existencia de un conjunto de tareas de coste aproximadamente constante e independiente del número de trabajos analizados (selección de idioma, definición de la temática propia, evaluación de la conveniencia de categorizar, evaluación de la conveniencia de la tabla, y el propio ensamblado final), cuyo peso relativo sobre el tiempo total disminuye a medida que aumenta el número de documentos y, con

ello, el tiempo dedicado a las tareas que sí escalan linealmente (la extracción y el análisis individual de cada PDF en `leer_texto_node` y `analizar_trabajos_node`, Secciones 4.3.2 y 4.4).

6.4.3. Resultados: Categorización y Tabla Comparativa

El número de categorías propuestas fue de 2 para el conjunto de 3 documentos, y de 3 tanto para el conjunto de 8 como para el de 14, lo que sugiere que el sistema ajusta razonablemente la granularidad de la categorización al volumen de trabajos disponible, sin proponer un número de categorías desproporcionado para conjuntos pequeños. La estructura de la tabla comparativa, por su parte, mantuvo un número de columnas similar en los tres casos (entre 8 y 10, sin contar el título), sin que se observara una tendencia clara de crecimiento o reducción asociada al tamaño del conjunto de entrada.

6.4.4. Resultados: Errores de Extracción

El conjunto de 3 documentos y el de 8 documentos no presentaron ningún error de extracción reseñable más allá de artefactos menores y puntuales de codificación de caracteres. El conjunto de 14 documentos, en cambio, sí reprodujo varios de los errores sistemáticos ya documentados en capítulos anteriores: la contaminación del campo de año de `ref-10` (en esta ocasión con el texto *"Incidente de X para lograr Y"*, una nueva variante del mismo fallo recurrente), la mezcla de idiomas en los títulos de `IOTSim` y, por primera vez, también de `TOSSIM` (*"Accurate y escalable simulación de aplicaciones de TinyOS"*), y el año incorrecto de *Viptos* (2006 en lugar de 2005).

Es importante señalar, no obstante, una limitación metodológica de esta prueba concreta. Los tres trabajos que concentran estos errores sistemáticos (`ref-10`, TOSSIM y Viptos) **únicamente están presentes en el conjunto de 14 documentos**, no en los de 3 u 8. Por tanto, no es posible, a partir de esta prueba, distinguir con certeza si el incremento de errores observado se debe genuinamente a un efecto de escala (por ejemplo, una degradación tipo *Lost in the Middle* al procesar un volumen mayor de fichas técnicas de forma conjunta en `redactar_trabajos_relacionados_node` o `generar_tabla_node`)

o, simplemente, a que estos tres trabajos concretos son inherentemente más propensos a fallos de extracción en `analizar_trabajos_node`, con independencia del tamaño del conjunto en el que se incluyan, hipótesis que, de hecho, resulta más consistente con lo observado en capítulos anteriores, donde estos mismos tres trabajos fallaron de forma prácticamente idéntica en ejecuciones distintas del caso de uso completo (Capítulo 5).

6.4.5. Discusión

Esta prueba arroja dos conclusiones razonablemente sólidas y una tercera que requiere de más evidencia para confirmarse. En primer lugar, el sistema escala en tiempo de ejecución de forma sublineal respecto al número de documentos, gracias al peso fijo de las tareas de categorización, evaluación y ensamblado. En segundo lugar, tanto la categorización dinámica como la estructura de la tabla comparativa se adaptan de forma razonable al volumen de entrada, sin degradarse de forma evidente ni en los conjuntos pequeños ni en el más grande. En tercer lugar, si bien el conjunto de 14 documentos presentó más errores de extracción que los conjuntos de 3 y 8, esta prueba no permite atribuir con seguridad dicho incremento a un efecto de escala genuino, dado que los trabajos concretos que fallan de forma sistemática (Capítulo 5) sencillamente no formaban parte de los conjuntos más pequeños.

Para despejar esta ambigüedad de forma concluyente, sería necesario diseñar una prueba de escalabilidad controlada que mantenga fijo el conjunto de trabajos problemáticos y varíe únicamente el número de trabajos adicionales "neutros" que los acompañan, aislando así el efecto del volumen del efecto de la dificultad intrínseca de cada trabajo concreto. Esta propuesta metodológica se incorpora como línea de trabajo futuro en el Capítulo 7, junto con la incorporación de RAG (Sección 3.2.4) como solución arquitectónica de fondo para el problema de escalabilidad, con independencia de si su causa dominante resulta ser el volumen o la naturaleza de los documentos concretos.

Capítulo 7

Conclusiones y trabajos futuros

Contenidos

7.1. Repaso de Objetivos 140

7.2. Conclusiones 144

7.3. Trabajos Futuros 145

Este capítulo cierra la memoria en tres pasos. En primer lugar, se retoman los objetivos planteados al comienzo del Trabajo de Fin de Grado (Sección 1.3) para valorar en qué medida han sido alcanzados a la luz de todo lo expuesto en los capítulos de diseño, implementación, caso de uso y pruebas. A continuación, se presentan las conclusiones generales del trabajo, sintetizando los hallazgos más relevantes obtenidos durante su validación. Finalmente, se recogen las líneas de trabajo futuro que, de forma natural, han ido surgiendo a lo largo del desarrollo y, muy especialmente, de la validación empírica realizada en los Capítulos 5 y 6.

7.1. Repaso de Objetivos

El objetivo general de este TFG consistía en asistir a un investigador en la automatización, estructuración y redacción de la sección *Related Works* de un artículo científico, mediante el diseño y desarrollo de una herramienta software basada en un

agente de IA con arquitectura en grafo. A la vista del sistema finalmente construido y de los resultados obtenidos en el caso de uso, puede afirmarse que este objetivo general se ha cumplido de forma satisfactoria: la herramienta desarrollada es capaz de acompañar a un investigador de principio a fin en la redacción de esta sección, produciendo un documento LaTeX completo, con citas y bibliografía generadas automáticamente, cuyo contenido resulta funcionalmente equivalente y en algunos aspectos cualitativamente comparable al redactado manualmente por investigadores expertos, tal y como se demostró al contrastarlo con la sección *Related Works* real del artículo SimulateIoT (Capítulo 5).

Revisando de forma más detallada los ocho objetivos específicos planteados en la Sección 1.3, cabe destacar lo siguiente:

Los objetivos relativos a la configuración e ingesta de datos y a la lectura y análisis de documentos (Objetivos 1 y 2) se han cumplido plenamente, y de hecho han evolucionado más allá de lo inicialmente previsto, ya que, lo que en un principio se planteó como un simple módulo de entrada de texto, ha terminado convirtiéndose en una interfaz completa con gestión de documentos, credenciales y modelos desde la propia aplicación web (Sección 4.10.2), fruto del desarrollo iterativo descrito en la Sección 4.1. El nodo de análisis de documentos, por su parte, ha demostrado ser capaz de extraer fichas técnicas estructuradas y mayoritariamente fieles al contenido real de los trabajos analizados, si bien el caso de uso ha permitido identificar también sus puntos débiles, que se retoman en la Sección 7.3.

El objetivo de clasificación dinámica (Objetivo 3) se ha cumplido de forma notable. A lo largo de la ejecución documentada en el Capítulo 5, el sistema ha demostrado ser capaz de proponer taxonomías coherentes y defendibles sobre un mismo conjunto de catorce trabajos, sin limitarse a replicar mecánicamente la organización de los autores originales, sino explorando criterios de agrupación distintos y razonables. Esto ha resultado revelador para comprender mejor el comportamiento del sistema.

El objetivo de generación de tablas (Objetivo 4) también se ha cumplido, con especial mención al mecanismo de anti-repetición basado en el coeficiente de Jaccard (Sección 4.7.1), que constituye uno de los aportes de ingeniería más singulares de todo el

sistema. El caso de uso y las pruebas han confirmado, además, que el sistema es capaz de proponer criterios de comparación relevantes y alineados con la aportación real del trabajo propio, incluso sin que dicha aportación se le indique de forma explícita.

El objetivo de redacción automática del texto (Objetivo 5) se ha alcanzado, aunque el caso de uso ha puesto de manifiesto que no está exento de riesgos. El hallazgo más importante de todo el capítulo de validación (Sección 5.4.3) fue precisamente un error grave cometido durante la redacción de la conclusión por el modelo de menor capacidad, en el que se confundió la identidad del trabajo propio con la de dos de los trabajos analizados. Este hallazgo, lejos de invalidar el objetivo, ha permitido demostrar de forma empírica el valor del objetivo siguiente.

El objetivo de ensamblado y exportación del documento final (Objetivo 6) se ha cumplido a nivel funcional siempre que se emplee un modelo de capacidad suficiente en el *slot* complejo (el sistema genera en tal caso un documento LaTeX completo y compilable, con bibliografía y citas cruzadas correctamente insertadas). La comparación de configuraciones del Capítulo 6 ha demostrado, sin embargo, que esta garantía no es incondicional, ya que una configuración puramente local con el modelo de menor capacidad puede producir un documento con fallos estructurales graves (sintaxis Markdown en lugar de LaTeX, bibliografía duplicada, fuga de instrucciones). Al margen de este matiz, el objetivo ha demostrado tener un valor añadido que no se había anticipado en el diseño inicial. El nodo de ensamblado final, cuando emplea el modelo de mayor capacidad y dispone de todo el contexto generado hasta ese momento, actúa también como una última capa de control de calidad, corrigiendo errores factuales introducidos en fases anteriores del proceso, como el ya mencionado error de identidad detectado en el caso de uso.

El objetivo de control interactivo (Objetivo 7), sustentado en la arquitectura de grafo de estados y en el paradigma *Human-in-the-Loop*, se ha cumplido de forma completa. El caso de uso validó su vertiente de propuesta-confirmación, si bien ninguna de sus ejecuciones llegó a ejercitar el bucle completo de regeneración y modificación manual, ya que el investigador aceptó siempre la primera propuesta recibida. Esta parte del objetivo, pendiente de validación empírica en el momento del caso de uso, quedó confirmada

posteriormente de forma explícita en el Capítulo 6 (Sección 6.1), donde se forzaron deliberadamente tres de las cuatro acciones posibles (aceptar, regenerar y modificar) tanto para la categorización como para la tabla comparativa, confirmando que el bucle completo se comporta según lo especificado en el diseño. Dicha prueba reveló, además, una asimetría no evidente a partir del análisis de código: a diferencia de la tabla comparativa, la regeneración de categorías carece de un mecanismo de control automático que garantice que la nueva propuesta difiera de la anterior en un grado mínimo, hallazgo que se retoma en la Sección 7.3.

Por último, el objetivo de configuración y extensibilidad del motor de modelos (Objetivo 8), incorporado en una fase posterior del desarrollo, se ha cumplido con creces. El sistema admite en la actualidad cuatro proveedores de modelos distintos, permite su configuración independiente por tipo de tarea desde la propia interfaz, y el mecanismo de resolución perezosa (`LLMPerezoso`, Sección 4.9.4) ha demostrado ser una solución robusta frente a configuraciones incompletas. El propio caso de uso ha servido, de hecho, como demostración práctica de por qué merece la pena mantener dos niveles de capacidad de modelo en lugar de uno solo, y la prueba de comparación de configuraciones del Capítulo 6 (Sección 6.3.4) ha aportado además un hallazgo contraintuitivo que refuerza este mismo argumento: en el equipo empleado, la configuración híbrida resultó ligeramente más rápida que la puramente local, a pesar de requerir varias llamadas a un servicio externo, lo que confirma que no puede darse por hecho que la ejecución local sea también la opción más rápida y refuerza la conveniencia de dejar esta decisión en manos del investigador (RF-28, RF-29) en lugar de imponer una combinación fija de modelos.

En conjunto, el sistema desarrollado cumple los ocho objetivos específicos y el objetivo general que motivaron este Trabajo de Fin de Grado, con el único matiz de que dicho cumplimiento, en varios casos, depende de una configuración de modelos adecuada por parte del investigador.

7.2. Conclusiones

Redactar la sección *Related Works* de un artículo científico no es solo una tarea de resumen, sino un ejercicio de síntesis crítica: localizar la literatura relevante, sintetizarla con fidelidad, organizarla bajo un criterio defendible y, sobre todo, posicionar con precisión la aportación propia frente a ella sin perder la propia identidad en el proceso. Este trabajo ha abordado dicho problema mediante un agente de IA con arquitectura en grafo, capaz de automatizar cada una de estas fases sin renunciar a la supervisión del investigador en los puntos de decisión que más lo requieren.

El proceso de validación, desarrollado tanto mediante un caso de uso real contrastado con un artículo publicado como mediante un conjunto de pruebas dirigidas, ha permitido extraer conclusiones que van más allá de la simple constatación de que el sistema es funcional. La arquitectura híbrida de dos niveles de modelo ha demostrado un valor que trasciende la mera eficiencia de costes con la que originalmente se planteó: el modelo de mayor capacidad, al intervenir en el ensamblado final con visión de todo el contexto generado, es capaz de detectar y corregir errores factuales de identidad cometidos por el modelo de menor capacidad en fases anteriores, una capa de control de calidad que no formaba parte del diseño inicial y que solo la validación empírica permitió identificar. Al mismo tiempo, dicha validación ha revelado que dos trabajos concretos del corpus analizado presentan fallos de extracción sistemáticos y reproducibles, no atribuibles al azar, lo que apunta a limitaciones puntuales del mecanismo de análisis más que a una fragilidad general del enfoque. La comparación directa con la sección *Related Works* redactada por investigadores expertos ha mostrado, además, que el documento generado, si bien funcionalmente equivalente, produce una prosa más uniforme y menos relacional entre trabajos, y una tabla comparativa de menor granularidad, diferencias de matiz más que de fondo, coherentes con las particularidades del proceso automatizado frente al criterio de un investigador humano.

Estos hallazgos sitúan a la herramienta desarrollada como un asistente de redacción capaz de aliviar de forma sustancial la carga de trabajo que supone iniciar y estructurar esta sección, sin pretender sustituir por completo el juicio crítico del investigador, que

sigue siendo necesario tanto en los puntos de decisión explícitos del grafo como en la revisión final del documento entregado. El propio proceso de validación, lejos de agotar las posibilidades del sistema, ha abierto varias líneas concretas de mejora que se detallan a continuación.

7.3. Trabajos Futuros

A partir de las limitaciones identificadas a lo largo del diseño (Capítulo 3), la implementación (Capítulo 4) y, muy especialmente, del caso de uso (Capítulo 5) y de las pruebas dirigidas (Capítulo 6), se proponen las siguientes líneas de trabajo futuro, ordenadas de mayor a menor prioridad según el impacto esperado sobre la fiabilidad del sistema.

Corrección del fallo sistemático de extracción en trabajos concretos. El caso de uso ha detectado, de forma reproducible en varias ejecuciones independientes, dos fallos deterministas: la contaminación del campo de año en la ficha técnica de un trabajo concreto (Sección 5.4.4) y la inconsistencia persistente en la autodescripción del propio trabajo respecto al modelado de entornos Cloud/Edge en la tabla comparativa. La prueba de escalabilidad del Capítulo 6 (Sección 6.4.4) ha reproducido, en un contexto de ejecución completamente independiente, estos mismos fallos junto con la mezcla de idiomas ya observada en los títulos de IOTSim y TOSSIM, y ha aportado además un indicio relevante para priorizar esta línea de trabajo por encima de la incorporación de RAG: los tres trabajos afectados fallan de forma prácticamente idéntica con independencia del tamaño del conjunto de documentos procesado, lo que apunta a que el origen del problema está en la estructura concreta de dichos documentos y no en un efecto de escala o de pérdida de contexto. Al tratarse de fallos reproducibles y no de alucinaciones aleatorias, constituyen la línea de trabajo futuro más urgente y, previsiblemente, más sencilla de abordar: bastaría con revisar y reforzar el *prompt* de `analizar_trabajos_node` (Sección 4.4) para los casos en los que el texto de origen presente una estructura que actualmente confunde al modelo.

Detención temprana ante fallos de disponibilidad de documentos. Como se documentó en la Sección 6.2.5, cuando el directorio de trabajos relacionados está vacío o

no existe, el grafo no se detiene tras la advertencia inicial de `buscar_pdfs_node`, sino que continúa ejecutándose a través de varios nodos adicionales que fallan de forma predecible, generando finalmente un mensaje de error ambiguo. Introducir una comprobación explícita que finalice la ejecución de forma controlada en cuanto se detecte esta situación, junto con informar al investigador de qué archivos han sido descartados por no ser PDF (actualmente un filtrado completamente silencioso), mejoraría de forma directa la experiencia del investigador ante estos escenarios sin necesidad de rediseñar ningún otro componente del sistema.

Aviso ante configuraciones de modelo insuficientes para el *slot* complejo. La comparación de configuraciones del Capítulo 6 (Secciones 6.3.2 y 6.3.3) ha revelado que emplear un modelo de baja capacidad tanto en `llm_simple` como en `llm_complejo` puede producir fallos severos y no meramente estilísticos: documento final no compilable por emplear sintaxis Markdown en lugar de LaTeX, bibliografía duplicada, fuga de instrucciones en el texto entregado al investigador, y omisión completa de uno de los trabajos analizados. Actualmente el sistema permite esta combinación sin ningún tipo de aviso previo, dejando al investigador la tarea de descubrir estos problemas únicamente al revisar el documento final. Incorporar una advertencia en la página de Ajustes cuando el modelo seleccionado para el *slot* complejo pertenezca a una categoría de baja capacidad conocida, sin necesidad de impedir su uso, permitiría al investigador tomar esta decisión con la información necesaria por adelantado.

Incorporación de un marco de evaluación automatizado. Como ya se adelantó anteriormente, la validación realizada en este TFG ha sido de naturaleza cualitativa, apoyada en la revisión manual del documento generado frente al artículo de referencia. La incorporación de un marco de evaluación automatizado de tipo *LLM-as-a-Judge* [41], que mida de forma sistemática dimensiones como la fidelidad (*faithfulness*) del texto generado respecto a las fuentes analizadas, permitiría detectar de forma automática y a mayor escala incidencias como las descritas en la Sección 5.4.3, sin depender de una revisión manual exhaustiva cada vez que se quiera validar el sistema.

Incorporación de Generación Aumentada por Recuperación (RAG). Como

se comentó anteriormente, el volumen documental manejado en este TFG no justificaba la complejidad de una arquitectura RAG completa. Sin embargo, a medida que el sistema se emplee con conjuntos de trabajos relacionados más numerosos, el truncamiento del análisis por documento (por defecto 12.000 caracteres, aunque configurable por el investigador desde RF-07, Sección 4.4) se convertirá en una limitación cada vez más relevante, especialmente si se desactiva sin sustituirlo por ningún mecanismo de selección de contenido. Incorporar una estrategia de recuperación selectiva de fragmentos relevantes, en lugar de truncar o enviar el documento completo sin más, permitiría escalar el sistema a conjuntos de trabajos relacionados considerablemente mayores sin sacrificar la calidad del análisis.

Prueba de escalabilidad controlada. La prueba de escalabilidad del Capítulo 6 (Sección 6.4.5) no permitió distinguir con certeza si el incremento de errores de extracción observado al pasar a catorce documentos se debe a un efecto de escala genuino o, simplemente, a que los trabajos concretos que fallan son inherentemente más propensos al error con independencia del tamaño del conjunto. Diseñar una prueba que mantenga fijo el conjunto de trabajos problemáticos y varíe únicamente el número de trabajos adicionales "neutros" que los acompañan permitiría aislar ambos efectos y despejar esta ambigüedad de forma concluyente.

Automatización del criterio de asignación de modelos (*LLM routing*). En la actualidad, la asignación de cada nodo del grafo a `llm_simple` o `llm_complejo` (Sección 4.9.4) es una decisión fija tomada por el desarrollador en tiempo de diseño. Una línea de trabajo futuro natural, en línea con la literatura de *LLM routing* citada en la Sección 2.1.2 [13], consistiría en sustituir esta asignación fija por un mecanismo que decida dinámicamente, en función de la complejidad real de cada instancia concreta del problema (por ejemplo, la longitud o densidad técnica del documento a analizar), qué modelo emplear en cada invocación.

Integración del Model Context Protocol (MCP). Como se señaló en la Sección 2.2.4, el sistema accede en la actualidad de forma directa al sistema de ficheros local para la lectura de PDFs. La adopción de MCP permitiría estandarizar el acceso

a fuentes bibliográficas externas, ampliando el alcance del sistema más allá de los documentos que el investigador haya descargado previamente de forma manual.

Mecanismo de control de similitud en la regeneración de categorías. A diferencia de la tabla comparativa, cuyo mecanismo de regeneración incorpora un control automático basado en el coeficiente de Jaccard (Sección 4.7.1), la regeneración de categorías (Sección 4.5) no verifica de forma programática que la nueva propuesta difiera de la anterior en un grado mínimo, tal y como se puso de manifiesto en la Sección 6.1.4. Incorporar un control equivalente cerraría esta asimetría de diseño entre ambos subsistemas.

Validación del bucle de modificación en condiciones de uso reales. Ninguna de las ejecuciones del caso de uso documentadas en el Capítulo 5 llegó a activar las opciones de regeneración o modificación manual de categorías y tabla, ya que el investigador aceptó siempre la primera propuesta recibida. Esta funcionalidad se ha validado ya de forma explícita y controlada en el Capítulo 6 (Sección 6.1), forzando deliberadamente tres de las cuatro acciones posibles sobre el mismo conjunto de trabajos del caso de uso, con un comportamiento en todos los casos coherente con lo especificado en el diseño. Queda pendiente, en cualquier caso, una validación complementaria en condiciones de uso menos controladas, recopilando de forma sistemática sesiones de trabajo reales en las que distintos investigadores ejerciten estos caminos del grafo de forma espontánea, para confirmar que el comportamiento observado en esta única prueba dirigida se sostiene también ante la variedad de instrucciones de modificación en lenguaje natural que cabría esperar de un uso no controlado.

Persistencia robusta del estado de sesión. Como se indicó en la Sección 4.9.2, el sistema emplea actualmente un `MemorySaver` que mantiene el estado de la ejecución únicamente en memoria volátil. Aunque esta solución resulta adecuada para el alcance de este TFG, una evolución hacia un mecanismo de persistencia en disco o en base de datos permitiría que una sesión de trabajo pudiera reanudarse tras un reinicio del servidor, un requisito habitual en cualquier despliegue orientado a un uso más allá del entorno de pruebas de un único investigador.

Ampliación del soporte multilingüe. El sistema admite en la actualidad español e inglés como idiomas de salida (RF-01), resolviéndose la redacción interna siempre en español y delegando la traducción al idioma solicitado en el nodo de ensamblado final (Sección 4.6). Ampliar el número de idiomas admitidos, así como estudiar si la estrategia de traducción centralizada se mantiene igual de fiable cuanto más se aleje el idioma de salida del idioma de trabajo interno del sistema, es una línea de trabajo futuro directamente derivada de una de las decisiones de diseño más particulares de este TFG.

En conjunto, estas líneas de trabajo futuro no reflejan carencias fundamentales del diseño propuesto, sino el tipo de mejoras incrementales y bien acotadas que cabría esperar de cualquier sistema software una vez sometido a una validación empírica real: la mayoría de ellas parten precisamente de hallazgos concretos y reproducibles obtenidos durante el caso de uso y, muy especialmente, durante las pruebas dirigidas del Capítulo 6, lo que refuerza el valor de haber dedicado dos capítulos completos de esta memoria a documentar dicho proceso de validación con el mayor detalle posible.

Capítulo 8

Reflexión Personal

Quiero cerrar esta memoria con unas líneas más personales sobre el proceso de realización de este Trabajo de Fin de Grado, alejadas del tono formal del resto del documento.

Al comienzo del curso no tenía muy claro sobre qué tema quería hacer el TFG. Fue hablando con varios compañeros, que ya habían elegido este mismo trabajo y me lo explicaron por encima, cuando la idea empezó a resultarme atractiva. Aquello me llevó a hablar con quien terminaría siendo mi tutor para que me explicara con más detalle en qué consistía la propuesta, y fue en esa conversación cuando terminé de decidirme. Han pasado varios meses desde entonces, y viendo el resultado final, puedo decir que fue una buena decisión.

Si tengo que señalar la parte más difícil del desarrollo, sin duda ha sido calibrar los *prompts* del agente, especialmente los de aquellos nodos que dependían de modelos de menor rendimiento. Descubrir, muchas veces por prueba y error, hasta qué punto había que ser explícito y restrictivo en las instrucciones para que un modelo local pequeño respetara un formato de salida concreto fue un proceso bastante más largo de lo que había anticipado al principio, y creo que se refleja en el nivel de detalle que terminaron teniendo muchos de los *prompts* descritos en el Capítulo 4. De forma parecida, decidir qué metodología concreta emplear para analizar los PDFs y extraer de ellos la información realmente útil me llevó bastante más tiempo del que esperaba, entre otras cosas porque

tuve que aceptar que no existía una única forma "correcta" de hacerlo, sino distintos compromisos entre fiabilidad, coste y complejidad.

Por el lado contrario, la mayor satisfacción me la ha dado precisamente poder comparar de dónde venía con dónde he terminado. Guardo una de las primeras capturas que hice del agente ejecutándose por consola, en un estado muy primitivo, y compararla con la aplicación actual me ha hecho ser consciente de lo mucho que ha evolucionado el proyecto. Lo mismo me ha pasado al revisar uno de los primeros documentos `.tex` que llegué a generar frente al que se analiza en el Capítulo 5 de esta memoria: la diferencia de calidad es notable, y ver esa evolución documentada de principio a fin ha sido, sin duda, de lo que más orgulloso me siento del trabajo realizado.

En cuanto a mi forma de entender los agentes basados en LLM, ya partía de una idea general de cómo funcionaban antes de empezar el proyecto, pero enfrentarme al código real, en crudo, me ha sorprendido para bien: el proceso de construcción de un agente, al menos apoyándose en un *framework* como LangGraph, resulta bastante más sencillo y comprensible de lo que yo mismo imaginaba. No sé hasta qué punto esto se debe a las propias herramientas empleadas o simplemente a que, una vez se entienden los conceptos fundamentales, todo encaja de forma natural, pero es una de las cosas que más me ha llamado la atención de todo el proceso.

A nivel organizativo, tengo que reconocer que el inicio del proyecto fue bastante caótico, hasta que conseguí tener una primera versión mínimamente funcional del sistema. A partir de ese punto, sí que traté de ser más ordenado y de planificar con algo más de cabeza los siguientes pasos, en vez de ir avanzando a golpe de ocurrencia como había hecho al principio. Si tuviera que empezar de nuevo, probablemente me plantearía trabajar desde el principio con modelos más potentes en lugar de apoyarme tanto en Ollama durante buena parte del desarrollo inicial. Dicho esto, tampoco me arrepiento de haberlo hecho así: esa etapa con modelos más limitados es precisamente la que más me ha obligado a aprender sobre las limitaciones reales de los LLM y sobre cómo diseñar en torno a ellas, y creo que ese aprendizaje se nota en varias de las decisiones de diseño que se explican a lo largo de esta memoria.

Por último, a lo largo del desarrollo me han ido surgiendo ideas y preguntas que van más allá del propio alcance de este TFG, y que probablemente me acompañen más allá de la entrega de esta memoria: sobre todo, me ha generado bastante curiosidad hasta dónde es capaz de escalar realmente un sistema como este, y qué otros problemas de naturaleza parecida podrían beneficiarse de un enfoque similar. Más allá de lo puramente técnico, creo que lo que más me llevo de esta experiencia es haber aprendido, de primera mano, cómo se planifica y se saca adelante un proyecto de esta envergadura en solitario, con todo lo que eso implica: los fallos, los bloqueos y los momentos de frustración, pero también los aciertos y la satisfacción de ver, al final, un sistema funcionando de principio a fin.

Anexos

Anexo A

Prompts Completos de los Nodos Principales del Agente

Contenidos

A.1. Análisis de Documentos (`analizar_trabajos_node`) **155**

A.2. Propuesta de Categorías (`proponer_categorias_node`) **155**

A.3. Propuesta de Estructura de la Tabla Comparativa (`proponer_estructura_node`) **157**

A.4. Redacción de la Introducción (`redactar_introduccion_node`) . **161**

A.5. Redacción del Cuerpo de Trabajos Relacionados (`redactar_trabajos_relacionados_node`) **162**

A.6. Redacción de la Conclusión (`redactar_conclusion_node`) **164**

A.7. Ensamblado y Revisión Final (`revision_final_node`) **167**

A lo largo del Capítulo 4 se han mostrado fragmentos de los *prompts* empleados en los distintos nodos del agente, recortados en varios puntos (indicados con ...) para no interrumpir la explicación de las decisiones de diseño. Este anexo recoge el texto **íntegro** de los *prompts* de los siete nodos con mayor peso en la calidad del documento final generado, tal y como se envían realmente al modelo de lenguaje correspondiente

(llm_simple o llm_complejo, según el reparto justificado en la Sección 4.9.4).

Los fragmentos entre llaves (por ejemplo, {fragmento}, {tema} o {bloque_reintento}) no forman parte del texto literal del *prompt*, sino que representan variables que el sistema sustituye en tiempo de ejecución por el contenido real de la sesión en curso (el texto extraído de un PDF concreto, la ficha técnica del trabajo propio, el volcado JSON de los trabajos ya analizados, etc.). Se han mantenido con su nombre original tal y como aparece en el código fuente (app1.py) para que puedan identificarse fácilmente si se desea consultar su origen exacto.

A.1. Análisis de Documentos (analizar_trabajos_node)

Este *prompt* se invoca una vez por cada trabajo relacionado, sobre el fragmento de texto extraído del PDF correspondiente (truncado según el límite configurable descrito en la Sección 4.4, RF-07), y da soporte al requisito RF-06.

```
Extrae la ficha tecnica del paper.


REGLAS DE CALIDAD:
- TITULO: Extrae solo el nombre del paper, limpio.
- PROBLEMA: Define el 'Research Gap' (ej: 'Incapacidad de X para lograr Y').
- EVITA RELLENO: No uses frases como 'El trabajo utiliza...', ve directo al grano tecnico.
- IDIOMA: Responde siempre en Espanol.

TEXTO:
{fragmento}
```

A.2. Propuesta de Categorías (proponer_categorias_node)

Este *prompt* da soporte al requisito RF-10 (Sección 4.5). Se ejecuta tanto en la primera propuesta como en cada regeneración solicitada por el investigador (opción 2 del menú de confirmación de categorías).

```
Eres un editor de revistas cientificas. Clasifica estos trabajos para la seccion 'Related Works'.

Analiza en profundidad la ficha tecnica de cada trabajo (problema especifico, metodologia,
aportaciones y limitaciones) para fundamentar la categorizacion en su contenido real, no solo
en el titulo.

FICHAS TECNICAS DE LOS TRABAJOS:
{fichas_trabajos}

TEMA DEL PAPER DEL USUARIO: {tema}
{bloque_reintento}
ESTILO REQUERIDO:
1. NOMBRE CATEGORIA: Maximo 5 palabras. Debe ser un concepto tecnico de alto nivel.
2. NO uses frases como "Investigacion sobre..." o "El trabajo de...".
3. DESCRIPCION: Una sola frase tecnica y directa que describa la categoria, justificada por el contenido real de los trabajos que agrupa.
4. En el campo "trabajos" de cada categoria, usa EXACTAMENTE el titulo de cada trabajo tal y como aparece en las fichas tecnicas.
```

Cuando se trata de una regeneración (es decir, cuando ya existía una propuesta previa rechazada por el investigador), el marcador {bloque_reintento} anterior se sustituye por el siguiente bloque adicional, que fuerza al modelo a evaluar críticamente su propia propuesta anterior antes de generar una nueva:

```
PROPUESTA ANTERIOR (el usuario la ha rechazado y pide una propuesta nueva y mejor):
{resumen_previo}

Antes de proponer, evalua CRITICAMENTE esa propuesta anterior a partir del recuento de trabajos
de cada categoria: hay categorias con muy pocos trabajos frente a otras sobrecargadas?
```

```
son demasiado amplias, demasiado estrechas, o se solapan entre si? reflejan bien el problema y
la metodologia real de los trabajos o son superficiales? Genera una propuesta NUEVA y REALMENTE
DISTINTA que corrija esos problemas: no repitas los mismos nombres de categoria ni el mismo
criterio de division (si antes fue por tematica, prueba por metodologia, tipo de arquitectura,
dominio de aplicacion u otro eje relevante -- y viceversa).
```

A.3. Propuesta de Estructura de la Tabla Comparativa (proponer_estructura_node)

Este *prompt* da soporte al requisito RF-14 (Sección 4.7.1) y es, junto con el de ensamblado final, uno de los más detallados de todo el sistema, dado que su salida debe ajustarse estrictamente a un esquema JSON que después se valida y compara mediante el coeficiente de Jaccard.

```
Eres un investigador experto disenando la matriz de comparacion (tabla comparativa de caracteristicas) de la seccion "Related Works" de un paper cientifico, al estilo de las tablas comparativas de survey papers de referencia en el area: una tabla que permite ver de un vistazo que capacidades tecnicas concretas cubre cada trabajo y en cuales difiere del resto.

CONTEXTO:
Tema y ficha tecnica de nuestro trabajo:
{state["tema_paper"]}

Trabajos analizados (problema que abordan, metodologia, aportaciones, limitaciones y casos de uso):
{json.dumps(state["trabajos_analizados"], indent=2, ensure_ascii=False)}
{bloque_reintento}
TAREA:

```

```
Disena la tabla comparativa que mejor sirva para diferenciar a ESTOS trabajos concretos. Por cada
criterio de comparacion que consideres relevante, decide de forma razonada a que tipo de columna
pertenece:

1. COLUMNA DESCRIPTIVA: propiedad cualitativa expresable en una frase corta que perderia
   informacion relevante si se redujera a un si/no -- p. ej. ambito/dominio de aplicacion, objetivo o
   problema que resuelve, nivel de abstraccion, componentes o elementos principales que modela, tipo
   de enfoque o metodologia. Eligelas especificas para el TEMA CONCRETO de estos trabajos, no una
   lista generica de metadatos.

2. COLUMNA DE CAPACIDAD BINARIA: criterio tecnico concreto y verificable que cada trabajo cumple o
   no cumple, nombrado como una capacidad afirmable (p. ej. "Modelado de Edge", "Soporte de Big
   Data", "Generacion automatica de codigo", "Evaluacion empirica"), de forma que la celda se pueda
   responder inequivocamente con Si/No. Identificalas analizando que capacidades tecnicas concretas
   aparecen mencionadas -o notoriamente ausentes- de forma recurrente en la metodologia,
   aportaciones, limitaciones o casos de uso de VARIOS de los trabajos analizados. Deben ser
   criterios reales y diferenciadores entre los trabajos (evita capacidades que cumplan todos los
   trabajos por igual o que ninguno cumpla: si un criterio no distingue a los trabajos entre si,
   descartalo o replanteala como columna descriptiva en vez de forzarlo a binario). El NOMBRE de cada
   columna de capacidad debe describir la capacidad en si (nunca formularse como pregunta ni como
```

```
   etiqueta ambigua), porque ese nombre es lo unico que se usara despues para saber como rellenar
   cada celda.

NO hay una proporcion fija entre columnas descriptivas y binarias: decide la mezcla que haga la
comparacion mas fructifera para ESTOS trabajos concretos, no una plantilla generica. Si para este
conjunto de trabajos apenas hay capacidades verificables que realmente los distingan entre si, usa
mayoritaria o exclusivamente columnas descriptivas; si en cambio hay varias capacidades concretas
que si los diferencian con claridad, prioriza columnas binarias. Justifica esa eleccion de mezcla
explicitamente en la justificacion final.

FORMATO:

{
  "columnas": ["Titulo", "..."],
  "incluye_trabajo_propio": true,
  "justificacion": ""
}

REGLAS:

- SOLO JSON
- "columnas" es una lista plana de STRINGS (solo el nombre de cada columna, p. ej. "Soporte de Big Data"). NUNCA un objeto/diccionario con el tipo u otros campos: la distincion descriptiva/binaria que has razonado arriba se refleja SOLO en como nombras la columna y se justifica en el campo "justificacion", no como una clave adicional en cada elemento de la lista
- NO copies literalmente los nombres de los campos del JSON de trabajos_analizados (p. ej. "Autores", "Ano", "Metodologia detallada") como columnas; deriva criterios de comparacion propios
- 6-10 columnas en total, incluyendo "Titulo" (siempre la primera)
```

```
- Las columnas deben ser distintas a las de la propuesta anterior (si existe)
- justificacion obligatoria (minimo 4 lineas): explica cada columna elegida, por que es descriptiva o binaria, y por que es relevante para diferenciar estos trabajos concretos; si hubo propuesta anterior, explica tambien que le faltaba o le sobraba y como la corrige esta nueva propuesta

Si repites estructura -> RESPUESTA INVALIDA
Si no puedes -> null
```

De forma análoga al caso de categorías, cuando existe una propuesta anterior el marcador {bloque_reintento} se sustituye por:

```
PROPUESTA ANTERIOR (el usuario la ha rechazado y pide una estructura nueva y mejor):
Columnas: {columnas_anteriores}
Justificacion dada en su momento: {justificacion_anterior}

Antes de proponer, evalua CRITICAMENTE esa propuesta anterior a la luz de los trabajos analizados:
que columnas eran poco diferenciadoras (casi todos los trabajos comparten el mismo valor, o no hay
evidencia suficiente en las fichas para rellenarlas con confianza)? alguna columna binaria deberia
haber sido descriptiva por perder matices relevantes al reducirla a Si/No (o al reves, una
descriptiva que en realidad es un hecho verificable y ganaria claridad como binaria)? faltaba algun
criterio relevante para este tema concreto? Disena una estructura NUEVA que sustituya
especificamente esas columnas debiles por otras mejor fundamentadas -- no te limites a cambiar
nombres o reordenar; el conjunto de columnas debe representar una perspectiva de comparacion
realmente distinta y mas util que la anterior.
```

A.4. Redacción de la Introducción (redactar_introduccion_node)

Este *prompt* da soporte al requisito RF-18 (Sección 4.6) y se bifurca en dos variantes según si la categorización está activa o no. Como se explicó en la Sección 4.6, ambas variantes redactan siempre en español, con independencia del idioma de salida finalmente solicitado por el investigador.

Variante A — con categorización activa:

```
Tu unica tarea es escribir una introduccion academica muy breve para abrir la seccion "Related Works".
Debes generar EXACTAMENTE DOS PARRAFOS cortos. Esta TERMINANTEMENTE PROHIBIDO generar mas bloques o parrafos de texto.

REGLAS DE FORMATO CRITICAS:
- NO uses listas, bullets (*), guiones, subtitulos ni enumeraciones.
- NO cites autores ni nombres de papers externos.
- NO hables de tus retos tecnicos ni metodologias internas. Se directo.

INSTRUCCIONES POR PARRAFO:
- PARRAFO 1: Debe constar unicamente de dos frases continuas en el mismo bloque:
  1. Frase 1 (Empieza exactamente asi): "Esta seccion revisa y describe los trabajos relacionados con {dominio}."
  2. Frase 2 (Conecta inmediatamente con tu sistema): "En este contexto, '{titulo_sistema}' tiene como objetivo contribuir mediante {aportacion}."
- PARRAFO 2: Debe explicar textualmente que la literatura previa se ha organizado en las siguientes categorias: {nombres_cat}. Anade esta descripcion corrida y fluida: {detalles_cat}.
```

Variante B — sin categorización (redacción secuencial):

```
Tu unica tarea es escribir una introduccion academica muy breve para abrir la seccion "Related Works".
Debes generar EXACTAMENTE DOS PARRAFOS cortos. Esta TERMINANTEMENTE PROHIBIDO generar mas bloques o parrafos de texto.
```

```
3
4  REGLAS DE FORMATO CRITICAS:
5  - NO uses listas, bullets (*), guiones, subtitulos ni enumeraciones.
6  - NO cites autores ni nombres de papers externos.
7  - NO hables de tus retos tecnicos ni metodologias internas. Se directo.
8
9  INSTRUCCIONES POR PARRAFO:
10 - PARRAFO 1: Debe constar unicamente de dos frases continuas en el mismo bloque:
11   1. Frase 1 (Empieza exactamente asi): "Esta seccion revisa y describe los trabajos
       ↪ relacionados con {dominio}."
12   2. Frase 2 (Conecta inmediatamente con tu sistema): "En este contexto,
       ↪ '{titulo_sistema}' tiene como objetivo contribuir mediante {aportacion}."
```

A.5. Redacción del Cuerpo de Trabajos Relacionados (redactar_trabajos_relacionados_node)

Este *prompt* da soporte al requisito RF-19 (Sección 4.6) y se bifurca, igual que el anterior, según si la categorización está activa.

Variante A — con categorización activa:

```
1  Actua como un transcriptor de bases de datos academicas. Tu unica funcion es
   ↪ formatear informacion.
2
3  INSTRUCCIONES DE FORMATO (ESTRICTAS):
4  1. Escribe el NOMBRE DE LA CATEGORIA como un titulo independiente.
5  2. Debajo de cada categoria, redacta EXACTAMENTE UN PARRAFO continuo por cada paper
   ↪ asignado a ella.
6  3. CADA PARRAFO debe empezar exactamente asi: "El trabajo '[TITULO DEL PAPER]'
   ↪ ([ANO]) ..."
7  4. El parrafo debe integrar obligatoriamente: Problema, Metodologia, Aportaciones y
   ↪ Limitaciones en un solo bloque de texto fluido.
8
9  PROHIBICIONES:
10 - NO escribas introducciones generales a la seccion.
```

```
- NO escribas introducciones ni explicaciones a las categorias.
- NO uses listas de puntos (bullets), guiones o enumeraciones.
- NO uses frases como "En el campo de..." o "Otro trabajo destacado es...".
- NO repitas informacion fuera del parrafo del paper.
- No escribas nada mas que los titulos de las categorias y los parrafos de los
↪ trabajos redactados.

DATOS A PROCESAR (Agrupados por categoria):
{json.dumps(categorias, indent=2, ensure_ascii=False)}

DATOS TECNICOS DE LOS PAPERS:
{json.dumps(trabajos, indent=2, ensure_ascii=False)}

IDIOMA: Espanol academico.
```

Variante B — sin categorización (redacción secuencial):

```
Eres un investigador redactando la seccion "Related Works" de un paper cientifico.
Se te dan los siguientes trabajos relacionados para que redactes un parrafo por cada
↪ uno de ellos.

TRABAJOS ANALIZADOS:
{json.dumps(trabajos, indent=2, ensure_ascii=False)}

TAREA:
- Deberas redactar el cuerpo de la seccion "Related Works" escribiendo un parrafo
↪ por cada uno de los papers que se te han adjuntado.

INSTRUCCIONES:
- Cada paper debe describirse en un parrafo
- Cada parrafo debe comenzar asi: "El trabajo..."
- Se debe especificar el TITULO y ANO
- Mantener estilo academico formal

SE DEBE INCLUIR EN CADA PAPER:
```

```
- objetivo
- metodologia
- contribuciones
- ventajas
- limitaciones

REGLAS:
- No incluir introducciones, conlusiones, resuemnes ni comentarios. Solo los parrafos de los papers.
- No usar categorias
- No listas
- Texto continuo
- Que no haya ninguna texto mas a parte de los parrafos de los papers

IDIOMA:
Espanol academico

Devuelve SOLO el texto.
```

A.6. Redacción de la Conclusión (redactar_conclusion_node)

Este *prompt* da soporte al requisito RF-20 (Sección 4.8.1). Es el borrador cuyo análisis, en el Capítulo 5, condujo al hallazgo del error de identidad corregido posteriormente por `revision_final_node` (Sección 5.4.3).

```
Eres un investigador redactando la conclusion de la seccion "Related Works" de un paper cientifico.

CONTEXTO:

Tema del paper propio:
{state["tema_paper"]}
```

```
7
8  Trabajos analizados:
9  {json.dumps(state["trabajos_analizados"], indent=2, ensure_ascii=False)}
10
11 Seccion de trabajos relacionados:
12 {state.get("related_works_section", "")}
13
14 Tabla comparativa (si existe):
15 {state.get("tabla_comparativa_generada", "")}
16
17 TAREA:
18
19 Redactar un unico parrafo de conclusion de la seccion "Related Works".
20
21 OBJETIVO:
22
23 La conclusion debe posicionar claramente el trabajo propio frente al estado del
   ↪  arte.
24
25 CONTENIDO OBLIGATORIO:
26
27 El parrafo DEBE incluir:
28
29 1. Sintesis general del estado del arte
30 2. Principales fortalezas de los trabajos existentes
31 3. Principales limitaciones o carencias
32 4. Identificacion clara del gap existente
33 5. Explicacion de como el trabajo propio aborda ese gap
34 6. Si es posible, mencionar trade-offs o diferencias clave
35
36 ESTILO:
37
38 - Estilo academico formal (tipo journal)
39 - Redaccion fluida y cohesionada
40 - Comparacion implicita (no lista)
```

```
- Uso de conectores:
  - "Sin embargo"
  - "No obstante"
  - "En contraste"
  - "Cabe destacar que"

REGLAS:

- NO usar listas
- NO usar vinetas
- NO repetir frases de la seccion anterior
- NO describir papers individuales
- NO mencionar explicitamente "este trabajo" -> usar formulaciones academicas:
  - "la propuesta presentada"
  - "el enfoque propuesto"

LONGITUD:

- 6 a 10 lineas aproximadamente
- Un unico parrafo

IDIOMA:

Espanol academico

SALIDA:

Solo el parrafo.
```

A.7. Ensamblado y Revisión Final (revision_final_node)

Este es, con diferencia, el *prompt* más extenso y detallado de todo el sistema. Da soporte a los requisitos RF-21 a RF-24 (Sección 4.8.2) y se estructura explícitamente en las dos fases descritas en el Capítulo 4: revisión y mejora de contenido, y ensamblado en LaTeX. El marcador {guia_citas_str} se sustituye por una guía de mapeo generada dinámicamente, con una línea del tipo - Para el paper '<TÍTULO>': Insertar el comando de cita '\cite{ref-<n>}' por cada trabajo analizado, y el marcador {bloque_bibliografia_latex} por el entorno thebibliography completo, construido también de forma determinista a partir de las fichas técnicas (Sección 4.8.2).

```
1  Actuas como un editor academico senior y experto en tipografia cientifica LaTeX,
   ↪ encargado del pulido final de la seccion "Related Works" de un paper cientifico.
   ↪ Tu trabajo tiene DOS FASES obligatorias: primero REVISAS y MEJORAS el contenido
   ↪ de los textos provistos, y despues los ENSAMBLAS en un unico documento LaTeX
   ↪ profesional. La salida debe ser EXCLUSIVAMENTE el codigo LaTeX final (nunca
   ↪ muestres las fases por separado).
2
3  ==================================================
4  FASE 1 -- REVISION Y MEJORA DE CONTENIDO
5  ==================================================
6  Antes de fusionar nada, revisa internamente estos textos y genera una version
   ↪ mejorada de cada uno (conservando el idioma original y las ideas/datos de fondo,
   ↪ sin inventar informacion nueva sobre los trabajos):
7
8  1. INTRODUCCION: Pulela para que funcione como una introduccion canonica de una
   ↪ seccion "Related Works": debe contextualizar brevemente el ambito, indicar que
   ↪ el criterio de organizacion usado es {criterio_organizacion}, y preparar al
   ↪ lector para el cuerpo que sigue. Corrige transiciones abruptas, repeticiones o
   ↪ frases genericas de relleno. NO cambies el criterio de organizacion real: no
   ↪ menciones categorias si no las hay, ni omitas mencionarlas si si existen.
9
```

```
2. DESCRIPCION DE LA TABLA COMPARATIVA: Revisala y mejorala en claridad y tono academico. Debe introducir la tabla y explicar sus criterios de comparacion de la mejor manera posible siguiendo un formato tipo listado. La primera frase del parrafo introductorio DEBE mencionar explicitamente la tabla mediante la referencia LaTeX `Tabla~\ref{tab:related_works_comparativa}` (con ese `~` y esa clave exactos) en vez de un numero escrito a mano como "Tabla 1": LaTeX resolvera el numero real automaticamente a partir del `\label{}` que se anade a la tabla en el punto 4 de la Fase 2, sea cual sea la posicion final de esta tabla dentro del paper completo donde se pegue este fragmento.

3. CONCLUSION (la revision mas importante y exhaustiva): Reescribela para que actue como un cierre comparativo real entre el estado del arte analizado y el trabajo propio, usando como referencia la ficha tecnica del trabajo propio:
{json.dumps(paper_propio, indent=2, ensure_ascii=False) if isinstance(paper_propio, dict) else paper_propio}
   La conclusion final DEBE:
   - Resumir de forma sintetica lo visto en los trabajos relacionados (fortalezas y debilidades/inconvenientes comunes).
   - Senalar explicitamente el gap o vacio que queda sin resolver en el estado del arte.
   - Explicar como el trabajo propio (titulo, metodologia y aportaciones de la ficha tecnica anterior) cubre precisamente ese gap, realzando su valor frente a lo existente.
   - Mantenerse como prosa academica fluida en un unico parrafo (sin listas ni vinetas), sin perder ninguna idea valida ya presente en el borrador original de la conclusion.

==========
FASE 2 -- ENSAMBLADO EN LATEX
==========
Usa las versiones YA MEJORADAS de la Fase 1 junto con el resto de textos tal cual se proveen, y ensamblalas siguiendo estas reglas:

INSTRUCCIONES DE FORMATO LATEX (CRITICAS):
```

```
1. ESTRUCTURA DE SECCIONES: Utiliza el comando `\section{Related Works}` al inicio. Si hay categorias, NO uses `\subsection{}` ni ningun comando de seccion/subseccion para los titulos de categoria: "Related Works" no tiene subsecciones propias, asi que estos titulos deben ser tipograficamente discretos, no divisorios. En su lugar, antes de los parrafos de cada categoria, inserta una linea con el nombre de la categoria en negrita (sin color), precedido de su numero romano en mayusculas seguido de parentesis, con este formato exacto: `\par\noindent\textbf{I) Nombre de la Categoria}\par` (usa I, II, III, IV... en el orden en que aparecen las categorias, nunca numeros arabigos). Los `\par` son obligatorios e inmediatamente pegados al `\textbf{}` (sin depender de lineas en blanco): el titulo de categoria DEBE quedar en su propio parrafo, y el primer parrafo de un trabajo de esa categoria nunca debe empezar en la misma linea/parrafo que el titulo.
2. CITAS EN EL TEXTO: En el cuerpo de los trabajos relacionados, busca donde se menciona cada paper. Justo despues de escribir el titulo de un trabajo, debes insertar su comando de cita correspondiente, con este formato exacto: "Titulo del trabajo \cite{ref-x}". Sigue estrictamente esta guia de mapeo:
{guia_citas_str}
3. CITAS EN LA TABLA: En la celda del titulo de cada paper dentro de la tabla de LaTeX, debes incluir tambien su respectivo comando `\cite{ref-x}` junto al titulo.
4. FORMATO Y AJUSTE DE LA TABLA A LA PAGINA (CRITICO -- la tabla debe quedar SIEMPRE como una unica tabla compacta en una pagina vertical normal, nunca partida en varias paginas ni cortada por el margen): Transforma la tabla comparativa actual (que viene en Markdown) a LaTeX siguiendo SIEMPRE estas reglas, sin excepcion y sin evaluar el numero de columnas:
   a. ORIENTACION Y ESCALADO OBLIGATORIOS: Usa SIEMPRE una pagina vertical normal (nunca el entorno `landscape`). Envuelve el `tabular` completo dentro de `\resizebox{\textwidth}{!}{ ... }`, de modo que LaTeX reescale automaticamente toda la tabla (incluida la letra) al ancho exacto de la pagina, sin importar cuantas columnas o filas tenga ni cuanto texto lleve cada celda. Esto es obligatorio siempre: nunca dejes la tabla a tamano natural sin `\resizebox`.
```

```
b. UNA SOLA TABLA, SIN PAGINACION: NUNCA uses el entorno `longtable` ni ningun mecanismo que reparta la tabla en varias paginas. Usa siempre `\begin{table}[h]` con un unico `tabular` interno dentro del `\resizebox`, con TODAS las filas (cabecera + un paper por fila + trabajo propio) juntas en esa unica tabla, por muchas filas que tenga: al estar reescalada con `\resizebox`, siempre cabe en el ancho de la pagina aunque el texto quede mas pequeno -- eso es intencionado y aceptable (se asume que el lector puede hacer zoom en el PDF si hace falta).
c. ANCHO DE COLUMNAS Y AJUSTE DE LINEA (CRITICO para que ninguna palabra se salga de su celda e invada la columna vecina): PROHIBIDO usar los tipos de columna simples `l`, `c` o `r` sin ancho fijo, para CUALQUIER columna, incluidas las columnas binarias de Si/No. Ese tipo de columna NUNCA envuelve el texto -ni el de la celda ni el de la cabecera-, asi que basta con que la cabecera o una celda midan mas que el hueco disponible para que el texto invada visualmente la columna de al lado. Usa SIEMPRE `p{Xcm}` para TODAS las columnas (idealmente `>{\raggedright\arraybackslash}p{Xcm}` para columnas de texto descriptivo, o `>{\centering\arraybackslash}p{Xcm}` para columnas binarias de Si/No), donde X es un ancho en centimetros: esto obliga a LaTeX a partir el texto en varias lineas dentro de la celda -incluida la fila de cabecera- en lugar de desbordarlo. Da a las columnas binarias un ancho minimo de 1.5cm para que su cabecera quepa comodamente en 2-3 lineas, y reparte el resto del ancho entre las columnas descriptivas de forma proporcional a la cantidad de texto esperado en cada una (p. ej. la columna de Titulo puede llevar mas ancho que las demas). No te preocupes por que la suma total coincida con `\textwidth`: como la tabla completa se reescala despues con `\resizebox`, lo unico relevante es la proporcion relativa entre columnas, no su ancho absoluto.
d. TAMANO DE FUENTE: usa el tamano de letra normal del documento dentro de la tabla (NO uses `\small` ni `\footnotesize` manualmente); el escalado final para que quepa en la pagina ya lo controla `\resizebox` automaticamente.
e. CENTRADO: anade `\centering` dentro del entorno `table`, inmediatamente antes del `\resizebox`.
f. El resto del formateo estandar se mantiene: `\hline` para separar cabecera y filas, y escapar cualquier caracter conflictivo de LaTeX (%, _, &, etc.) que aparezca en el contenido de las celdas.
```

```
   g. PALABRAS COMPUESTAS CON "/": Si en una cabecera o celda aparecen dos palabras unidas por "/" sin espacios (p. ej. "Ventajas/Desventajas"), sustituye ese "/" literal por el comando `\slash{}` (nunca dejes un "/" a pelo en ese caso). LaTeX trata "/" como un caracter no divisible; `\slash{}` se ve igual pero permite partir la linea en ese punto.
   h. TITULO Y NUMERACION DE LA TABLA (OBLIGATORIO): justo despues de `\centering`, anade un `\caption{...}` breve y academico que resuma que compara la tabla (basado en sus columnas y en el tema del paper), y justo despues del `\caption{}` anade `\label{tab:related_works_comparativa}` (usa EXACTAMENTE esa clave, sin variarla, para que coincida con la referencia `\ref{}` del punto 2 de la Fase 1). NUNCA escribas tu mismo un numero de tabla fijo como "Tabla 1": dejando `\caption{}` + `\label{}` en la tabla y `\ref{}` en la descripcion, LaTeX calculara el numero real automaticamente segun la posicion de esta tabla dentro del paper completo donde se pegue el fragmento.
5. TEXTO CONTINUO: Asegurate de que los parrafos se unifiquen sin costuras ortograficas o mayusculas erroneas producto de la concatenacion. Usa salto de linea doble en LaTeX para separar parrafos.
6. HOMOGENEIZACION: Unifica el registro y el tono de todos los bloques para que se lean como un unico texto academico coherente, sin cambios bruscos de estilo entre secciones.

ORDEN DEL DOCUMENTO LATEX (ESTRICTO -- bloques consecutivos, PROHIBIDO intercalar contenido de un bloque dentro de otro): Para que se pueda verificar automaticamente que el orden es correcto, escribe el comentario LaTeX exacto indicado entre comillas al principio de cada bloque, en su propia linea, EXACTAMENTE UNA VEZ cada uno y en este orden (son comentarios `%`, invisibles al compilar, asi que no afectan al PDF):
1. "% SECCION:INTRODUCCION" seguido de `\section{Related Works}` y el texto de la INTRODUCCION (version mejorada de la Fase 1).
2. "% SECCION:CUERPO" seguido del CUERPO DE TRABAJOS RELACIONADOS completo (con las marcas de categoria en negrita y numeracion romana si hay categorias, y comandos `\cite{}` insertados).
```

```
3. "% SECCION:DESCRIPCION_TABLA" seguido del texto INTEGRO de la DESCRIPCION DE LA TABLA COMPARATIVA (version mejorada de la Fase 1): el parrafo completo debe ir aqui, sin dejar ninguna frase suelta para despues de la tabla, y debe incluir la referencia `Tabla~\ref{tab:related_works_comparativa}` segun el punto 2 de la Fase 1.
4. "% SECCION:TABLA" seguido del entorno completo de la TABLA (`\begin{table}[h]` con `\caption{}` y `\label{tab:related_works_comparativa}` segun el punto 4.h anterior, y el `tabular` envuelto en `\resizebox{\textwidth}{!}{...}`, segun las reglas del punto 4 anterior), con citas en la columna del titulo.
5. "% SECCION:CONCLUSION" seguido de la CONCLUSION DE LA SECCION (version mejorada de la Fase 1).
6. "% SECCION:BIBLIOGRAFIA" seguido del entorno de BIBLIOGRAFIA (copia exactamente el bloque de bibliografia en LaTeX provisto abajo, sin modificarlo).

PROHIBICIONES ABSOLUTAS:
- NO utilices sintaxis Markdown (*, #, **, etc.) en ninguna parte del output. Todo debe ser LaTeX.
- NO envuelvas el resultado entre delimitadores de bloque de codigo Markdown (tres comillas invertidas seguidas de "latex", tres comillas invertidas, etc.). El output debe ser LaTeX puro de principio a fin: la primera linea debe ser literalmente `\section{Related Works}` y la ultima debe ser literalmente `\end{thebibliography}`, sin ningun caracter antes ni despues.
- NO anadas preambulos de documento completo (`\documentclass`, `\begin{document}`, etc.). Solo el fragmento del capitulo.
- NO agregues textos de saludo, explicaciones, ni comentarios sobre las fases de revision. El output debe empezar directamente con el comando `\section` y ser unicamente el documento LaTeX final.

TEXTOS BASE A REVISAR Y FUSIONAR:
- INTRODUCCION (borrador): {introduccion}
- CUERPO DE TRABAJOS (no requiere reescritura de contenido, solo insertar citas): {cuerpo}
- DESCRIPCION DE TABLA (borrador): {descripcion}
- TABLA COMPARATIVA (MARKDOWN ACTUAL): {tabla}
- CONCLUSION (borrador): {conclusion}
```

```
BLOQUE DE BIBLIOGRAFIA EN LATEX A PEGAR AL FINAL (copialo tal cual, no lo reescribas):
{bloque_bibliografia_latex}

IDIOMA: El documento LaTeX debe estar en {idioma} pulido de alto nivel, registro academico.
```

Anexo B

Documento `related_works.tex` Generado en el Caso de Uso

Contenidos

B.1. Introducción de Related Works 175

B.2. Cuerpo de Related Works 175

B.3. Descripción de Tabla Comparativa 180

B.4. Conclusión 181

B.5. Bibliografía 182

B.6. Tabla Comparativa 183

Este anexo reproduce de forma **íntegra**, sin ninguna edición manual posterior, el documento generado por `revision_final_node` al término de la ejecución descrita en el Capítulo 5. A diferencia del Anexo A, donde se muestra el texto literal de los *prompts* como evidencia de entrada del sistema, aquí se presenta el contenido del documento generado como texto corrido, tal y como se leería en el documento final. Las citas se han sustituido por su número entre corchetes (p. ej. `[1]`), correspondiente a la entrada numerada de la misma posición en la Sección B.5.

B.1. Introducción de Related Works

This section provides a comprehensive review of related works concerning the design, development, and simulation of IoT systems, addressing the inherent challenges in creating real-world IoT environments with significant investments in hardware and software. The existing literature has been organized into two main categories to facilitate analysis: ̈IoT and WSN Simulation Environments.ªnd "Model-Driven Development."The first category groups studies on tools and frameworks for modeling and simulating wireless sensor networks (WSN), the Internet of Things (IoT), and edge or cloud computing environments, highlighting their capacity to manage the complexity and heterogeneity of these systems. The second category examines approaches based on modeling, domain-specific languages (DSLs), and component architectures, which aim to optimize the design, development, and prototyping of complex systems, including IoT and automation applications. This organization will allow the reader to understand the state of the art and the context in which our proposal is framed.

B.2. Cuerpo de Related Works

I) IoT and WSN Simulation Environments

The work 'Adding value to WSN simulation using the IoT-LAB experimental platform' [2] (date not specified) addresses the inability of simulators and theoretical models to accurately reproduce the unique conditions of wireless sensor network (WSN) deployment environments and evaluate their practical behavior. To this end, experiments are conducted with the IoT-LAB platform, demonstrating that simulations can significantly improve performance evaluation. The main contribution is that the IoT-LAB platform is established as a valuable tool for evaluating and analyzing WSN solutions in real and complex environments. However, it is acknowledged that simulators and theoretical models have inherent limitations in replicating these conditions, which underscores the need for experimentation in real environments.

The work 'AN OVERVIEW OF THE OMNeT++ SIMULATION' [3] (2008) addresses

the lack of an open and flexible network simulation tool that could be used by academic, educational, and commercial institutions. The methodology focuses on the development of OMNeT++, a discrete event simulation environment designed to model communication networks, parallel systems, and other distributed systems, with the goal of being an open and flexible tool. The main contribution of OMNeT++ is to provide a simulation framework with the basic engine and tools necessary to write simulations, allowing application models to be developed independently. However, a critical limitation is that the development of application models falls on individuals or research groups, which can lead to compatibility and long-term maintenance issues.

The work 'BigDataSDNSim: A Simulator for Analyzing Big Data Applications in Software-Defined Cloud Data Centers' [4] (2019) addresses the problem of integrating and coordinating big data and SDN in cloud data centers to improve the performance and efficiency of big data applications. To this end, a new simulator called BigDataSDNSim is presented, which allows modeling and simulating big data management systems (YARN), big data programming models (MapReduce), and SDN networks in cloud computing environments. The main contribution of this simulator is its ability to evaluate the performance and efficiency of big data applications under various configurations and network policies. As a limitation, it is noted that existing simulators lacked the ability to comprehensively model and simulate these big data systems and SDN networks in cloud environments, a gap that BigDataSDNSim seeks to close.

The work 'CupCarbon: A Multi-Agent and Discrete Event Wireless Sensor Network Design and Simulation Tool' [5] (2023) addresses the lack of simulation tools for wireless sensor networks that allow the creation of realistic models and the evaluation of their performance in complex environments. The methodology is based on the presentation of CupCarbon, a simulation tool that uses multi-agent systems and discrete event simulation to design and simulate wireless sensor networks, integrating with the OpenStreetMap platform to generate realistic models. The main contribution of CupCarbon is its ability to create realistic WSN models and evaluate their performance in complex environments thanks to its integration with OpenStreetMap and its multi-agent approach. However, it

is acknowledged that the tool has limitations in its ability to simulate extremely complex environments and in the exhaustive evaluation of network performance, although it is projected as a valuable research tool.

The work 'IOTSim: A simulator para analizar aplicaciones IoT' [7] (2016) addresses the lack of simulators capable of analyzing IoT applications in cloud computing environments. The methodology employed is based on the use of Cloudsim simulation software to simulate cloud computing resources and, in this way, evaluate the efficiency of IoT applications. The main contribution is the creation of a new simulator that allows analyzing IoT applications in cloud computing environments using the MapReduce model. A persistent limitation in the IoT research and development community is identified as the lack of adequate simulators for this purpose, which this work seeks to mitigate.

The work 'IoTSim-Edge: A Simulation Framework for Modeling the Behaviour of IoT and Edge Computing Environments' [8] (2020) addresses the inability to develop a simulation environment that captures the complexity and heterogeneity of IoT and edge computing environments. The proposed methodology is IoTSim-Edge, a simulation tool that allows users to evaluate edge computing scenarios in a configurable and customizable environment, built on existing simulators to capture the complexity of IoT and edge computing. IoTSim-Edge provides the ability to model various scenarios, including IoT application graphs, heterogeneous protocols, and mobile devices, in addition to supporting cooperative communication between devices for processed data transfer. A critical limitation is that the tool does not simplify the complexities of configuring and customizing the simulation environment, which may require considerable technical knowledge and time for its implementation.

The work 'Modelado de Redes de Sensores en Ptolemy II' [11] (date not specified) addresses the lack of an adequate modeling and simulation framework for sensor networks that allows integrating different computation models, such as continuous, dataflow, synchronous/reactive, and time-activated. The methodology describes an extension of the discrete event (DE) domain of Ptolemy II to model sensor networks, based on the creation of actor-oriented models that allow defining components and their connections

flexibly. The main contribution is this extension of the DE domain of Ptolemy II, which facilitates flexible and scalable modeling of sensor networks, simplifying the creation of complex models. However, a critical limitation is the absence of support for real-time simulation, which restricts its ability to model dynamic systems, and integration with other modeling frameworks can be complex due to the modular nature of the DE domain.

The work 'TOSSIM: Accurate y escalable simulación de aplicaciones de TinyOS' [12] (date not specified) addresses the persistent lack of scalable and accurate simulation tools for the study of sensor network systems. To solve this, the methodology focused on the development of a sensor network simulator capable of managing a large number of nodes and capturing the complexity of their interactions. The main contribution is the creation of TOSSIM, a specific simulator for TinyOS that meets these requirements, demonstrating the viability of developing accurate and scalable simulation tools for this type of system. No critical limitations inherent to the TOSSIM tool are specified in the provided summary; rather, it is highlighted that the work arises to overcome the general limitation of simulation tools in the field.

The work 'Uso del Contiki Cooja Simulator' [13] (date not specified) addresses the problem of establishing RPL networks in virtual environments using the Contiki Cooja Simulator. The detailed methodology in the work consists of using the Contiki Cooja Simulator to configure an RPL network in a virtual environment, providing exhaustive instructions for program compilation, mote type creation, node addition to the network, and RPL bridge configuration. The main contribution is a detailed guide that facilitates researchers and developers in efficiently performing tests and simulations of RPL networks. However, it is acknowledged that the Contiki Cooja Simulator's ability to simulate RPL networks in virtual environments may be limited by network complexity and the need for precise adjustment of configuration parameters to obtain accurate results.

The work 'Viptos: A Graphical Development and Simulation Environment for TinyOS-based Wireless Sensor Networks' [14] (2006) addresses the inability of wireless sensor systems to model and simulate complex hardware and software systems efficiently. The proposed methodology is based on the integration of TinyOS and VisualSense

development and simulation tools to create Viptos, an integrated graphical design and simulation environment for wireless sensor systems. The main contribution is the development of Viptos, a graphical tool that allows WSN developers to create models and simulate complex systems efficiently and securely. The critical limitation that this work seeks to overcome is the lack of integration between existing development and simulation tools, which has historically hindered the efficient creation of complex models and simulations.

II) Model-Driven Development

The work 'A component based approach for the Web of Things' [1] (date not specified) addresses the lack of standards and guidelines for building smart devices, as well as the need for a more robust and reusable architecture for the Web of Things (WoT). The methodology presents a component-based approach for the WoT, with the aim of creating models and tools that guide developers in creating smart devices. The main contribution is the proposal of a customized meta-model for the WoT, which facilitates the creation of reusable and debuggable components, and integrates algorithms and virtualities to improve interoperability and efficiency in building smart devices. A critical limitation is that the absence of consolidated standards and guidelines for the WoT can restrict the reusability and interoperability of the proposed components.

The work 'Desarrollo de Sistemas de Automación del Hogar y Inteligencia Ambiental utilizando Modelado Dirigido' [6] (date not specified) addresses the lack of an adequate modeling tool for developing home automation and ambient intelligence systems, which generates numerous problems in their development and maintenance. The central methodology is Model-Driven Development, applied to create a solution to this problem. The main contribution is the presentation of HAAIS-DSL, a modeling tool that allows software architects to create models of home automation and ambient intelligence systems quickly and efficiently, using model-driven techniques to generate the necessary code and configuration. However, a critical limitation is that the HAAIS-DSL tool may be limited in its ability to handle excessively complex systems and requires a considerable amount of data and technical knowledge for its correct functioning.

The work 'Model Driven Development Applied to Complex Event Processing for Near Real-Time Open Data' [10] (date not specified) addresses the inability of current systems to analyze and process open data in real time. The proposed methodology is based on the use of Model-Driven Development (MDD) and Complex Event Processing (CEP) to raise the level of abstraction in the management and analysis of open data sources. The main contribution is the presentation of a methodology for real-time open data analysis using MDD and CEP, demonstrating its application in two case studies. The main limitation of this research lies in the inherent complexity of real-time open data analysis, which requires a large amount of computational and storage resources.

The work 'IoTSuite: A ToolSuite for Prototyping Internet of Things Applications' [9] (date not specified) addresses the need to enable rapid development of IoT applications with minimal effort from the various stakeholders involved in the process. The methodology focuses on a design and compilation approach to reduce development effort in IoT applications. The main contribution of IoTSuite is to provide a tool suite that includes an editor, compiler, mapping, and execution system, facilitating the creation of IoT applications with less effort. However, a critical limitation is that the tool specifically focuses on Android and JavaSE-enabled devices, uses MQTT middleware, and requires a detailed understanding of the domain and application architecture for effective use.

B.3. Descripción de Tabla Comparativa

Table B.1 presents a detailed comparison between the analyzed works and the SimulateIoT proposal, using a series of key criteria to evaluate their capabilities and approaches. These criteria are:

- **Main Approach:** Describes the fundamental methodology or paradigm underlying the work.
- **Application Domain:** Specifies the primary area of application or the type of system addressed.

- **Design Abstraction Level:** Indicates how the system or simulation design and configuration are performed.
- **MDD/DSL Support:** Indicates whether the work employs a Model-Driven Development (MDD) approach or a Domain-Specific Language (DSL).
- **Automatic Code Generation:** Determines if the work is capable of automatically generating executable code or configurations from a design or model.
- **Heterogeneous IoT Environment Simulation:** Evaluates whether the simulator is designed to model and simulate a wide range of devices, protocols, and layers of the IoT stack.
- **Edge/Cloud Computing Modeling:** Indicates whether the work explicitly includes capabilities for modeling or simulating interaction with Edge or Cloud Computing environments.
- **Graphical Interface for Design/Modeling:** Notes whether the tool provides a graphical user interface for designing or modeling the environment or system.

This comparison aims to highlight the distinctive features of SimulateIoT and position it against the state of the art, providing a clear overview of its technical advantages.

B.4. Conclusión

The analysis of the state of the art reveals a diversity of tools and approaches for simulating wireless sensor networks and IoT environments, as well as for model-driven development of complex systems. While robust simulators exist for evaluating network performance (such as OMNeT++, TOSSIM, or Contiki Cooja) and tools address specific aspects like Big Data, SDN, or Edge/Cloud Computing (e.g., BigDataSDNSim, IOTSim, IoTSim-Edge), many of them require detailed configuration and low-level programming. This complicates the design and experimentation of complex and heterogeneous scenarios without considerable effort. Furthermore, Model-Driven Development (MDD) approaches

have demonstrated their value in home automation or complex event processing, offering high-level abstractions and, in some cases, code generation. However, a significant gap is observed in solutions that effectively integrate high-level IoT simulation environment design through MDD and DSL, with the capability to automatically generate the necessary code for execution, without requiring substantial investment in devices or extensive programming. This gap is precisely what SimulateIoT addresses. Our work, "SimulateIoT: A Model-Driven Approach for Designing, Code Generation, and Execution of IoT Simulation Environments,"stands out due to its Model-Driven Development methodology, which enables users to design complex IoT simulation environments at a high level of abstraction using a Domain-Specific Language (DSL) and its associated metamodel. This approach eliminates the need for manual code writing, facilitating the experimentation and validation of heterogeneous IoT systems before their real-world deployment, and offering a more efficient and accessible solution to the problem of developing and testing IoT systems with limited investments.

B.5. Bibliografía

[1] Andreas Ruppen, Jacques Pasquier, Sonja Meyer, Alexander Rüedlinger. "A component based approach for the Web of Things," No especificado.

[2] Georgios Z. Papadopoulos, Julien Beaudaux, Antoine Gallais, Thomas Noël, Guillaume Schreiner. "Adding value to WSN simulation using the IoT-LAB experimental platform," no especificado.

[3] András Varga, Rudolf Hornig. "AN OVERVIEW OF THE OMNeT++ SIMULATION," 2008.

[4] Khaled Alwasel, Rodrigo N. Calheiros, Saurabh Garg, Rajkumar Buyya, Rajiv Ranjan. "BigDataSDNSim: A Simulator for Analyzing Big Data Applications in Software-Defined Cloud Data Centers," 2019.

[5] Mehdi, Kamal, Lounis, Massinissa, Bounceur, Ahcène, Kechadi, Tahar. "CupCarbon: A Multi-Agent and Discrete Event Wireless Sensor Network Design and Simulation Tool," Diciembre de 2023.

[6] Pedro J. Clemente, José M. Conejero, Juan Hernández, Lara Sánchez. "Desarrollo de Sistemas de Automación del Hogar y Inteligencia Ambiental utilizando Modelado Dirigido," Inciso.

[7] Xuezhi Zeng, Saurabh Kumar Garg, Peter Strazdins, Prem Prakash Jayaraman, Dimitrios Georgakopoulos, Rajiv Ranjan. "IOTSim: A simulator para analizar aplicaciones IoT," This work was done in 2016.

[8] Devki Nandan Jha, Khaled Alwasel, Areeb Alshoshan, Xianghua Huang, Ranesh Kumar Naha, Sudheer Kumar Battula, Saurabh Garg, Deepak Puthal, Philip James, Albert Zomaya, Schahram Dustdar. "IoTSim-Edge: A Simulation Framework for Modeling the Behaviour of IoT and Edge Computing Environments," 2020.

[9] Dimitris Soukaras, Pankesh Patel, Hui Song, Sanjay Chaudhary. "IoTSuite: A ToolSuite for Prototyping Internet of Things Applications," desconocido.

[10] Pedro J. Clemente, Adolfo Lozano-Tello. "Model Driven Development Applied to Complex Event Processing for Near Real-Time Open Data," This work presents a methodology to tackle the analysis of open data sources using Model-Driven Development (MDD) and Complex Event Processing (CEP), which help users to raise the abstraction level utilised to manage and analyse open data sources.

[11] Philip Baldwin, Sanjeev Kohli, Edward A. Lee, Xiaojun Liu, Yang Zhao. "Modelado de Redes de Sensores en Ptolemy II," Información no proporcionada.

[12] Philip Levis, Nelson Lee, Matt Welsh, David Culler. "TOSSIM: Accurate y escalable simulación de aplicaciones de TinyOS," Not especificado.

[13] Anuj Sehgal. "Uso del Contiki Cooja Simulator," No especificado.

[14] Elaine Cheong, Edward A. Lee, Yang Zhao. "Viptos: A Graphical Development and Simulation Environment for TinyOS-based Wireless Sensor Networks," Feb 2006.

B.6. Tabla Comparativa

Tabla B.1: Comparative analysis of related works and SimulateIoT.

Title	Main Approach	Application Domain	Design Abstraction Level	MDD/DSL Support	Automatic Code Generation	Heterogeneous IoT Environment Simulation	Edge/Cloud Computing Modeling	Graphical Interface for Design/Modeling
A component based approach for the Web of Things [1]	Component-based, Meta-modeling	Web of Things (WoT), Smart devices	Component-based, Reusable	Yes	No	No	No	No
Adding value to WSN simulation using the IoT-LAB experimental platform [2]	Experimental platform, WSN	WSN, IoT-LAB platform	Experimental platform configuration	No	No	No	No	No
AN OVERVIEW OF THE OMNeT++ SIMULATION [3]	Discrete event simulation, Simulation framework	Communication networks, Distributed systems	Low-level code, Framework	No	No	No	No	No
BigDataSDNSim: A Simulator for Analyzing Big Data Applications in Software-Defined Cloud Data Centers [4]	Simulation, Big Data, SDN	Big Data, SDN, Cloud Data Centers	Simulator configuration	No	No	No	Yes	No
CupCarbon: A Multi-Agent and Discrete Event Wireless Sensor Network Design and Simulation Tool [5]	Multi-agent simulation, Discrete events	WSN, Geolocation	Model design, Simulator configuration	No	No	No	No	Yes
Desarrollo de Sistemas de Automación del Hogar y Inteligencia Ambiental utilizando Modelado Dirigido [6]	Model-Driven Development (MDD), DSL	Smart Home, Ambient Intelligence	Model-based, DSL, High-level	Yes	Yes	No	No	Yes
IOTSim: A simulator para analizar aplicaciones IoT [7]	Simulation, Cloud Computing	IoT, Cloud Computing	Simulator configuration	No	No	No	Yes	No
IoTSim-Edge: A Simulation Framework for Modeling the Behaviour of IoT and Edge Computing Environments [8]	Simulation, Edge Computing	IoT, Edge Computing	Simulator configuration, Scenario modeling	No	No	Yes	Yes	No
IoTSuite: A ToolSuite for Prototyping Internet of Things Applications [9]	Prototyping, Design and compilation	IoT, Automation, Inventory	Development tools, Configuration	No	No	No	No	Yes
Model Driven Development Applied to Complex Event Processing for Near Real-Time Open Data [10]	MDD, CEP	Open Data, Event analysis	Model-based, High-level	Yes	No	No	No	No
Modelado de Redes de Sensores en Ptolemy II [11]	Modeling, Simulation, Discrete events	WSN	Actor-based, Components, High-level	No	No	No	No	Yes
TOSSIM: Accurate y escalable simulación de aplicaciones de TinyOS [12]	Simulation, WSN	WSN, TinyOS	Low-level code, Simulator configuration	No	No	No	No	No
Uso del Contiki Cooja Simulator [13]	Simulation, WSN	WSN, RPL	Simulator configuration, Low-level code	No	No	No	No	Yes
Viptos: A Graphical Development and Simulation Environment for TinyOS-based Wireless Sensor Networks [14]	Graphical design, Simulation, WSN	WSN, TinyOS	Graphical design, Modeling	No	No	No	No	Yes
SimulateIoT: Un enfoque dirigido por modelos para diseñar, generar código y ejecutar entornos de simulación IoT	Model-Driven Development, Simulation	IoT Simulation, Smart Buildings, Agriculture	Model-based, DSL, High-level, Code-free	Yes	Yes	Yes	No	Yes

Bibliografía

[1] Blagoj Delipetrev, Chrisa Tsinaraki, and Uroš Kostić. Ai watch: Historical evolution of artificial intelligence — analysis of the three main paradigm shifts in ai. Technical Report EUR 30221 EN, Publications Office of the European Union, Luxemburgo, 2020.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[3] Wayne Xin Zhao, Kun Zhou, Junyi Li, et al. A survey of large language models, 2023. arXiv:2303.18223.

[4] Lizhou Fan, Lingyao Li, Zihui Ma, Sanggyu Lee, Huizi Yu, and Libby Hemphill. A bibliometric review of large language models research from 2017 to 2023. *ACM Transactions on Intelligent Systems and Technology*, 15(5), 2024.

[5] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[6] Zhiheng Xi, Wenxiang Chen, Xin Guo, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68, 2025. art. 121101.

[7] Jiaxuan Li, Zhe Jin, Tianyu Men, et al. Agentic environment engineering for large language models: A survey of environment modeling, synthesis, evaluation, and application, 2026. arXiv:2606.12191.

[8] Ziwei Ji, Nayeon Lee, Rita Frieske, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[10] Yidong Wang, Qi Guo, Wenjin Yao, et al. Autosurvey: Large language models can automatically write surveys. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[11] Ang Xie, Wei Su, Jiaxin Mao, Yiqun Liu, Min Zhang, Shaoping Ma, and Qingyao Ai. Rwgbench: Evaluating scholarly positioning in related work generation, 2026. arXiv:2606.24894.

[12] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. Security and privacy challenges of large language models: A survey. *ACM Computing Surveys*, 57(6), 2025. art. 152.

[13] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data. In *International Conference on Learning Representations (ICLR)*, 2025.

[14] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020.

[15] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.

[16] Lilian Weng. Llm-powered autonomous agents, 2023. Lil'Log.

[17] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Qiong Dai, Jieming Zhu, Zhiheng Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. arXiv:2404.13501.

[18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[20] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. From llms to llm-based agents for software engineering: A survey of current, challenges and future, 2024. arXiv:2408.02479.

[21] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Sheng Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges, 2024. arXiv:2402.01680.

[22] Xinzhe Li, Sanwoo Wang, Siyu Zeng, Yawen Wu, and Yan Yang. A survey on llm-based multi-agent systems: Workflow, infrastructure, and challenges. *Vicinagearth*, 1, 2024. art. 9.

[23] Carlos Gañán Caballero. Introducción a ollama y los llms libres. Oficina de Software Libre, Universidad de Granada, 2025. 8 de abril de 2025.

[24] CrewAI, Inc. Crewai documentation, 2025.

[25] LlamaIndex. Llamaindex: A data framework for llm applications. Repositorio GitHub, 2025.

[26] Amazon Web Services. Langchain and langgraph for agentic ai frameworks. AWS Prescriptive Guidance, 2025.

[27] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions, 2025. arXiv:2503.23278.

[28] Tianyi Bao, Mahammed Tanzim Nayeem, Davood Rafiei, and Chen Zhang. Surveygen: Quality-aware scientific survey generation with large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2712–2736, 2025.

[29] Shubham Agarwal, Issam H. Laradji, Laurent Charlin, and Christopher Pal. Litllm: A toolkit for scientific literature review, 2024. arXiv:2402.01788.

[30] Shubham Agarwal, Gaurav Sahu, Abhay Puri, Issam H. Laradji, Krishnamurthy DJ Dvijotham, Jason Stanley, Laurent Charlin, and Christopher Pal. Litllms, llms for literature review: Are we there yet?, 2024. arXiv:2412.15249.

[31] Yutong Li, Lu Chen, Aiwei Liu, Kai Yu, and Lijie Wen. Chatcite: Llm agent with human workflow guidance for comparative literature summary, 2024. arXiv:2403.02574.

[32] Bruno Nogueira, Werner Geyer, Ariful Anderson, Toby J.-J. Li, Dongwoo Kim, Nuno Moniz, and Nitesh V. Chawla. From verification burden to trusted collaboration: Design goals for llm-assisted literature reviews. In *Companion Proceedings of the 31st International Conference on Intelligent User Interfaces (IUI)*, 2026.

[33] Rui Qiu, Shijie Chen, Yu Su, Po-Yin Yen, and Han-Wei Shen. Completing a systematic review in hours instead of months with interactive ai agents, 2025. arXiv:2504.14822.

[34] Srishti Palani, Aakanksha Naik, Doug Downey, Amy X. Zhang, Jonathan Bragg, and Joseph Chee Chang. Relatedly: Scaffolding literature reviews with existing related work sections. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*, 2023.

[35] Wenqi Wei, Jiaxin Zheng, Zhen Wang, and Wenjie Feng. Lr-robot: An human-in-the-loop llm framework for systematic literature reviews with applications in financial research, 2026. arXiv:2604.14793.

[36] IEEE. Ieee recommended practice for software requirements specifications. Technical Report IEEE Std 830-1998, IEEE Computer Society, 1998.

[37] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 3 edition, 2012.

[38] Mehdi Bahrami and Mohsen Bahrami. An overview to software architecture in intrusion detection system. *International Journal of Soft Computing and Software Engineering (JSCSE)*, 1(1), 2011.

[39] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[40] José Ángel Barriga, Pedro José Clemente, Encarna Sosa-Sánchez, and Álvaro E. Prieto. Simulateiot: Domain specific language to design, code generation and execute iot simulation environments. *IEEE Access*, 9:92531–92552, 2021.

[41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.